



GALALA UNIVERSITY

Powered by
Arizona State University®

Faculty of Computer Science and Engineering

Comprehensive Intelligent Scheduling Platform

By

221100246

Bassel Mahmoud Younes

221101546

Ziad Islam Fathy Ali Abdallah

221101073

Mohamed Ashraf Mahmoud Ali

Supervised By

Main: Dr. Nora Niazzy

Co: Dr. Shaker Elsappagh

June 2026

Abstract

The University Course Scheduling Problem (UCSP) is a complex and computationally challenging optimization task that involves assigning courses, instructors, and classrooms under multiple hard and soft constraints. Inefficient scheduling in higher education leads to substantial operational costs, resource underutilization, and conflicts affecting administrators, faculty, and students.

This project aims to design and evaluate an automated course scheduling system that addresses these challenges by integrating Artificial Intelligence (AI) and cyber security mechanisms. A Genetic Algorithm (GA) is employed to generate and optimize feasible timetables by evolving candidate solutions through crossover, mutation, and selection processes, while hybrid approaches using local search techniques refine solutions further. The system considers multiple objectives, including minimizing scheduling conflicts, optimizing room usage, and balancing faculty workload.

To ensure security, the platform incorporates encryption (AES, SSL/TLS), multi-factor authentication, and role-based access control, along with AI-driven anomaly detection for proactive threat prevention. Tested on real and anonymized institutional datasets, the proposed system demonstrates improved scheduling efficiency, reduced manual intervention, enhanced resource utilization, and robust protection of sensitive academic data. By combining AI optimization with cyber security measures, this project provides a secure, flexible, and high-quality solution to the UCSP, suitable for real-world university environments.

Acknowledgement

The completion of this project would not have been possible without the support, guidance, and encouragement of many individuals who have stood by us throughout this journey. We would like to take this opportunity to express our deepest gratitude to everyone who contributed to making this work a reality.

First and foremost, we extend our sincere thanks to our project supervisor, **Dr. Nora Niazi**, for their unwavering support and invaluable guidance throughout every phase of this project. Your expertise in Scheduling and genetic algorithms and your willingness to challenge our thinking pushed us to explore depths we hadn't initially considered. We are particularly grateful for the countless hours you spent reviewing our methodology during the critical redesign phase in the first semester. Your door was always open, even during exam season and the final submission deadline, and your constructive feedback shaped this work in fundamental ways.

We would also like to thank the co-supervisor, **Dr. Shaker Elsappagh** for sharing insights from your industry experience and helping us navigate the literature on the Scheduling topic. Your perspective on the project was instrumental in refining our approach.

Our heartfelt appreciation goes to **CSE faculty** at **Galala University** for providing us with the resources and infrastructure necessary to conduct this research. Special thanks to all the TAs for training us on the equipment and troubleshooting when things went wrong at 2 AM and helping us secure the materials we needed, and to the Library for tracking down those obscure journal articles.

We are deeply indebted to the **Timetable** website and specifically **Oliver Jones**, for providing the internship opportunity that inspired this research. The practical insights we gained from the internship bridged the gap between theory and application in ways textbooks never could.

Table of Contents

Abstract	i
Acknowledgements	ii
Table of Contents	iii
List of Figures	ix
List of Tables	x
Chapter 1 Introduction	1
1.1 Overview	2
1.2 Problem Statement	2
1.3 Motivation	3
1.4 Work Plan	3
Chapter 2 State-of-the-Art	5
2.1 Background and Problem Definition	5
2.1.1 Constraint Classification	5
2.2 Solution Approaches	6
2.2.1 Integer Linear Programming and Mixed-Integer Programming	6
2.2.2 Constraint Programming	7
2.2.3 Heuristic Approaches	7
2.2.3.1 Graph-Coloring Heuristics	7
2.2.3.2 Greedy Assignment Heuristics	8
2.2.4 Metaheuristic Approaches	8
2.2.4.1 Tabu Search (TS)	8
2.2.4.2 Simulated Annealing (SA)	8
2.2.4.3 Iterated Local Search (ILS)	8
2.2.4.4 Genetic Algorithms (GA)	9
2.2.4.5 Ant Colony Optimization (ACO)	9
2.2.4.6 Particle Swarm Optimization (PSO)	9
2.2.5 Hybrid Approaches	9
2.2.5.1 Memetic Algorithms (GA + Local Search)	9
2.2.5.2 Two-Phase Approaches (ILS + SA)	9

2.2.5.3 Algorithm-Cycling	10
2.2.5.4 Adaptive Hybrids	10
2.2.6 Machine Learning and AI Approaches	10
2.2.6.1 Algorithm Selection	10
2.2.6.2 Learning-Guided Metaheuristics	10
2.2.6.3 ML for Tuning and Surrogates	10
2.2.7 Recent Benchmark Results	10
Chapter 3 Proposed Solution Analysis and Design	15
3.1 Solution Methodology	16
3.1.1 SDLC Phases Applied to This Project	16
3.1.1.1 Phase 1: Requirements Gathering (Completed in Chapter 2)	16
3.1.1.2 Phase 2: System Analysis (Current Chapter - Section 3.2)	16
3.1.1.3 Phase 3: System Design (Current Chapter - Sections 3.3-3.7)	16
3.1.1.4 Phase 4: Implementation (Chapter 4)	17
3.1.1.5 Phase 5: Testing and Validation (Chapter 5)	17
3.1.1.6 Phase 6: Deployment and Maintenance (Chapter 6)	17
3.1.2 Implementation Approach Selection	18
3.1.2.1 Evaluated Approaches	18
3.1.2.2 Selected Approach: Web-Based Implementation	19
3.1.3 Software and Hardware Requirements	19
3.2 Functional and Non-Functional Requirements	22
3.2.1 User Requirements	22
3.2.2 System Requirements	25
3.2.3 Non-Functional Requirements	27
3.3 Use Case Modeling	29
3.4 Entity-Relationship Diagram	38
3.5 UML Diagrams	41
3.5.1 Class Diagram	41
3.5.2 Sequence Diagram - Generate Schedule	45
3.5.3 Activity Diagram - Generate Schedule Workflow	47
3.5.4 Component Diagram	52
3.5.5 Deployment Diagram	55

3.6 Constraint Modeling	57
3.6.1 Hard Constraints	58
3.6.1.1 HC1 - No Instructor Double-Booking	58
3.6.1.2 HC2 - No Student Group Double-Booking	58
3.6.1.3 HC3 - No Room Double-Booking	58
3.6.1.4 HC4 - Room Capacity Compliance	59
3.6.1.5 HC5 - Instructor Availability Compliance	59
3.6.1.6 HC6 - Room Type Matching	59
3.6.1.7 HC7 - Fixed Event Non-Overlap	60
3.6.2 Soft Constraints	60
3.6.2.1 SC1 - Workload Balance (Weight: w_1)	60
3.6.2.2 SC2 - Minimize Idle Gaps (Weight: w_2)	60
3.6.2.3 SC3 - Efficient Room Utilization (Weight: w_3)	61
3.6.2.4 SC4 - Time Preference Satisfaction (Weight: w_4)	61
3.6.2.5 SC5 - Schedule Compactness (Weight: w_5)	61
3.6.2.6 SC6 - Room Preference Satisfaction (Weight: w_6)	62
3.6.2.7 SC7 - Day Balance (Weight: w_7)	62
3.6.3 Fitness Function	62
3.6.3.1 Fitness Function Formulation	62
3.6.3.2 Example Calculation	63
Summary	63
Chapter 4 Implementation	67
4.1 Introduction	67
4.2 Core Technology Choices	67
4.2.1 Python as the Foundation	67
4.2.2 DEAP for Genetic Algorithm Operations	67
4.2.3 Dynamic Programming for Classroom Allocation	69
4.2.4 PostgreSQL for Data Storage	70
4.2.5 FastAPI for the Web Interface	70
4.2.6 Celery for Background Processing	70
4.2.7 Sigma Design System for User Interface	70
4.3 Implementation of POGA-DP Algorithm	74

4.4 Two-Phase Scheduling Process	74
4.4.1 Phase 1: Joint Course Scheduling	74
4.4.2 Phase 2: Independent Course Scheduling	74
4.5 Fitness Function Implementation	74
4.5.1 Hard Constraint Violations (heavily penalized)	74
4.5.2 Soft Constraint Optimization (weighted)	74
4.5.3 Genetic Operations with Safety Mechanisms	75
4.6 Progressive Optimization Strategy	76
4.6.1 Early Generations (exploring)	76
4.6.2 Later Generations (refining)	76
4.7 System Integration	76
4.8 Handling Real-World Scheduling Scenarios	77
4.9 Performance and Optimization	77
4.9.1 Computational Efficiency	77
4.9.2 Solution Quality Metrics	77
4.10 Export and Reporting	78
4.11 Testing and Validation	78
4.11.1 Algorithm Validation	78
4.11.2 Performance Validation	78
4.11.3 Integration Testing	79
4.12 Deployment Setup	79
4.12.1 Development Environment	79
4.12.2 Production Environment	79
4.13 Summary	79
Chapter 5 Results	81
5.1 Introduction	82
5.2 Definition of the Performance Metrics	83
5.3 Performance Evaluation Methodology	84
5.3.1 Experimental Environment	84
5.3.2 Benchmark Suite	85
5.3.3 Algorithm Configuration	85
5.3.4 Comparison Baseline	86

5.4 Presentation of Results on Benchmark Instances	86
5.4.1 Overall Validation Results	86
5.4.2 Hard-Constraint Satisfaction	87
5.4.3 Classroom Utilisation	88
5.4.4 GA Convergence Behaviour	89
5.5 Verification and Validation	90
5.5.1 Overview	90
5.5.2 Unit Test Results	90
5.5.3 Integration Test Results	91
5.5.4 Defect Tracking and Resolution	92
5.6 Testing Scenarios and Case Studies	93
5.6.1 Comparative Analysis Across Methods	93
5.6.2 Galala University Real-World Case Study	94
5.7 Non-Functional Requirement Tests	97
5.8 Summary and Explanation of Key Findings	98
Chapter 6 Discussion	101
6.1 Introduction	102
6.2 Analysis and Interpretation of the Results	102
6.2.1 Feasibility and Hard-Constraint Satisfaction	103
6.2.2 Occupancy and Room-Utilisation Trends	105
6.2.3 Convergence and Population Dynamics	106
6.2.4 Comparative Analysis	106
6.2.5 Security and Non-Functional Results	107
6.3 Significance of the Findings	110
6.3.1 Practical Significance	112
6.3.2 Academic and Scientific Significance	112
6.3.3 Significance of Soft-Constraint Analysis	113
6.4 Alignment with Project Objectives and Expected Outcomes	113
6.5 Limitations	115
6.5.1 Overview	115
6.5.2 Student Sectioning	115
6.5.3 Single-Semester Scope	116

6.5.4 Manual Parameter Tuning	116
6.5.5 Deployment and Usability Limitations	117
6.6 Reflection on the Development Process	117
6.7 Comparison with Related Work	118
6.8 Summary	119
Chapter 7 Conclusion	120
References	121

List of Figures

Figure 3.1 Use Case Diagram	29
Figure 3.2 The ER diagram depicts the following key entities and relationships:	39
Figure 3.3 Class Diagram	42
Figure 3.4 Sequence Diagram for Generate Schedule Use Case	45
Figure 3.5 Activity Diagram for Schedule Generation Workflow	49
Figure 3.6 Component Diagram	52
Figure 3.7 Deployment Diagram	55
Figure 4.1 POGA-DP Chromosome Structure	68
Figure 4.2 Chromosome Data Structure Implementation	68
Figure 4.3 Swap Operation with Judgment Mechanism	69
Figure 4.4 Dynamic Programming Classroom Allocation Process	69
Figures 4.5-4.7 Main Dashboard Interface	72
Figures 4.8 - 4.10 Schedule Visualization	73
Figure 4.11 Fitness Calculation Component	75
Figure 5.1 Our Project shows it finds feasible path	88
Figure 5.2 Classroom Occupancy (%) vs. Instance Number	90
Figure 5.3 GA Convergence Curve for Instance 15	91
Figure 5.4 Feasibility Rate (%) and Runtime (s) for Instance 15	97
Figure 5.5 Average Classroom Occupancy (%) by method and instance group	98
Figure 5.6 Schedule in action · Galala University instance	100
Figure 5.7 Soft-penalty breakdown · Galala University instance	101
Figure 5.8 Backend test line coverage (%) by module · NFR3 threshold = 60%	102.

List of Tables

Table 2.1 Comparison of UCSP Solution Approaches	11
UC1 User Login	30
UC2 Import Course Data	31
UC3 Define Instructor Availability	31
UC4 Generate Schedule	32
UC5 View Schedule - Single Entity	33
UC6 View Schedule - Comprehensive	34
UC7 Manually Adjust Schedule	35
UC8 Compare Schedule Versions	36
UC9 Export Schedule	37
UC10 Receive Schedule Notification	38
Table 5.1 Performance Metrics and Their Definitions	85
Table 5.2 POGA-DP Validation Results (15 of 15 instances solved feasibly; median of five runs)	88
Table 5.3 GA Convergence for Selected Instances	90
Table 5.4 Unit Test Results (30 of 30 passed)	91
Table 5.5 Integration Test Results (12 of 12 passed)	94
Table 5.6 Defect Log (8 defects, all resolved)	95
Table 5.7 Comparative Performance Across Methods	97
Table 5.8 Galala University Case Study Results	99
Table 5.9 Non-Functional Requirement Tests Results (10 of 10 passed)	101
Table 6.1 Alignment of Results with Project Objectives	113
Table 6.2 Known Limitations and Future Development Directions	115

Chapter 1

Introduction

1 Introduction

1.1 Overview

The University Course Scheduling Problem (UCSP) is a complex optimization challenge that requires allocating courses, instructors, classrooms, and time slots while adhering to numerous institutional constraints. Efficient scheduling is critical, as poor timetables can lead to significant resource underutilization, increased operational costs, and additional manual workload for administrators. UCSP is recognized as an NP-hard problem, meaning that finding optimal solutions for large institutions is computationally challenging.

This project aims to design and develop an automated course scheduling system capable of generating feasible timetables that satisfy both hard constraints, such as classroom capacity and instructor availability, and soft constraints, such as instructor preferences. The system will also handle joint courses shared across departments and detect scheduling conflicts to ensure feasible and conflict-free schedules.

A key focus of the project is the integration of basic security mechanisms to protect sensitive scheduling data and restrict unauthorized access. The system will be evaluated using real or anonymized datasets from GU and potentially other educational institutions, making it adaptable to various academic environments. University administrators are the primary users, while faculty and students benefit indirectly from more organized and reliable schedules.

The project is constrained to educational institutions, primarily GU, will be developed using free and open-source tools, and will operate exclusively in English. By addressing both the optimization and security challenges of UCSP, the project seeks to improve scheduling efficiency, reduce manual intervention, and enhance overall resource utilization in higher education institutions.

1.2 Problem Statement

The UCSP is classified as a computationally hard (NP-hard) optimization problem [5], indicating that finding an optimal solution requires substantial computational resources as problem size increases. The constraints of UCSP are commonly divided into hard and soft constraints. Hard

constraints must be strictly satisfied for a timetable to be feasible and include institutional curriculum policies, classroom capacity and equipment limitations, and the availability of instructors and students. Soft constraints represent desirable conditions that may be violated at the expense of solution quality, such as instructor timetable preferences, student course expectations, and institution-specific administrative requirements

1.3 Motivation

Many existing university course scheduling systems suffer from security weaknesses, including insufficient access control and inadequate protection of sensitive student and faculty data [7]. In addition, current solutions often fail to properly support joint and inter-departmental courses, leading to scheduling conflicts and increased reliance on manual intervention [8]. These limitations affect all institutional stakeholders, including administrators, instructors, and students. Improving scheduling systems can therefore lead to lower operational costs, reduced manual workload, and more efficient use of academic resources [9].

1.4 Work Plan

Topic/Task	Proposed time period
Learn USCP	1 week
Requirements Analysis	1 week
System Design	1 week
Modeling Constraints	1 week
Algorithm Development	2 weeks
Evaluation & Performance Analysis	2 weeks

Chapter 2

State-of-The-Art

2 State-of-The-Art

2.1 Background and Problem Definition

The University Course Scheduling Problem (UCSP) represents a complex combinatorial optimization challenge that educational institutions worldwide confront during each academic term. This problem involves the systematic allocation of courses, instructors, student groups, classrooms, and time slots while satisfying institutional constraints and policies. The primary objective is to generate timetables that are both feasible and efficient, ensuring no conflicts occur where students or instructors are scheduled simultaneously in multiple locations, classroom capacities are respected, and mandatory academic requirements are fulfilled [1].

Due to the exponential growth of possible combinations relative to institution size, UCSP is classified as NP-complete, rendering it computationally intensive to find optimal solutions for large universities [2]. Academic weeks typically span five to six working days, subdivided into time slots ranging from one to two hours. Courses possess attributes including credit hours, theoretical and practical components, section counts, and weekly frequency. Instructors introduce additional complexity through specific teaching loads, subject expertise, availability constraints, and personal preferences. Classrooms must be matched to course requirements based on seating capacity, location, and facilities such as laboratories or multimedia equipment. Furthermore, curricula define coherent course groups taken by student cohorts, necessitating conflict-free scheduling within programs. Universities also host special events occupying fixed time slots and rooms, further constraining the scheduling space [3].

2.1.1 Constraint Classification

Constraints in UCSP are categorized into hard and soft constraints. Hard constraints represent mandatory rules that cannot be violated, including preventing overlapping courses, respecting room capacities, and adhering to instructor availability. Soft constraints reflect desirable conditions such as balanced instructor workloads, minimized gaps between classes, preferred room scheduling, and even course distribution across the week. While violating soft constraints does not render a timetable infeasible, it may reduce overall schedule quality [4].

Modern UCSP systems must also address security considerations. Scheduling systems store sensitive information including instructor availability, student group compositions, and internal

resource data. Protecting this information requires proper authentication, role-based access control, encryption, and data integrity mechanisms. Unauthorized schedule modifications can lead to conflicts, misallocations, or disruption of academic operations. System availability is particularly critical during registration periods, examinations, or other high-demand intervals [5].

2.2 Solution Approaches

2.2.1 Integer Linear Programming and Mixed-Integer Programming

Integer Linear Programming (ILP) and Mixed-Integer Programming (MIP) formulations encode course-timeslot-room assignments as binary decision variables with linear constraints for hard requirements such as student conflicts, room conflicts, and capacity limits. The objective function minimizes a weighted sum of soft-violation penalties [6].

Rappos et al. [7] presented a MIP approach for the International Timetabling Competition 2019 (ITC-2019), achieving second place in the competition. Their method implemented a two-stage solution strategy: the first stage focused on obtaining an initial feasible solution, while the second stage employed local search procedures to iteratively improve solution quality. The implementation utilized IBM CPLEX 12.8 and Gurobi 8.11 solvers on a 32 GB, 4-core Linux machine, achieving feasible timetables within 1-2 hours for most instances. However, they noted the approach "solved 29 of 30 benchmark instances," with one instance failing to fit into 256 GB of RAM, demonstrating scalability challenges for extremely large or complex problems [7], [8].

Advantages: When convergent, ILP yields provably optimal or near-optimal timetables. The approach flexibly models complex constraints through assignment variables, logical constraints, and multiple objectives. Commercial solvers such as CPLEX and Gurobi, along with open-source alternatives like GLPK, handle substantial computational burden [9].

Limitations: Very high computational cost characterizes large instances. Model size grows rapidly, potentially reaching millions of rows and columns after preprocessing. Solvers can require impractically long execution times on tightly constrained datasets. Scalability remains poor, as Rappos et al. observed that their approach "works well for small/medium problems but poorly when the number of students was large" [7]. Crafting efficient ILP models also demands careful variable elimination and warm-start strategies [10].

2.2.2 Constraint Programming

Constraint Programming (CP) approaches treat timetabling as a constraint-satisfaction problem where hard constraints are enforced exactly, and soft constraints are encoded via penalty variables. Modern CP solvers like Google OR-Tools CP-SAT employ propagation techniques to prune the search space [11]. Moreira and Freitas [36] developed a CP-SAT model using Google OR-Tools for scheduling courses and resources at a public university, reporting that the model "produced an optimized solution in all analyzed cases," demonstrating scalability to real institutional data. The CP approach often finds feasible schedules faster than ILP while elegantly expressing rich institutional rules [36].

Advantages: CP handles complex hard constraints elegantly, including sequencing and resource limits. It quickly finds feasible timetables via propagation and benefits from good software support through tools like OR-Tools, IBM CP Optimizer, and others with user-friendly APIs [12].

Limitations: CP remains exponential-time in general cases. Large or highly constrained timetabling instances can exhaust search time and memory without proving optimality. Like ILP, guaranteeing global optimality requires potentially massive search efforts [13].

2.2.3 Heuristic Approaches

2.2.3.1 Graph-Coloring Heuristics

Graph-coloring heuristics model courses as graph vertices with edges representing student or instructor conflicts, then assign timeslots (colors) greedily. Basic coloring heuristics such as largest-degree-first and saturation-degree-first rapidly produce conflict-free schedules [14].

Hoshino and Fabris employed graph-coloring to bundle single-section courses before feeding remaining courses into an ILP solver, finding that this two-stage scheme "reduced solve time drastically while solution quality only deteriorates marginally" [15].

Advantages: Extremely fast and simple implementation. Guaranteed to produce conflict-free schedules when solutions exist. Scales to very large universities through simple iterative loops without requiring specialized solvers [16].

Limitations: Ignores most soft preferences, resulting in initial schedules that typically violate many soft constraints. Solution quality heavily depends on heuristic ordering. Output is usually far from optimal without subsequent improvement phases [17].

2.2.3.2 Greedy Assignment Heuristics

Greedy algorithms sequentially place courses in slots by simple priority rules (e.g., largest-enrollment-first or most-constrained-first), choosing available slots that minimize conflicts at each step. These algorithms run virtually instantaneously and are frequently used to generate starting timetables or provide warm-starts for optimization [18].

Advantages: Trivial to implement and produces complete timetables in microseconds [19].

Limitations: Highly myopic with no backtracking capability. Resulting schedules usually exhibit poor soft-constraint satisfaction compared to global search methods, necessitating post-improvement phases [20].

2.2.4 Metaheuristic Approaches

Metaheuristics iteratively improve timetables through neighborhood moves or population-based search strategies. Common approaches for course timetabling include:

Tabu Search (TS): Explores solution neighborhoods (e.g., swapping timeslots of two lectures) while maintaining a tabu list to forbid recent moves and avoid cycles. TS-based timetablers have achieved state-of-the-art results, rapidly finding high-quality solutions and escaping local minima. Parameter tuning (tabu tenure, move definitions) is critical. In practice, TS refinements substantially outscore simple heuristics, though at the cost of longer runtime [21].

Simulated Annealing (SA): Performs local moves while probabilistically accepting uphill moves based on a cooling schedule. Well-engineered SA solvers often match TS in solution quality. SA is simpler (no tabu list) but typically requires careful temperature tuning and longer runs to converge. Sylejmani et al. [22] developed an SA-based solver for ITC-2019 that detected local optima and applied gradual penalization to force solutions into new neighborhoods, finding feasible solutions for all competition instances.

Iterated Local Search (ILS): Applies local search (e.g., steepest descent) to reach local optima, then perturbs solutions (random swaps or reassignments) and repeats. Lindahl et al. [23] reported an ILS approach that found feasible solutions for a substantial number of standard benchmark instances. ILS is generally easier to tune than TS/SA, though it can still become trapped if perturbations are too weak [23].

Genetic Algorithms (GA): Maintain populations of timetables encoded as assignment strings, applying crossover and mutation to generate offspring. GAs explore diverse solution regions and can handle multi-objective selection. However, pure GAs can be slow to converge and often require local improvement steps [24].

Ant Colony Optimization (ACO): Simulates ant colonies that probabilistically build timetables guided by pheromone trails stored as biases on course-time choices. After each iteration, trails are reinforced for good solutions. ACO can learn exploitable patterns and achieve top results when combined with local search, though it incurs high CPU usage and sensitivity to evaporation and intensification parameters [25].

Particle Swarm Optimization (PSO): Treats candidate timetables as particles moving in solution space, influenced by their own and neighbors' best solutions. PSO can converge quickly with simple update rules, though it risks premature convergence and requires careful adaptation to discrete timetabling moves [26].

2.2.5 Hybrid Approaches

Hybrid algorithms combine multiple techniques to balance exploration and exploitation:

Memetic Algorithms (GA + Local Search): Genetic algorithms augmented by local-improvement steps on each offspring typically outperform pure GAs or local search alone. Dunke and Nickel [27] developed a matheuristic implementing a genetic algorithm to evolve lecture and tutorial plans, employing an artificial neural network (ANN) metamodel to quickly estimate each timetable's penalty. This hybrid approach produced high-quality schedules on multi-criteria benchmarks, effectively creating Pareto-optimal timetables by combining GA-guided search with embedded optimization solves assisted by ANN surrogates [27].

Two-Phase Approaches (ILS + SA): Use one method to satisfy hard constraints, then another to optimize soft criteria. Goh et al. [28] first generated conflict-free schedules via TS/ILS, then refined them using reheating SA. They incorporated reinforcement learning to dynamically adjust search phases, balancing feasibility and optimization phases to speed convergence to high-quality timetables. Their Simulated Annealing with Improved Reheating and Learning (SAIRL) approach achieved new best results for multiple instances and demonstrated scalability when runtime was extended [28].

Algorithm-Cycling: Sequentially applies different heuristics (e.g., running SA, then hill-climbing, then SA again). Studies have shown that alternating SA, Great Deluge, and hill-climbing on initial graph-coloring solutions yields better schedules than any single method alone, though determining effective sequences and time allocations for each component remains challenging [29].

Adaptive Hybrids: More advanced hybrids adapt their strategy dynamically. Adaptive Tabu Search frameworks might switch between intensification (short-term TS) and diversification (long-term ILS) based on progress. Such adaptive methods have produced record solutions in competitions, though at the cost of complex control logic and numerous parameters [30].

2.2.6 Machine Learning and AI Approaches

Machine learning is increasingly employed to enhance timetabling:

Algorithm Selection: Training ML models (e.g., decision trees, SVM) on instance features to predict the best algorithm for each case can reduce solve time by selecting the most suitable solver. While this is an emerging area in timetabling, related scheduling research demonstrates improved solver performance on diverse instances [31].

Learning-Guided Metaheuristics: Use learning to guide search moves or parameters. Goh et al. [28] embedded a reinforcement learning agent to choose among different neighborhood moves during their SA/TS search, effectively learning which moves lead to better improvements and automatically balancing exploration and exploitation [28].

ML for Tuning and Surrogates: ML can set algorithm parameters or approximate objective evaluations. Dunke and Nickel [27] trained an artificial neural network as a surrogate model to quickly predict timetable penalties, accelerating their GA search. Other work has trained models to set weightings of soft constraints or SA cooling schedules, reducing manual tuning effort and improving average solver performance [27].

2.2.7 Recent Benchmark Results

The International Timetabling Competition 2019 (ITC-2019) provided comprehensive benchmarks for evaluating solution approaches [8]. Müller et al. [8] discussed the organization and

characteristics of thirty competition instances with diverse features. The competition's key novelty lay in combining student sectioning with standard time and room assignment of course events.

Top-performing solvers connected graph theory and mathematical programming, building metaheuristics based on initial solution construction, fix-and-optimize enhancements, and bounding procedures [32], [33].

Ceschia et al. [34] provided a comprehensive survey of educational timetabling problems, benchmarks, and state-of-the-art results. Their analysis revealed that single solution-based metaheuristics, particularly Simulated Annealing and Tabu Search, showed the most promise in addressing timetabling problems across multiple benchmarks [34]. Recent work has also explored the integration of large language models for parsing natural language preferences into constraint formulations [35].

Table 2.1 Comparison of UCSP Solution Approaches

Method	Time Complexity	Solution Quality	Scalability	Advantages	Limitations	Key References
Integer Linear Programming (ILP/MIP)	Exponential (NP-complete)	Optimal or near-optimal with proven bounds	Poor for large instances (>1000 courses)	• Provably optimal solutions • Flexible constraint modeling	• Very high computational cost • Memory intensive (millions of variables) • Poor scalability with problem size • Requires expert modeling	[7], [8], [10]
				• Multiple objectives support • Commercial solver support		
Constraint Programming (CP)	Exponential (CSP-complete)	High quality, feasibility-focused	Moderate (better than ILP)	• Elegant constraint handling • Fast feasibility detection • Good propagation techniques • User-friendly APIs	• No optimality guarantees • Can exhaust memory on large instances • Requires careful modeling • Limited soft constraint handling	[11], [12], [42]

Graph Coloring Heuristics	Polynomial $O(n^2)$	Low to medium (conflict-free only)	Excellent (handles very large instances)	• Extremely fast execution • Simple implementation • Guaranteed conflict-free • No special solvers needed	• Ignores soft constraints • Far from optimal initially • Quality depends on ordering • Requires post-optimization	[14], [15], [16]
Greedy Assignment	Polynomial $O(n \cdot m)$	Low (myopic decisions)	Excellent	• Instant execution (microseconds) • Trivial to implement • Good warm-start provider • Minimal memory usage	• Highly myopic (no backtracking) • Poor soft constraint satisfaction • Easily stuck in poor configurations • Always needs refinement	[18], [19], [20]
Tabu Search (TS)	Depends on iterations (typically polynomial per iteration)	High (state-of-the-art results)	Good to excellent	• Escapes local minima effectively • Rapid improvement • High-quality solutions • Many successful implementations	• Requires parameter tuning (tenure) • Longer runtime than heuristics • No optimality guarantees • Complex move definitions	[21], [28]
Simulated Annealing (SA)	Depends on cooling schedule	High (competitive with TS)	Good	• Simpler than TS (no tabu list) • Global	• Requires careful temperature tuning	[22], [28], [29]
				• exploration capability • Well-studied convergence • Effective escape from local optima	• Slow convergence • Many iterations needed • Cooling schedule critical	

Iterated Local Search (ILS)	Polynomial per iteration	High (fast feasibility + improvement)	Good to excellent	• Fast feasible solutions • Systematic improvement • Easier to tune than TS/SA • Good balance of speed and quality	• Can get trapped with weak perturbations • Quality depends on local search • No optimality guarantees • Multiple restarts may be needed	[23], [28]
Genetic Algorithms (GA)	Depends on population and generations	Medium to high (with local search)	Moderate	• Explores diverse regions • Multi-objective handling • Population-based diversity • Natural parallelization	• Slow convergence (pure GA) • Requires local search augmentation • Complex encoding/crossover • Many parameters to tune	[24], [27]
Ant Colony Optimization (ACO)	Depends on ants and iterations	High (with local search)	Moderate	• Learns exploitable patterns • Pheromone-guided construction • Top results on some benchmarks • Distributed nature	• High CPU usage (many ants) • Sensitive to parameters • Requires many iterations • Complex pheromone management	[25]
Particle Swarm Optimization (PSO)	Depends on particles and iterations	Medium to high	Moderate	• Quick convergence • Simple update rules • Good for continuous problems • Swarm intelligence benefits	• Premature convergence risk • Difficult discrete adaptation • Loss of diversity issues • Limited success on hard instances	[26]
Hybrid: Memetic (GA + Local Search)	High (combines multiple methods)	Very high (Pareto-optimal possible)	Moderate to good	• Best of both worlds • Superior to pure GA or LS •	• Complex implementation • Higher computational cost	[27]

				Multi-criteria optimization • ANN surrogates speed evaluation	• Many components to tune • Requires expertise	
Hybrid: Two-Phase (ILS + SA)	Combined complexity	Very high (state-of-the-art)	Good	• Separates feasibility and optimization • RL-guided phase switching • New best results achieved • Balanced approach	• Complex control logic • Phase transition management • Higher development effort • Multiple parameters	[28]
ML-Enhanced (Algorithm Selection)	Depends on selected algorithm	High (chooses best method)	Variable	• Automatic method selection • Reduces solve time • Learns from features • Adapts to instance types	• Requires training data • Overhead for feature extraction • Emerging area in timetabling • ML model complexity	[31]
ML-Enhanced (RL-Guided Search)	Depends on base algorithm	Very high (learns effective moves)	Good	• Automatic strategy learning • Balances exploration/exploitation • Outperforms static strategies • Continuously improves	• Training overhead • Requires reward engineering • State representation on challenges • Black-box nature	[28]
ML-Enhanced (Surrogate Models)	Lower (fast evaluation)	High (speeds up optimization)	Good to excellent	• Fast fitness evaluation • Accelerates GA/metaheuristics • Reduces computational burden • Enables more iterations	• Approximation errors • Training data requirements • Model accuracy critical • Additional complexity	[27]

Chapter 3

Proposed solution analysis and design

3.1 Solution Methodology

The development of the University Course Scheduling System follows the Software Development Life Cycle (SDLC), a structured approach to software development that ensures systematic planning, analysis, design, implementation, and testing. The SDLC provides a framework for organizing the development process into distinct phases, each with specific objectives and deliverables.

SDLC Phases Applied to This Project

Phase 1: Requirements Gathering (Completed in Chapter 2)

This phase involved collecting comprehensive information about the scheduling problem domain. Activities included conducting interviews with scheduling administrators to understand current processes and pain points, surveying instructors to identify availability constraints and preferences, analyzing existing scheduling practices and historical timetable data, documenting institutional constraints including room capacities, course requirements, and curriculum structures, and identifying stakeholder needs across different user groups (administrators, instructors, students). The requirements gathering phase established a complete understanding of the problem context and formed the foundation for subsequent analysis and design decisions.

Phase 2: System Analysis (Current Chapter - Section 3.2)

The analysis phase transforms gathered information into structured requirements specifications. This involves identifying and documenting functional requirements that define what the system must do, specifying non-functional requirements that describe quality attributes the system must possess, analyzing constraints and dependencies between different system components, and evaluating feasibility of different implementation approaches. The analysis ensures that all stakeholder needs are properly understood and translated into technical specifications that guide the design phase.

Phase 3: System Design (Current Chapter - Sections 3.3-3.7)

The design phase translates requirements into a technical blueprint for implementation. Key design activities include defining the overall system architecture using a three-tier pattern with

clear separation of concerns, designing the database schema with Entity-Relationship diagrams to model data structures and relationships, creating UML diagrams to visualize system structure, behavior, and interactions, specifying the genetic algorithm approach for schedule optimization including chromosome encoding and fitness function design, and designing user interfaces with multiple views and interaction patterns. The design phase produces detailed specifications that developers can follow during implementation.

Phase 4: Implementation (Chapter 4)

The implementation phase involves actual coding of the system according to design specifications. This includes backend development using Python with Flask or FastAPI for REST API implementation, frontend development using React with TypeScript for the user interface, database implementation with PostgreSQL for persistent data storage, optimization engine implementation using genetic algorithms with DEAP or PyGAD libraries, and integration of all components into a cohesive working system. Implementation produces executable software that realizes the designed solution.

Phase 5: Testing and Validation (Chapter 5)

The testing phase verifies that the implemented system meets all specified requirements and functions correctly. Testing activities include unit testing of individual components and functions, integration testing to verify component interactions, system testing to validate end-to-end functionality, user acceptance testing with actual stakeholders to confirm the system meets their needs, and performance testing to ensure the system operates within acceptable time constraints. Testing identifies and facilitates correction of defects before deployment.

Phase 6: Deployment and Maintenance (Chapter 6)

The final phase involves deploying the system to the production environment and providing ongoing support. Activities include installation and configuration of the system in the target environment, user training to ensure effective system utilization, monitoring system performance and addressing issues as they arise, and implementing enhancements based on user feedback and changing requirements. Maintenance ensures the system continues to provide value over its operational lifetime.

Implementation Approach Selection

Several implementation approaches were evaluated to determine the most suitable architecture for the University Course Scheduling System. The evaluation considered factors including cost constraints, required computational capabilities, need for multi-user access, and maintenance requirements.

Evaluated Approaches:

Web-Based Implementation: A browser-based application where the system runs on a central server and users access it through web browsers. This approach requires only a web server and database, allows simultaneous access by multiple users from any location with internet connectivity, enables centralized data management and updates, requires no installation on client machines, and facilitates easy maintenance and updates on the server side.

Cloud-Based Implementation: Deployment on cloud infrastructure services such as AWS, Azure, or Google Cloud Platform. While this approach offers scalability and high availability, it incurs ongoing operational costs for hosting, storage, and data transfer. Given the budget constraints of this academic project and the limited user base within a single institution, cloud deployment represents an unnecessary expense compared to on-premises hosting.

Mobile Application: Native or hybrid mobile applications for iOS and Android platforms. This approach requires developing and maintaining separate codebases for multiple platforms, provides limited screen real estate for displaying complex timetable grids, and offers no significant advantage over responsive web design for this use case. The scheduling task involves extensive data entry and visualization that is better suited to larger screens.

Desktop Application: Standalone software installed on individual computers. This approach requires installation and updates on each user's machine, complicates multi-user access and data synchronization, and limits accessibility to specific machines where the software is installed. Desktop applications are less suitable for collaborative scheduling scenarios where multiple administrators need to work with shared data.

Hybrid Implementation: Combining web and mobile elements using frameworks like Electron or React Native. While hybrid approaches can provide cross-platform compatibility, they add

complexity without addressing the core requirements of this project. The additional development effort for mobile support is not justified by the use cases.

Embedded Systems Implementation: Software running on specialized hardware devices. This approach is entirely unsuitable for a scheduling application that requires significant computational power for optimization algorithms, substantial data storage capabilities, and flexible user interfaces. Embedded systems are designed for resource-constrained environments with specific control tasks, not general-purpose business applications.

Selected Approach: Web-Based Implementation

After evaluating all alternatives, the web-based implementation approach was selected as optimal for this project for the following reasons:

Cost-effectiveness is achieved through use of open-source technologies (Python, Flask/FastAPI, PostgreSQL, React) with zero licensing fees, deployment on standard server hardware or institutional infrastructure, and no costs for cloud services or mobile app store distribution. The approach provides universal accessibility where any user with a web browser can access the system from desktop, laptop, or tablet devices without installing specialized software. Centralized management enables all users to work with the same data in real-time, updates and bug fixes can be deployed instantly to all users, and a single codebase simplifies maintenance and reduces development costs. Adequate computational resources are provided by server-side execution that can leverage modern multi-core processors for genetic algorithm optimization, while client-side browsers handle only presentation logic. Scalability supports easy addition of application servers behind load balancers if user base grows, and database optimization and caching strategies can handle increased data volumes. Standard web technologies including HTML5, CSS3, JavaScript, and RESTful APIs ensure long-term maintainability and availability of developer talent.

The web-based architecture aligns perfectly with the project requirements, providing a cost-effective, accessible, and maintainable solution for the university course scheduling problem.

3.1.1 Software and Hardware Requirements

The system utilizes modern, open-source technologies selected for reliability, maintainability, and zero licensing costs. The technology stack is organized into several functional layers.

Backend Technologies

The backend implements core business logic, data management, and the optimization engine. **Python 3.10+** serves as the primary programming language, chosen for its extensive libraries for numerical computing and optimization, clear syntax that facilitates code maintenance, strong community support and documentation, and excellent integration with database and web frameworks.

Flask 3.0 or **FastAPI 0.104+** provides the web framework for building RESTful APIs. Flask offers simplicity and flexibility for routing, request handling, and response generation with extensive ecosystem of extensions. FastAPI provides automatic API documentation, built-in data validation using Pydantic models, and native support for asynchronous programming.

SQLAlchemy 2.0+ acts as the Object-Relational Mapping (ORM) layer, abstracting database operations into Python objects, providing database-agnostic code that can work with multiple database backends, handling relationship mappings and query generation, and supporting migrations for schema evolution.

Celery 5.3+ with **Redis 7.0+** manages asynchronous task execution. Celery is a distributed task queue that executes long-running schedule generation jobs in background worker processes without blocking the API server, while Redis serves as the message broker for task distribution and result storage.

Authentication and authorization are handled using **Flask-Login** for session-based authentication with secure session management and user state tracking, **bcrypt** for password hashing with adaptive cost factors resistant to brute-force attacks, and **PyJWT** for token-based authentication supporting stateless API authentication and mobile client integration.

Optimization and Algorithm Libraries

The genetic algorithm implementation leverages specialized libraries. **DEAP (Distributed Evolutionary Algorithms in Python)** or **PyGAD** provides genetic algorithm frameworks with customizable genetic operators, fitness evaluation, and population management. **OR-Tools** from Google offers constraint programming solvers for validating hard constraints and feasibility checking. **NumPy 1.24+** provides efficient numerical computing with multi-dimensional array operations and mathematical functions, while **Pandas 2.0+** enables data manipulation and analysis with DataFrame structures for tabular data and CSV/Excel import capabilities.

Frontend Technologies

The user interface is built using modern JavaScript frameworks. **React 18+** with **TypeScript** creates component-based UIs with strong typing and compile-time error checking, efficient rendering with virtual DOM, and rich ecosystem of libraries and tools.

State management is handled by **Redux Toolkit** or **Zustand** for centralized application state management, predictable state updates, and debugging capabilities. UI component libraries including **Material-UI (MUI) 5+** or **Ant Design 5+** provide pre-built, customizable components following modern design principles that accelerate development.

FullCalendar or **react-big-calendar** implements timetable visualization with interactive calendar views, drag-and-drop schedule editing, and multiple display modes. **Axios** manages API communication with promise-based HTTP client and request/response interceptors.

Database Technology

PostgreSQL 15+ serves as the relational database management system, providing ACID compliance for data integrity, support for complex queries with joins and aggregations, JSON/JSONB data types for flexible schema elements, robust indexing and query optimization, and point-in-time recovery for backups.

DevOps and Infrastructure

Development and deployment infrastructure includes **Git** with **GitHub** or **GitLab** for version control and collaboration with branching strategies and code review workflows. **Docker 24+** and **Docker Compose** provide containerization for consistent deployment across environments and simplified dependency management. **Nginx 1.24+** acts as reverse proxy and web server for SSL/TLS termination, static file serving, and load balancing. **Gunicorn** or **uWSGI** serves as the WSGI server for running Python web applications with multiple worker processes.

Optional monitoring can be implemented using **Prometheus** for metrics collection and **Grafana** for visualization dashboards.

Development and Testing Tools

The development environment includes **Visual Studio Code** or **PyCharm** as integrated

development environments with Python debugging support and extensions. **Postman** or **Insomnia** enable API testing with request builders and response inspection. **pgAdmin 4** or **DBeaver** provide database management with query editors and schema visualization. **pytest** implements backend testing with fixtures and test discovery, while **Jest** handles frontend testing with component testing and snapshot capabilities.

Reporting and Export

Document generation is supported by **ReportLab** or **WeasyPrint** for PDF generation, **openpyxl** or **xlsxwriter** for Excel file creation, and the **ics library** for iCalendar format exports.

Hardware Requirements

The system is designed to run efficiently on modern hardware with modest specifications. Development and testing can be performed on an average laptop with a **quad-core processor (Intel i5/i7 or AMD Ryzen 5/7)** providing sufficient parallel processing for genetic algorithm populations, **16 GB RAM** supporting concurrent execution of database, application server, and optimization workers, **512 GB SSD storage** offering fast disk I/O for database operations, and a **stable internet connection** for accessing external documentation and downloading dependencies.

For production deployment, a server with similar or better specifications is recommended, with consideration for additional resources as user base and data volume grow. The modular architecture allows scaling by adding worker nodes for optimization tasks or application servers for handling increased user traffic.

3.2 Functional and Non-Functional Requirements

Requirements analysis defines what the system must do (functional requirements) and what quality attributes it must possess (non-functional requirements). These requirements form the contractual specification between stakeholders and developers, and serve as the basis for testing and validation.

3.2.1 User Requirements

User requirements describe what end users need to accomplish using the system, expressed in non-technical terms that stakeholders can understand and validate.

UR1 - User Authentication and Access Control

Users must be able to securely access the system through a login mechanism. The system shall provide account registration functionality where new users can create accounts with unique usernames and secure passwords. Login functionality shall authenticate users and establish secure sessions. The system shall implement role-based access control to ensure users only access features and data appropriate to their roles (Administrator, Instructor, Student, Department Manager). Users shall be able to update their profiles and change passwords through self-service interfaces.

UR2 - Data Entry and Management

Administrators must be able to enter and manage institutional data required for schedule generation. The system shall provide forms and interfaces for entering course information including course codes, names, credit hours, lecture or lab types, and weekly frequency requirements. Room inventory can be entered including room numbers, capacities, types (classroom or laboratory), and equipment. Instructor information can be managed including names, departments, teaching expertise, and maximum teaching load constraints. Curriculum structures can be defined with required courses for each program and student cohort sizes. Time slot configurations can be specified with working days and daily time periods. The system shall support both manual data entry through web forms and bulk import from CSV or Excel files to facilitate efficient data population from existing sources.

UR3 - Multiple Levels of Schedule Display

Users must be able to view schedules at different levels of detail and scope according to their needs. The system shall provide single-entity views showing the complete schedule for one specific room, one specific instructor, or one specific student cohort. Comprehensive views shall display schedules for all rooms simultaneously, all instructors simultaneously, or all student cohorts simultaneously to provide an overview of the entire scheduling situation. Users shall be able to switch between these view levels easily and filter or search for specific entities. The display shall clearly indicate time slots, assigned courses, instructors, and locations with color coding to highlight different course types or potential conflicts.

UR4 - Three-Level Operational Scope

The scheduling system operates across three distinct organizational levels that must be managed consistently. At the instructor level, the system manages teaching assignments, instructor availability constraints, and workload distribution across all faculty members. At the facility level,

the system tracks room allocations, laboratory bookings, and equipment requirements for all physical spaces. At the cohort level, the system coordinates course schedules for student groups defined by curriculum requirements ensuring students can attend all required courses without conflicts. The system must maintain data integrity and constraint enforcement across all three levels simultaneously.

UR5 - Schedule Generation and Optimization

Administrators must be able to automatically generate conflict-free schedules by configuring parameters and initiating the optimization process. The system shall propose optimized schedule solutions that satisfy all hard constraints and minimize soft constraint violations. Users shall be able to configure optimization parameters such as the number of generations to run and relative priorities for different soft constraints. The system shall display progress during generation showing current fitness scores and constraint violation statistics. Upon completion, the system shall present the generated schedule with quality metrics and conflict reports if any hard constraints could not be satisfied.

UR6 - Schedule Storage and Version Management

Users must be able to save generated schedules and access them later for review or further modification. The system shall automatically assign version numbers to each saved schedule with timestamps and metadata about generation parameters. Users shall be able to list all saved schedule versions, select specific versions to view or compare, and revert to previous versions if needed. The system shall maintain a history of schedule modifications for audit purposes.

UR7 - Manual Schedule Adjustment

Administrators must be able to manually modify automatically generated schedules when specific adjustments are needed. The system shall provide an interactive interface allowing users to change course time slots, reassign rooms, or swap instructors. All manual modifications shall be validated in real-time with the system immediately checking for constraint violations and displaying warnings if conflicts are introduced. Users shall be able to see suggested alternative times or rooms when conflicts occur. The system shall require explicit confirmation before saving changes that violate hard constraints, ensuring administrators are aware of and accept any scheduling conflicts.

UR8 - Schedule Export and Integration

Users must be able to export schedules in multiple formats for distribution and integration with other systems. The system shall export schedules to PDF format with customizable layouts suitable for printing and posting. CSV exports shall provide machine-readable data for further processing or archival. iCalendar format exports shall enable integration with personal calendar applications like Google Calendar, Outlook, or Apple Calendar. Users shall be able to select whether to export complete schedules or filtered subsets such as schedules for specific departments or time periods.

UR9 - Change Notifications

Instructors must be notified when schedules affecting their teaching assignments are published or modified. The system shall send email notifications to instructors when new schedules are finalized and released. Modification notifications shall be sent when changes are made to published schedules that affect an instructor's assignments. Users shall be able to configure notification preferences including email frequency and types of changes that trigger notifications.

3.2.2 System Requirements

System requirements specify technical capabilities the system must provide to support user requirements. These define how the system operates internally to deliver required functionality.

SR1 - Multi-User Concurrent Access

The system shall support simultaneous access by multiple users without data corruption or conflicts. All database operations shall use appropriate transaction isolation levels to prevent race conditions. The system shall enforce role-based permissions at both the API level and database level to ensure users can only perform authorized operations. Session management shall track active users and maintain separate authentication contexts for each user.

SR2 - Persistent Data Storage

The system shall store all institutional data, user accounts, schedules, and system configurations in a relational database with ACID guarantees. Database schemas shall define appropriate primary keys, foreign keys, and constraints to maintain referential integrity. The system shall implement regular automated backups to prevent data loss and support point-in-time recovery. All data modifications shall be logged for audit trails.

SR3 - Automatic Schedule Proposal

The system shall implement an optimization engine capable of automatically generating schedule proposals that satisfy constraints. The engine shall use genetic algorithm techniques to explore the solution space and converge toward high-quality schedules. The system shall validate all generated schedules against defined hard constraints before presenting them to users. Multiple runs with different random seeds or parameters shall produce diverse schedule alternatives.

SR4 - Multi-Level Schedule Display and Abstraction

The system shall provide APIs and user interface components to display schedules at multiple levels of abstraction. Database queries shall efficiently retrieve schedule data for single entities or entire collections. The system shall aggregate and summarize schedule information to support comprehensive overview displays. Display components shall render schedules in grid, list, and calendar formats as appropriate for different view levels.

SR5 - Three-Level Data Model Integration

The system shall maintain a unified data model that integrates instructor assignments, facility allocations, and cohort schedules. Database foreign keys shall enforce relationships between courses, instructors, rooms, and curricula. Constraint validation logic shall check consistency across all three levels to detect conflicts such as instructor double-booking, room double-booking, or student group schedule conflicts. Reporting functions shall provide analytics across all three organizational levels.

SR6 - Real-Time Constraint Validation

The system shall validate schedule modifications in real-time as users make changes through the interface. Validation logic shall execute quickly enough to provide immediate feedback typically within one second. The system shall clearly communicate constraint violations with specific explanations identifying which courses, resources, or time slots are involved in conflicts.

Validation shall check both hard constraints that must not be violated and soft constraints that should be minimized.

SR7 - Asynchronous Task Processing

The system shall execute long-running schedule generation tasks asynchronously using a task queue and worker processes. API endpoints that initiate optimization shall return immediately with a job identifier allowing users to continue using the interface. Workers shall process optimization jobs in the background and update job status as they progress. Users shall be able to

monitor job progress and cancel running jobs if needed. Completed job results shall be stored and linked to user accounts for later retrieval.

SR8 - Comprehensive Audit Logging

The system shall maintain detailed logs of all schedule modifications including user identity, timestamp, and specific changes made. Logs shall capture both automated changes from optimization runs and manual edits from administrators. Log entries shall be tamper-proof and retained for a minimum of one academic year. Administrators shall be able to query logs to understand the history of schedule evolution and identify who made specific changes.

SR9 - Schedule Versioning and Comparison

The system shall assign unique version identifiers to each saved schedule and store complete schedule state for each version. Users shall be able to compare different schedule versions side-by-side to understand differences in assignments, constraint violations, or quality metrics.

The system shall compute and display comparison statistics such as changes in room utilization, instructor workload balance, and fitness scores. Reversion functionality shall allow restoring previous schedule versions if needed.

3.2.3 Non-Functional Requirements

Non-functional requirements specify quality attributes and constraints the system must satisfy. These requirements define how well the system performs its functions and are critical for user satisfaction and system success. Only requirements that can be tested and validated within the scope of this project are included.

NFR1 - Security

Security protects sensitive academic data and ensures only authorized users can perform privileged operations. The system shall use HTTPS with TLS 1.3 for all client-server communication to prevent eavesdropping and man-in-the-middle attacks. User passwords shall be hashed using bcrypt with appropriate cost factors before storage ensuring passwords cannot be recovered even if the database is compromised. The system shall implement role-based access control strictly enforcing that only users with administrator roles can generate or modify schedules while instructors and students have read-only access to relevant data. All user inputs shall be sanitized and validated to prevent SQL injection, cross-site scripting, and other common web vulnerabilities. The system shall implement rate limiting on login endpoints to mitigate brute-

force password attacks. Session tokens shall expire after configurable periods of inactivity forcing users to re-authenticate.

Security testing will verify that unauthorized users cannot access protected resources, password hashing is implemented correctly, and common vulnerability scanning tools do not identify critical security flaws.

NFR2 - Usability

Usability ensures the system is easy to learn and efficient to use reducing training requirements and user frustration. The user interface shall be intuitive with clear navigation, consistent layouts, and self-explanatory labels minimizing the learning curve. The system shall provide contextual help and tooltips for complex features such as constraint configuration or optimization parameters. Conflict indicators shall use color coding with red for hard constraint violations, yellow for soft constraint violations, and green for conflict-free schedules. The system shall offer wizards or guided workflows for multi-step processes such as initial data import or schedule generation. Error messages shall be informative explaining what went wrong and suggesting how to correct the problem rather than displaying technical error codes. The interface shall be responsive adapting layouts to work on desktop monitors, tablets, and mobile devices.

Usability testing will involve representative users performing common tasks while observers record task completion times, error rates, and user satisfaction ratings. The goal is for users to successfully complete basic tasks without training and complete advanced tasks after brief instruction.

NFR3 - Maintainability

Maintainability ensures the system can be easily modified, extended, and debugged over its operational lifetime. The codebase shall be organized into clear modules with separation of concerns grouping related functionality together such as authentication, data management, optimization, and reporting. Code shall follow consistent style guidelines specifically PEP 8 for Python and Airbnb JavaScript style guide for frontend code. Functions and classes shall be documented with docstrings explaining their purpose, parameters, return values, and any side effects. The database schema shall include comments documenting the purpose of each table and column. The system shall maintain test coverage of at least 60% for backend code with unit tests for critical business logic and constraint validation. A comprehensive README shall explain how to set up the development environment, run tests, and deploy the application.

Maintainability will be evaluated by having developers unfamiliar with the codebase perform modification tasks such as adding a new constraint type or creating a new report. The time required and number of questions asked indicate maintainability quality.

NFR4 - Modular Design

The system architecture shall follow modular design principles enabling independent development, testing, and replacement of components. The backend shall expose a well-defined RESTful API with documented endpoints, request formats, and response structures allowing the frontend to be replaced or alternative clients to be developed. The optimization engine shall be encapsulated as a separate module with clear interfaces for input constraints and output schedules allowing algorithm improvements without affecting other system components. Database access shall be abstracted through an ORM layer allowing database engine changes without modifying business logic. The frontend shall be organized into reusable React components with clear prop interfaces facilitating component reuse and testing.

Modularity will be tested by replacing individual components such as swapping the genetic algorithm implementation for a different optimization approach and verifying the system continues to function correctly.

3.3 Use Case Modeling

Use cases describe interactions between users (actors) and the system to accomplish specific goals. Each use case represents a distinct piece of functionality from the user's perspective.

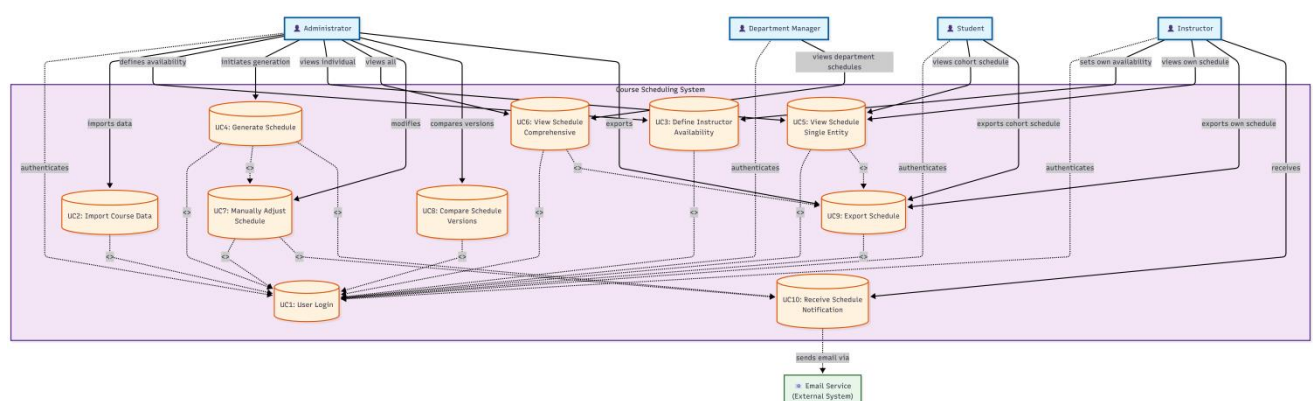


Figure 3.1 - Use Case Diagram

Use Case Diagram showing actors (Administrator, Instructor, Student) and their interactions with system functionalities including authentication, data management, schedule generation,

viewing, and export.

The use case diagram illustrates the primary actors and their interactions with the system. Administrators have full access to all functionalities including data management and schedule generation. Instructors can view their schedules and update availability constraints. Students have read-only access to published course schedules.

Use Case Descriptions

Each use case is described in detail with the following table structure:

UC1: User Login

Attribute	Description
Use Case ID	UC1
Use Case Name	User Login
Actor	All Users (Administrator, Instructor, Student, Department Manager)
Description	User authenticates with the system using username and password to access role-appropriate features
Preconditions	User has a registered account in the system
Basic Flow	1. User navigates to login page 2. User enters username and password 3. System validates credentials 4. System creates session and redirects to dashboard
Alternative Flow	3a. Invalid credentials: System displays error message and allows retry 3b. Account locked: System displays message to contact administrator
Post conditions	User is authenticated with active session and can access authorized features
Input	Username (string), Password (string)
Output	Session token, User role, Dashboard view
Business Rules	- Passwords must meet complexity requirements -Sessions expire after 60 minutes of inactivity - Failed login attempts are logged

UC2: Import Course Data

Attribute	Description
Use Case ID	UC2
Use Case Name	Import Course Data
Actor	Administrator
Description	Administrator uploads a CSV or Excel file containing course information to bulk populate the course catalog
Preconditions	User is logged in with Administrator role and has prepared data file
Basic Flow	1. Administrator navigates to Data Import section 2. Administrator selects "Import Courses" option 3. Administrator uploads file 4. System validates file format and data 5. System displays preview of courses to be imported 6. Administrator confirms import 7. System inserts courses into database
Alternative Flow	4a. Invalid file format: System displays error and format requirements 4b. Data validation errors: System displays row-specific errors and allows correction 6a. Administrator cancels: System discards imported data
Post conditions	Course catalog is populated with imported courses
Input	CSV or Excel file with columns: Course Code, Course Name, Credits, Type (Lecture/Lab), Weekly Hours
Output	Import summary report showing number of courses added, updated, or failed
Business Rules	- Course codes must be unique -Credit hours must be positive integers -Duplicate courses update existing records rather than creating duplicates

UC3: Define Instructor Availability

Attribute	Description
Use CaseID	UC3
Use Case Name	Define Instructor Availability

Actor	Administrator, Instructor
Description	User specifies time slots when an instructor is unavailable for teaching assignments
Preconditions	User is logged in, Instructor exists in system, Time slots are configured
Basic Flow	1. User selects instructor from list 2. System displays weekly schedule grid 3. User clicks on time slots to mark unavailable 4. User saves availability constraints 5. System validates and stores constraints
Alternative Flow	3a. User marks previously unavailable slot as available: System removes constraint 4a. User cancels: System discards changes
Post conditions	Instructor availability constraints are stored and will be enforced during schedule generation
Input	Instructor ID, List of unavailable time slots (day, time)
Output	Updated availability grid, Confirmation message
Business Rules	- Instructors can be marked unavailable for entire days or specific time slots - Availability constraints are soft deletable for history

UC4: Generate Schedule

Attribute	Description
Use Case ID	UC4
Use Case Name	Generate Schedule
Actor	Administrator
Description	Administrator initiates automatic schedule generation using genetic algorithm optimization
Precondition s	Courses, instructors, rooms, curricula, and time slots are configured in the system

Basic Flow	1. Administrator navigates to Schedule Generation 2. Administrator configures parameters (population size, generations, constraint weights) 3. Administrator clicks "Generate Schedule" 4. System creates optimization job and assigns job ID 5. System displays progress page showing generation number and fitness 6. System completes optimization and displays results 7. Administrator reviews schedule and conflict report 8. Administrator saves schedule with version name
Alternative Flow	5a. Administrator cancels generation: System terminates optimization job 6a. Optimization fails to find feasible solution: System displays best effort result with constraint violations 8a. Administrator discards result without saving
Post conditions	A new schedule version is saved in the database with generation metadata
Input	Optimization parameters (population size, generations, mutation rate, constraint weights)
Output	Generated schedule with course-time-room-instructor assignments, Fitness score, Conflict report
Business Rules	- Generation may run for several minutes to hours depending on problem size - Multiple schedules can be generated with different parameters for comparison- - Saved schedules are immutable once published

UC5: View Schedule - Single Entity

Attribute	Description
Use Case ID	UC5
Use Case Name	View Schedule - Single Entity
Actor	Administrator, Instructor, Student
Description	User views the schedule for a specific instructor, room, or student cohort
Preconditions	User is logged in, Schedule has been generated and published
Basic Flow	1. User navigates to Schedule View 2. User selects view type (Instructor, Room, or Cohort) 3. User selects specific entity from dropdown 4. System retrieves schedule data for selected entity 5. System displays schedule in weekly grid format

Alternative Flow	3a. User searches for entity by name 5a. User switches to list view or calendar view
Post conditions	None (read-only operation)
Input	View type (Instructor/Room/Cohort), Entity identifier
Output	Weekly schedule grid showing assigned courses with times and locations
Business Rules	- Instructors can only view their own schedules- Students can view schedules for their enrolled cohorts- Administrators can view all schedules

UC6: View Schedule - Comprehensive

Attribute	Description
Use Case ID	UC6

Use Case Name	View Schedule - Comprehensive
Actor	Administrator, Department Manager
Description	User views schedules for all entities of a type simultaneously to get an overview
Preconditions	User has authorization, Schedule has been published
Basic Flow	1. User navigates to Comprehensive View 2. User selects view type (All Instructors, All Rooms, or All Cohorts 3. System retrieves all schedule data for selected type 4. System displays consolidated view with all entities
Alternative Flow	4a. Too many entities: System paginates or allows filtering by department/building
Post conditions	None
Input	View type (All Instructors/All Rooms/All Cohorts), Optional filters
Output	Multi-entity schedule display showing utilization and assignments
Business Rules	- View may be restricted to departmental data for non-administrator users- Display uses color coding to distinguish different courses or conflicts

UC7: Manually Adjust Schedule

Attribute	Description
Use Case ID	UC7
Use Case Name	Manually Adjust Schedule
Actor	Administrator
Description	Administrator makes manual modifications to a generated schedule by changing time slots, rooms, or instructors
Preconditions	Administrator is logged in, Schedule exists and is unlocked for editing

Basic Flow	1. Administrator opens schedule for editing 2. Administrator selects course assignment to modify 3. Administrator changes time slot, room, or instructor 4. System immediately validates changes 5. System displays conflicts if any 6. Administrator confirms or reverts changes 7. System saves modifications
Alternative Flow	5a. No conflicts: System applies changes immediately 5b. Hard constraint violated: System shows warning and suggests alternatives 6a. Administrator reverts: System restores previous state
Post conditions	Schedule is updated with manual changes, Audit log records modification

Input	Course assignment ID, New time slot/room/instructor
Output	Updated schedule, Conflict warnings if applicable, Confirmation message
Business Rules	- All modifications are validated against constraints- Changes to published schedules trigger notifications to affected instructors-Modification history is maintained

UC8: Compare Schedule Versions

Attribute	Description
Use Case ID	UC8
Use Case Name	Compare Schedule Versions
Actor	Administrator
Description	Administrator compares multiple schedule versions to evaluate quality and select the best option
Preconditions	Multiple schedule versions exist in the system
Basic Flow	1. Administrator navigates to Schedule Comparison 2. Administrator selects two or more schedule versions 3. System retrieves selected schedules 4. System computes comparison metrics 5. System displays side-by-side comparison with differences highlighted
Alternative Flow	5a. Administrator exports comparison report to PDF
Post conditions	None

Input	List of schedule version IDs to compare
Output	Comparison table showing fitness scores, constraint violations, room utilization, instructor workload balance
Business Rules	- Up to 5 schedules can be compared simultaneously - Comparison highlights which schedules perform better on specific metrics

UC9: Export Schedule

Attribute	Description
Use Case ID	UC9
Use Case	Export Schedule
Actor	Administrator, Instructor, Student
Description	User exports schedule data in PDF, CSV, or iCalendar format
Preconditions	User is logged in, Schedule is published, User has permission to export
Basic Flow	1. User navigates to schedule view 2. User selects export option 3. User chooses export format (PDF/CSV/iCal) 4. User selects scope (own schedule or full schedule if authorized) 5. System generates export file 6. System initiates download
Alternative Flow	5a. PDF generation: User selects layout options (portrait/landscape, page size) 5b. iCal export: User selects date range
Postcondition s	Export file is downloaded to user's device
Input	Export format, Export scope, Optional formatting parameters
Output	File in requested format (PDF/CSV/ICS)
Business Rules	- PDF exports are formatted for printing - CSV exports include all assignment details - iCal exports create calendar events compatible with Google Calendar and Outlook

UC10: Receive Schedule Notification

Attribute	Description
Use Case ID	UC10
Use Case Name	Receive Schedule Notification
Actor	Instructor
Description	Instructor receives email notification when schedules affecting them are published or modified
Preconditions	Instructor account has valid email address, Schedule involving instructor is published or modified
Basic Flow	1. Administrator publishes or modifies schedule 2. System identifies affected instructors 3. System generates notification emails 4. System queues emails for delivery 5. Email service sends notifications 6. Instructor receives and reads email
Alternative Flow	5a. Email delivery fails: System retries and logs failure
Post conditions	Instructor is informed of schedule changes
Input	Schedule version, List of affected instructors
Output	Email notification containing summary of assignments or changes
Business Rules	- Notifications are sent asynchronously to avoid delaying publication - Users can configure notification preferences -Delivery status is logged

3.4 Entity-Relationship Diagram

The system's data model is designed using a relational database to ensure data integrity, enforce constraints, and support efficient querying. The Entity-Relationship (ER) diagram illustrates the main entities, their attributes, and the relationships between them.

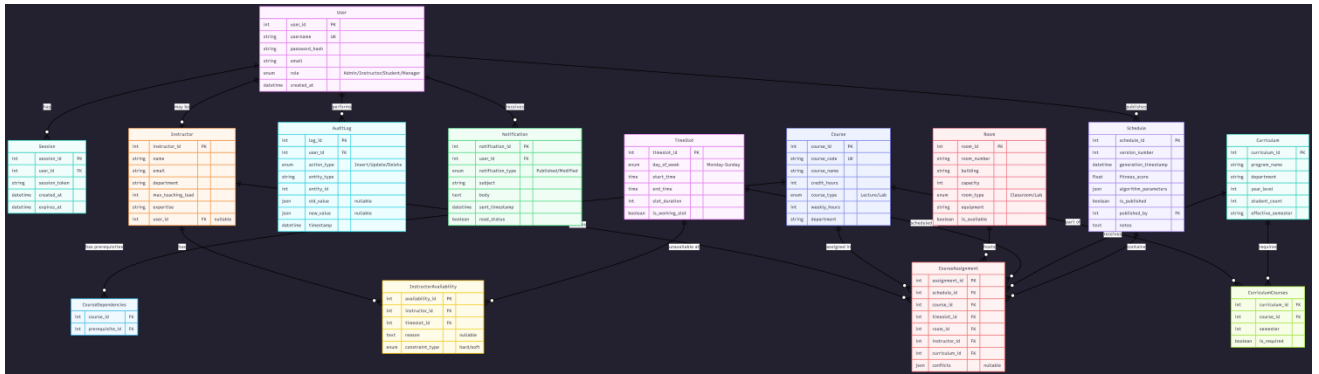


Figure 3.2 The ER diagram depicts the following key entities and relationships:

The **User** entity stores authentication and profile information for all system users with attributes including user_id (primary key), username, password_hash, email, role (Administrator/Instructor/Student/Manager), and created_at timestamp. Users can have multiple sessions tracked in a Sessions table.

The **Instructor** entity represents faculty members with attributes including instructor_id (primary key), name, email, department, max_teaching_load (hours per week), expertise (comma-separated specializations), and user_id (foreign key to User if instructor has system access). Instructors have a one-to-many relationship with InstructorAvailability defining when they cannot teach.

The **Course** entity defines courses offered by the institution with attributes including course_id (primary key), course_code (unique), course_name, credit_hours, course_type (Lecture/Lab), weekly_hours, and department. Courses have many-to-many relationships with Curricula through CurriculumCourses junction table and with Prerequisites through CourseDependencies self-referencing table.

The **Room** entity represents physical teaching spaces with attributes including room_id (primary key), room_number, building, capacity, room_type (Classroom/Lab), equipment (projector, whiteboard, computers), and is_available (boolean). Rooms can be marked unavailable for maintenance or special purposes.

The **Curriculum** entity defines degree programs and student cohorts with attributes including curriculum_id (primary key), program_name, department, year_level, student_count (cohort size), and effective_semester. Each curriculum specifies which courses are required through the CurriculumCourses junction table with semester and required/elective flags.

The **TimeSlot** entity defines the institutional calendar structure with attributes including timeslot_id (primary key), day_of_week (Monday-Sunday), start_time, end_time, slot_duration, and is_working_slot (boolean for holidays or breaks). Time slots define when courses can be scheduled.

The **Schedule** entity represents generated timetable versions with attributes including schedule_id (primary key), version_number, generation_timestamp, fitness_score, algorithm_parameters (JSON), is_published (boolean), published_by (foreign key to User), and notes. Each schedule contains multiple CourseAssignments.

The **CourseAssignment** entity is the core scheduling entity linking courses to times, rooms, and instructors with attributes including assignment_id (primary key), schedule_id (foreign key), course_id (foreign key), timeslot_id (foreign key), room_id (foreign key), instructor_id (foreign key), curriculum_id (foreign key to identify which cohort), and conflicts (JSON array of constraint violations if any). This entity captures the complete assignment decision.

The **InstructorAvailability** entity records time constraints with attributes including availability_id (primary key), instructor_id (foreign key), timeslot_id (foreign key), reason (optional text), and constraint_type (hard unavailable or soft preference).

The **AuditLog** entity maintains history of modifications with attributes including log_id (primary key), user_id (foreign key), action_type (Insert/Update/Delete), entity_type (Course/Instructor/Schedule), entity_id, old_value (JSON), new_value (JSON), and timestamp. This supports accountability and schedule history reconstruction.

The **Notification** entity tracks sent messages with attributes including notification_id (primary key), user_id (foreign key to recipient), notification_type (SchedulePublished/ScheduleModified), subject, body, sent_timestamp, and read_status.

Key relationships include one-to-many from Instructor to CourseAssignment indicating which courses each instructor teaches, one-to-many from Room to CourseAssignment indicating which courses are held in each room, one-to-many from TimeSlot to CourseAssignment indicating when each course meets, one-to-many from Schedule to CourseAssignment as each schedule version contains multiple assignments, many-to-many between Course and Curriculum through CurriculumCourses junction table, and one-to-many from Instructor to InstructorAvailability defining constraint time slots.

This relational schema enforces referential integrity through foreign key constraints, prevents orphaned records, enables efficient joins for complex queries like retrieving all assignments for an instructor or detecting conflicts, and supports ACID transactions ensuring data consistency during schedule generation and modification operations.

3.5 UML Diagrams

Unified Modeling Language (UML) diagrams provide visual representations of the system's structure, behavior, and interactions. These diagrams facilitate communication between stakeholders and developers while documenting design decisions.

3.5.1 Class Diagram



Figure 3.3 - Class Diagram

Class Diagram illustrating the object-oriented structure of the system including main classes for Schedule, Course, Instructor, Room, TimeSlot, GeneticAlgorithm, and ConstraintValidator with their attributes, methods, and relationships.

The class diagram depicts the software architecture from an object-oriented perspective. Key classes include:

The **Schedule** class represents a complete timetable solution with attributes `schedule_id`, `version_number`, `fitness_score`, and `assignments` (list of `CourseAssignment` objects), and methods including `validate()` to check all constraints, `get_conflicts()` to return list of violations, and `export(format)` to generate outputs. The `Schedule` class has composition relationship with `CourseAssignment`.

The **CourseAssignment** class encapsulates individual scheduling decisions with attributes `course`, `timeslot`, `room`, `instructor`, and `curriculum`, and methods `is_valid()` returning boolean and `get_conflicts()` returning list of constraint violations.

The **GeneticAlgorithm** class implements the optimization engine with attributes `population_size`, `generations`, `mutation_rate`, `crossover_rate`, `current_population` (list of `Schedule` objects), and `best_solution` (`Schedule`), and methods including `initialize_population()` creating random schedules, `evaluate_fitness()` calculating fitness for all individuals, `select_parents()` choosing chromosomes for reproduction, `crossover()` producing offspring, `mutate()` introducing random variations, and `run()` executing the complete algorithm and returning the best schedule.

The **ConstraintValidator** class centralizes constraint checking with static methods `check_instructor_conflict(assignment, schedule)`, `check_room_conflict(assignment, schedule)`, `check_student_conflict(assignment, schedule)`, `check_capacity(assignment)`, `check_availability(assignment)`, and `validate_schedule(schedule)` which calls all checks and aggregates results.

The **Course** class models academic courses with attributes `course_id`, `course_code`, `name`, `credits`, `type`, and `weekly_hours`.

The **Instructor** class represents faculty with attributes `instructor_id`, `name`, `department`, `expertise`, `max_load`, and `unavailable_slots` (list of `TimeSlot` objects).

The **Room** class models facilities with attributes `room_id`, `room_number`, `building`, `capacity`, `type`, and `equipment`.

The **TimeSlot** class defines time periods with attributes `timeslot_id`, `day`, `start_time`, and `end_time`.

The **Curriculum** class represents student groups with attributes `curriculum_id`, `program_name`, `year_level`, and `required_courses` (list of Course objects).

Relationships include GeneticAlgorithm uses ConstraintValidator to evaluate fitness and validate solutions, Schedule aggregates multiple CourseAssignment objects, CourseAssignment associates with Course, Instructor, Room, TimeSlot, and Curriculum, and Instructor has association with TimeSlot through `unavailable_slots` representing constraints.

This object-oriented design promotes modularity allowing independent testing and modification of components, encapsulation keeping constraint logic centralized in ConstraintValidator, and reusability enabling the GeneticAlgorithm class to be adapted for different scheduling problems.

3.5.2 Sequence Diagram - Generate Schedule

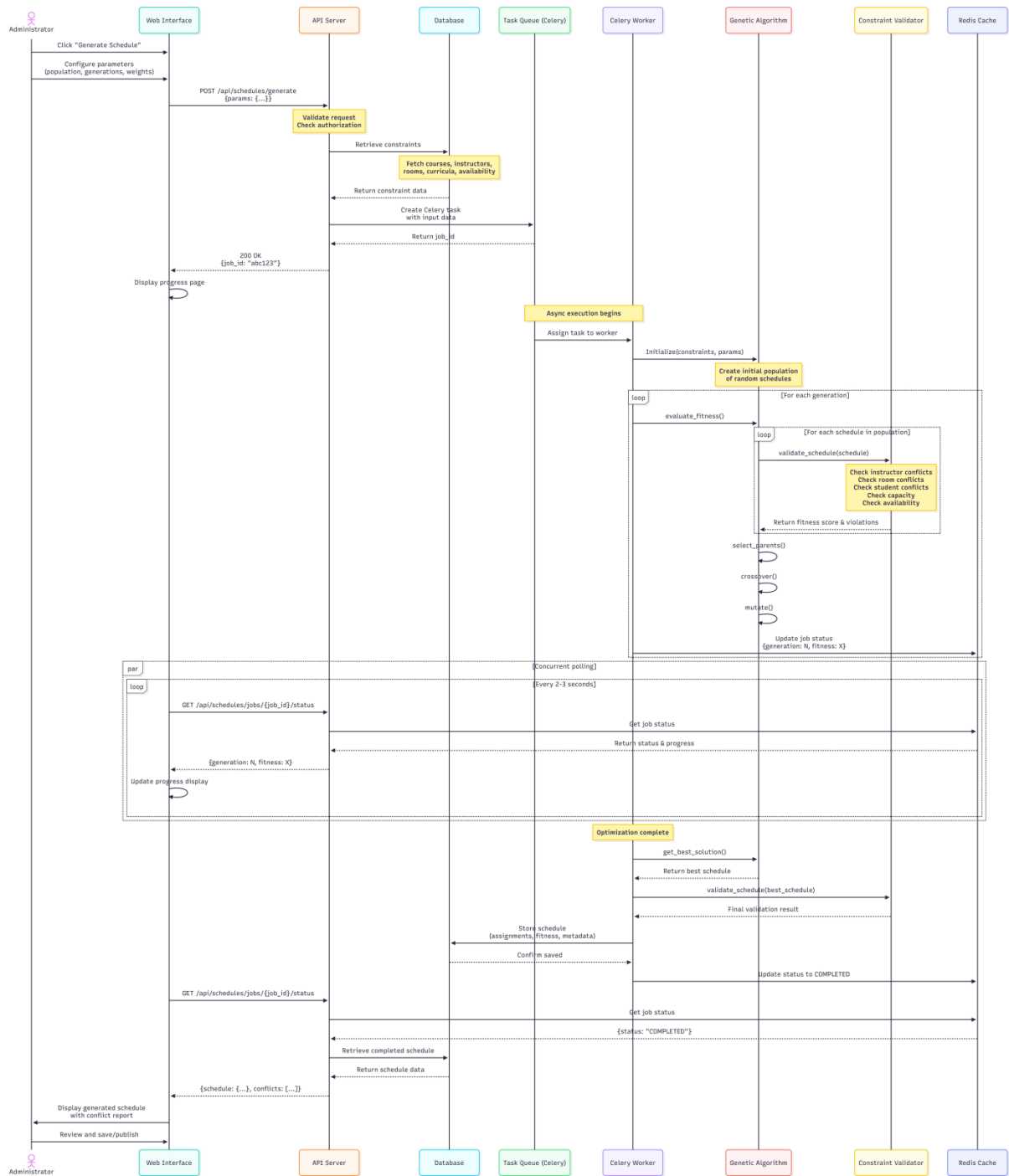


Figure 3.4 - Sequence Diagram for Generate Schedule Use Case

Caption: Sequence Diagram showing the interaction flow between User, Web Interface, API Server, Task Queue (Celery), Genetic Algorithm Worker, and Database during schedule generation process.

The sequence diagram illustrates the message flow and temporal ordering of operations during automatic schedule generation:

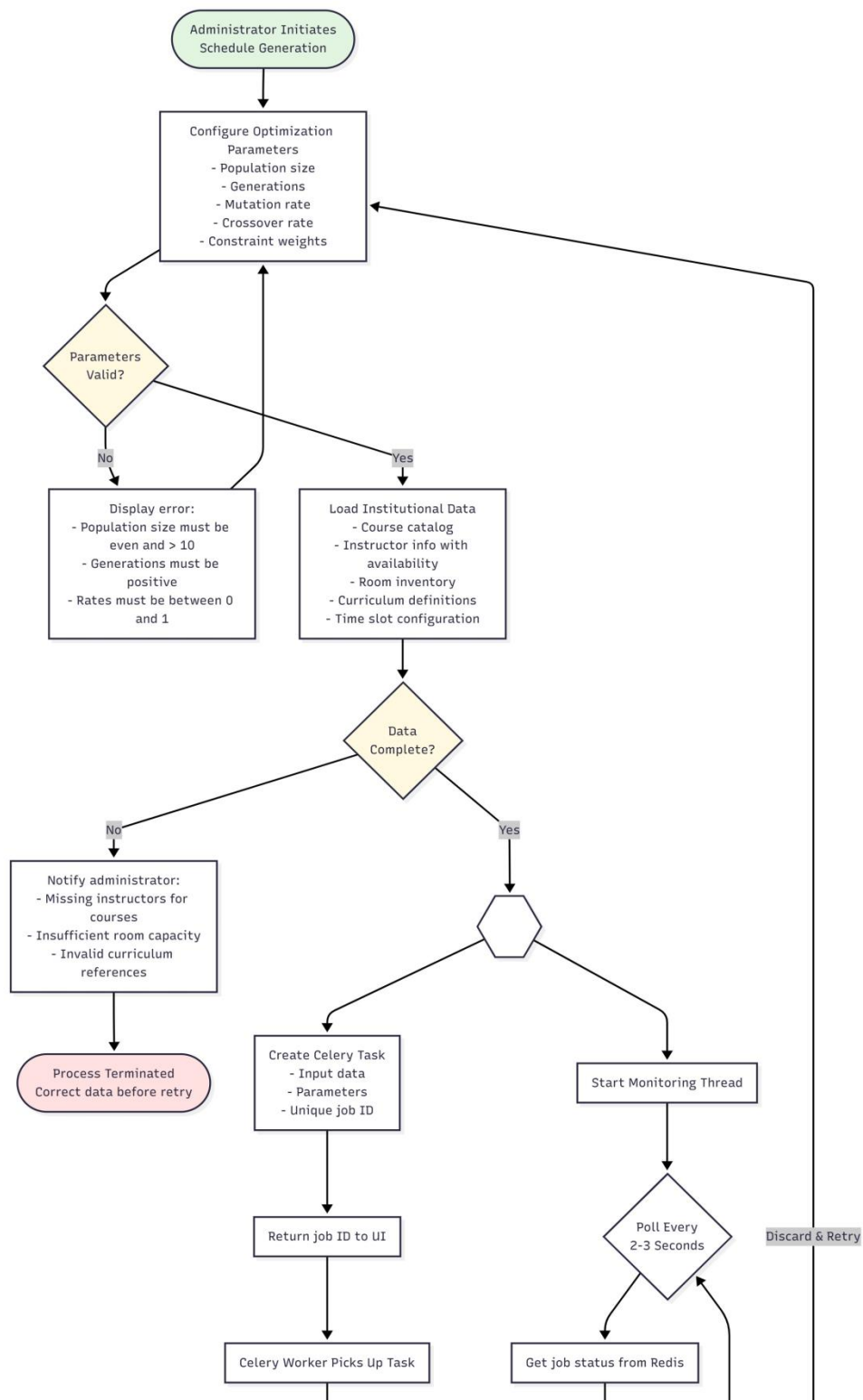
The user (Administrator) initiates the process by clicking "Generate Schedule" in the web interface and configuring optimization parameters. The web interface sends a POST request to the API server endpoint `/api/schedules/generate` with parameters in JSON format. The API server validates the request checking user authorization and parameter validity. The API server retrieves constraints from the database including courses, instructors, rooms, curricula, availability, and time slots. The API server creates a Celery task for asynchronous execution and returns immediately to the web interface with a `job_id`. The web interface displays a progress page and begins polling the API server periodically with GET requests to `/api/schedules/jobs/{job_id}/status`.

Meanwhile, the Celery worker picks up the queued task and begins execution. The worker initializes the GeneticAlgorithm class with retrieved constraints and parameters. The algorithm enters its main optimization loop for the specified number of generations. In each generation, the algorithm calls `evaluate_fitness()` on each schedule in the population which internally invokes `ConstraintValidator` to check for violations and compute penalty scores. The algorithm performs selection, crossover, and mutation operations to create the next generation. Periodically, the worker updates job status in Redis cache including current generation number and best fitness score so the web interface can display progress.

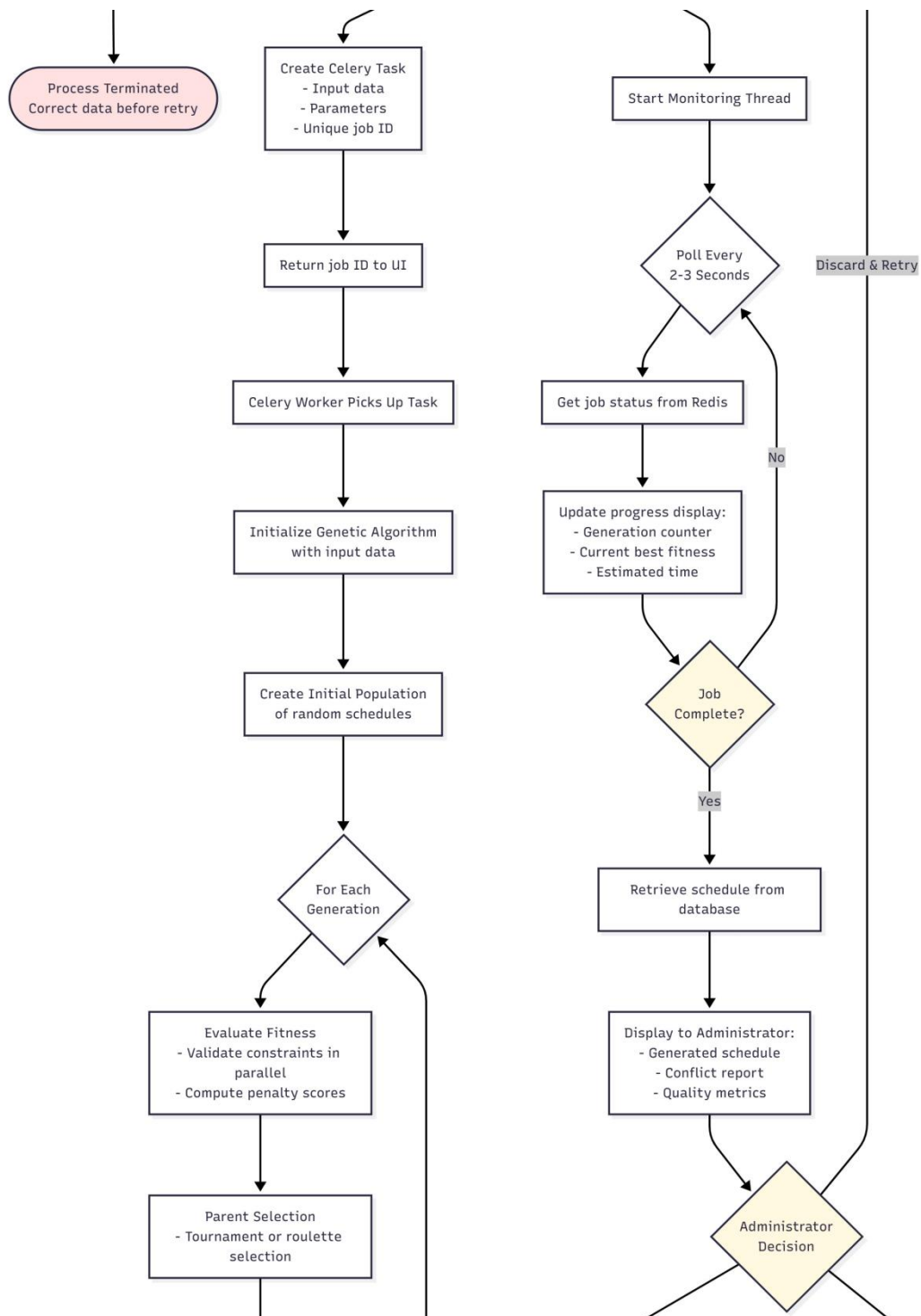
After completing all generations, the worker retrieves the best schedule from the final population. The worker validates the final schedule comprehensively ensuring all hard constraints are satisfied if possible. The worker stores the completed schedule in the database with assignment details, fitness score, and metadata. The worker updates job status to "COMPLETED" in Redis. On the next polling request from the web interface, the API server retrieves the completed schedule and returns it. The web interface displays the generated schedule to the user with options to view details, identify conflicts, compare with other versions, or save and publish.

This asynchronous architecture prevents the user interface from freezing during long-running optimizations, allows users to monitor progress in real-time, enables cancellation of running jobs if needed, and supports concurrent generation of multiple schedules by different users.

3.5.3 Activity Diagram - Generate Schedule Workflow



##



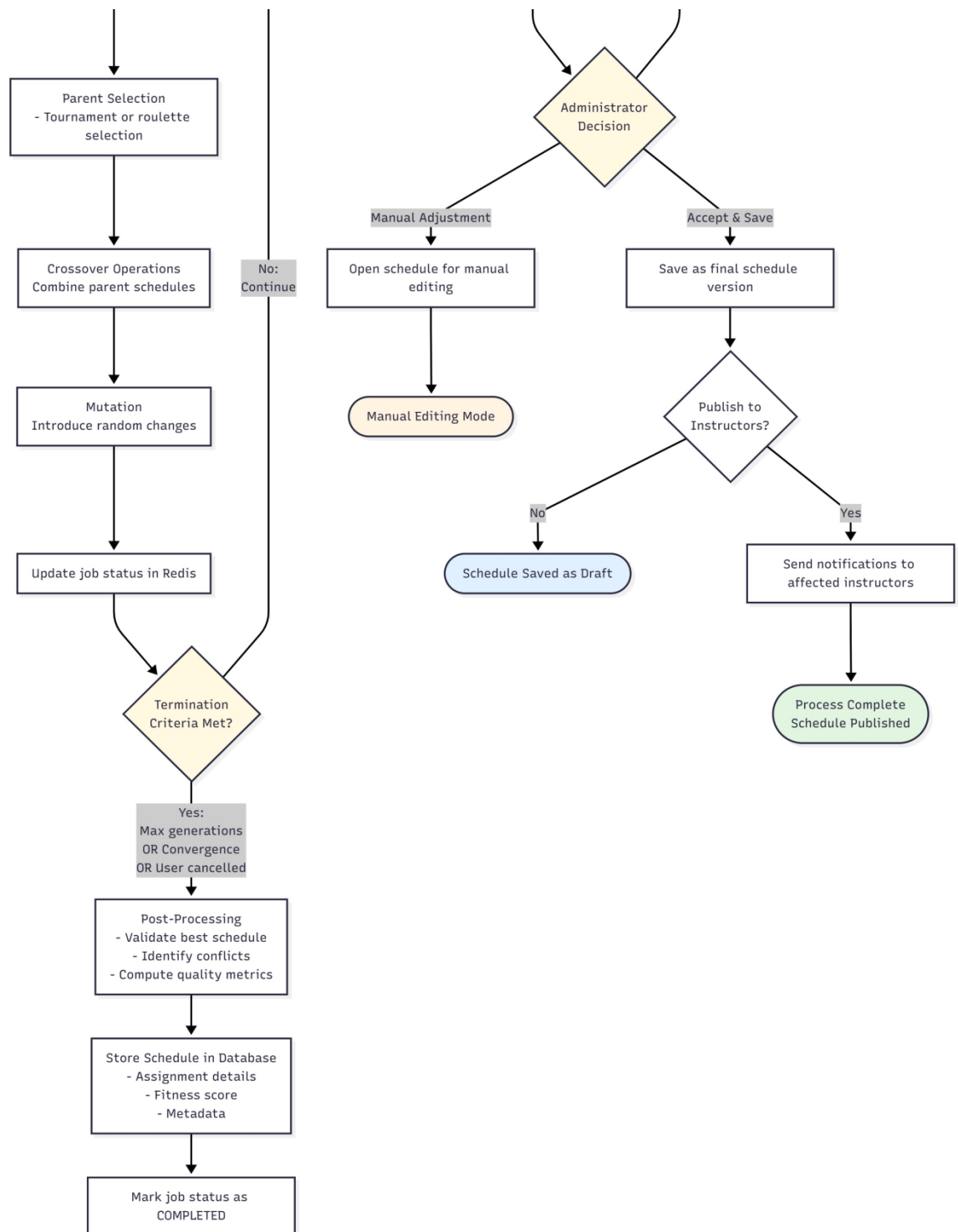


Figure 3.5 - Activity Diagram for Schedule Generation Workflow

Caption: Activity Diagram depicting the complete workflow of schedule generation including decision points, parallel activities, and error handling paths.

The activity diagram shows the flow of control and decisions throughout the schedule generation

process:

The workflow begins when an administrator navigates to the schedule generation page. The first activity is to configure optimization parameters including population size, number of generations, mutation rate, crossover rate, and constraint weight priorities. A decision point checks if parameters are valid with constraints like population size must be even and greater than 10, generations must be positive, rates must be between 0 and 1, and constraint weights must be non-negative.

If parameters are invalid, the flow returns to configuration with error messages. If valid, the system proceeds to load institutional data from the database including the course catalog, instructor information with availability constraints, room inventory with capacities and types, curriculum definitions with required courses, and time slot configuration.

The system then validates data completeness checking that all courses have qualified instructors available, room capacities can accommodate student cohorts, curricula reference valid courses, and at least one feasible time slot exists for each course. If data validation fails, the administrator is notified of specific issues such as missing instructors for required courses or insufficient room capacity, and the process terminates allowing the administrator to correct data before retrying.

If data is valid, two parallel activities begin. The main thread creates a Celery task with input data and parameters, stores the task with a unique job ID, and returns job ID to the user interface. Simultaneously, a monitoring thread starts polling job status every 2-3 seconds. The user interface displays a progress page showing generation counter, current best fitness, and estimated time remaining.

The Celery worker thread, running asynchronously, initializes the genetic algorithm with the input data. The algorithm creates an initial population of random schedules ensuring diversity. The main optimization loop begins, iterating for the configured number of generations. In each iteration, fitness evaluation computes fitness for all individuals in parallel using constraint validation, parent selection chooses high-fitness schedules using tournament or roulette selection, crossover operations combine pairs of parents to produce offspring, and mutation introduces random changes to offspring.

A decision point checks if termination criteria are met including maximum generations completed, fitness convergence where improvement has stalled, or user cancellation requested. If not

terminated, the loop continues with the next generation. The worker periodically updates status in Redis so the monitoring thread can display progress to the user.

When optimization completes, post-processing activities validate the best schedule against all constraints, identify any remaining conflicts, compute quality metrics including room utilization, instructor workload balance, gap minimization, and preference satisfaction, and store the complete schedule in the database with all assignment details and metadata.

The monitoring thread detects completion, retrieves results from the database, and displays the schedule to the administrator with conflict reports and quality metrics. The administrator reviews results and decides whether to accept and save the schedule, discard and try different parameters, or make manual adjustments to the generated schedule. If accepted, the schedule version is marked as final and can be published to instructors and students.

This workflow supports iterative refinement where administrators can generate multiple schedules with different parameters, compare results, and select the best option. Error handling paths ensure graceful failure with informative messages when data is incomplete or optimization cannot find feasible solutions.

3.5.4 Component Diagram

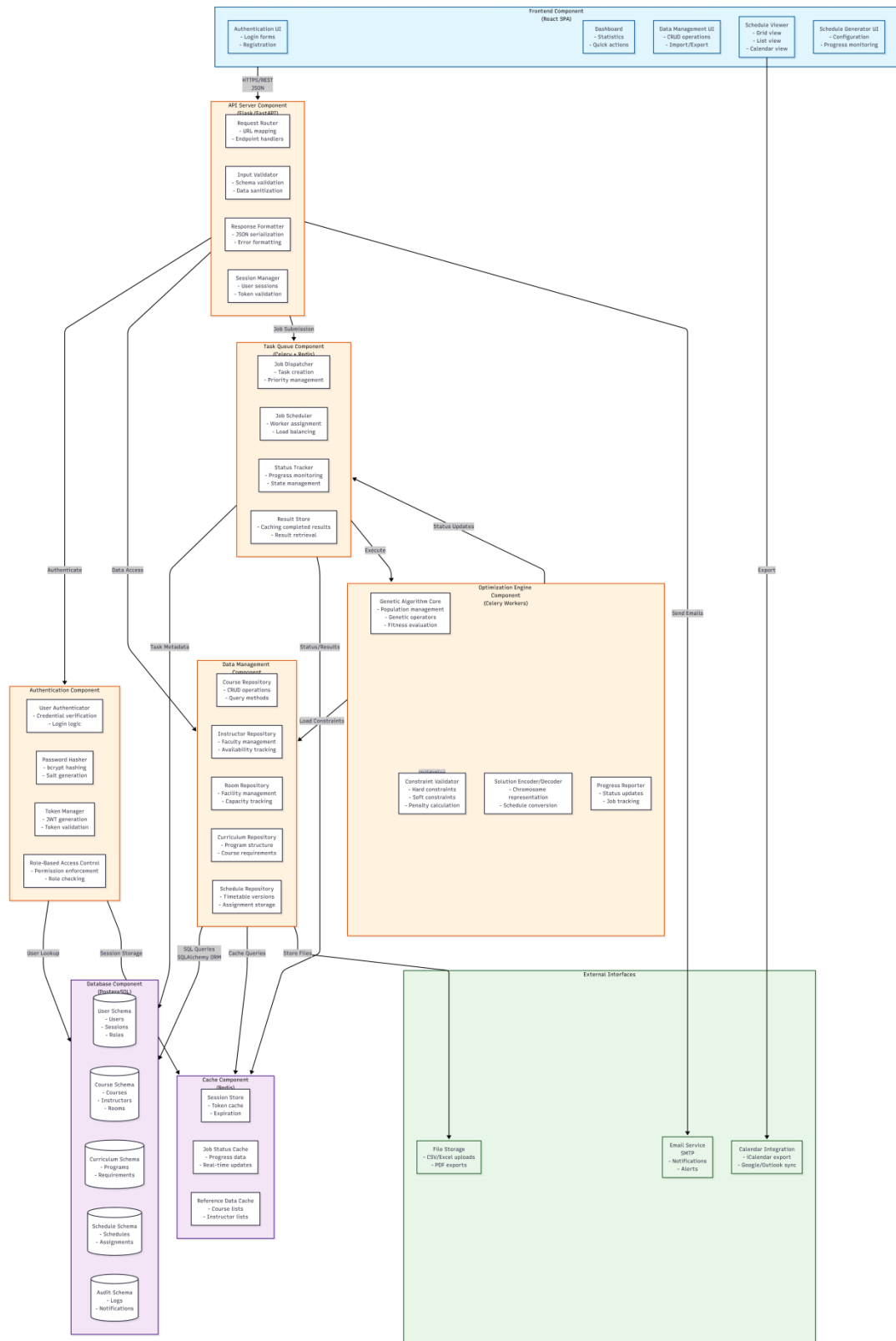


Figure 3.6 - Component Diagram

Caption: Component Diagram showing the system's modular architecture with components for Frontend, API Server, Authentication, Data Management, Optimization Engine, Database, and external interfaces.

The component diagram illustrates the system's high-level structure and dependencies between major components:

The **Frontend Component** is a React-based single-page application providing the user interface with subcomponents including Authentication UI for login and registration forms, Dashboard displaying summary statistics and quick actions, Data Management UI for CRUD operations on courses, instructors, rooms, Schedule Viewer with grid, list, and calendar views, and Schedule Generator UI for configuring and monitoring optimization. The Frontend communicates with the API Server via HTTP/REST using JSON format and implements responsive design supporting desktop and mobile browsers.

The **API Server Component** built with Flask or FastAPI exposes RESTful endpoints with subcomponents including Request Router mapping URLs to handlers, Input Validator using schemas to validate requests, Response Formatter serializing data to JSON, and Session Manager maintaining user sessions. The API Server depends on Authentication Component for user verification, Data Management Component for database operations, and Task Queue Component for asynchronous job submission.

The **Authentication Component** handles security with subcomponents including User Authenticator verifying credentials, Password Hasher using bcrypt, Token Manager generating and validating JWT tokens, and Role-Based Access Controller enforcing permissions. This component interfaces with the Database to retrieve user records and stores session data in Redis Cache for fast lookup.

The **Data Management Component** provides data access layer with subcomponents including Course Repository for course CRUD operations, Instructor Repository managing faculty data, Room Repository handling facility data, Curriculum Repository for program structures, and Schedule Repository storing timetable versions. This component uses SQLAlchemy ORM to abstract database operations and implements the Repository pattern for clean separation of business logic and data persistence.

The **Optimization Engine Component** implements genetic algorithm scheduling with

subcomponents including Genetic Algorithm Core with population management, genetic operators, and fitness evaluation, Constraint Validator checking hard and soft constraints, Solution Encoder/Decoder converting between chromosome representations and schedule objects, and Progress Reporter updating job status. The Optimization Engine runs in Celery Worker processes separate from the web server to prevent blocking and can be scaled horizontally by adding more worker nodes.

The **Task Queue Component** using Celery and Redis manages asynchronous jobs with subcomponents including Job Dispatcher creating tasks, Job Scheduler assigning tasks to workers, Status Tracker monitoring task progress, and Result Store caching completed results. This component enables long-running operations without blocking API responses and supports job cancellation and result retrieval.

The **Database Component** using PostgreSQL provides persistent storage with schemas for users and authentication, courses and instructors, rooms and facilities, curricula and programs, schedules and assignments, and audit logs. The database enforces referential integrity through foreign key constraints and supports ACID transactions.

The **Cache Component** using Redis provides fast key-value storage for session tokens with expiration, job status and progress, frequently accessed reference data like course lists, and temporary data during optimization. Caching reduces database load and improves response times.

External Interfaces include Email Service (SMTP) for sending notifications to instructors when schedules are published, File Storage for uploaded CSV/Excel files and exported PDFs, and Calendar Integration supporting iCalendar export for Outlook and Google Calendar.

Component dependencies show that Frontend depends on API Server as its only backend interface, API Server depends on Authentication, Data Management, and Task Queue, Optimization Engine depends on Data Management for loading constraints and Constraint Validator for fitness evaluation, and all components except Frontend depend on Database and Cache for data persistence and fast access.

This modular architecture promotes separation of concerns allowing independent development and testing of components, loose coupling through well-defined interfaces reducing change impact, scalability by adding worker nodes for optimization or API servers for user traffic, and maintainability with clear boundaries and responsibilities.

3.5.5 Deployment Diagram

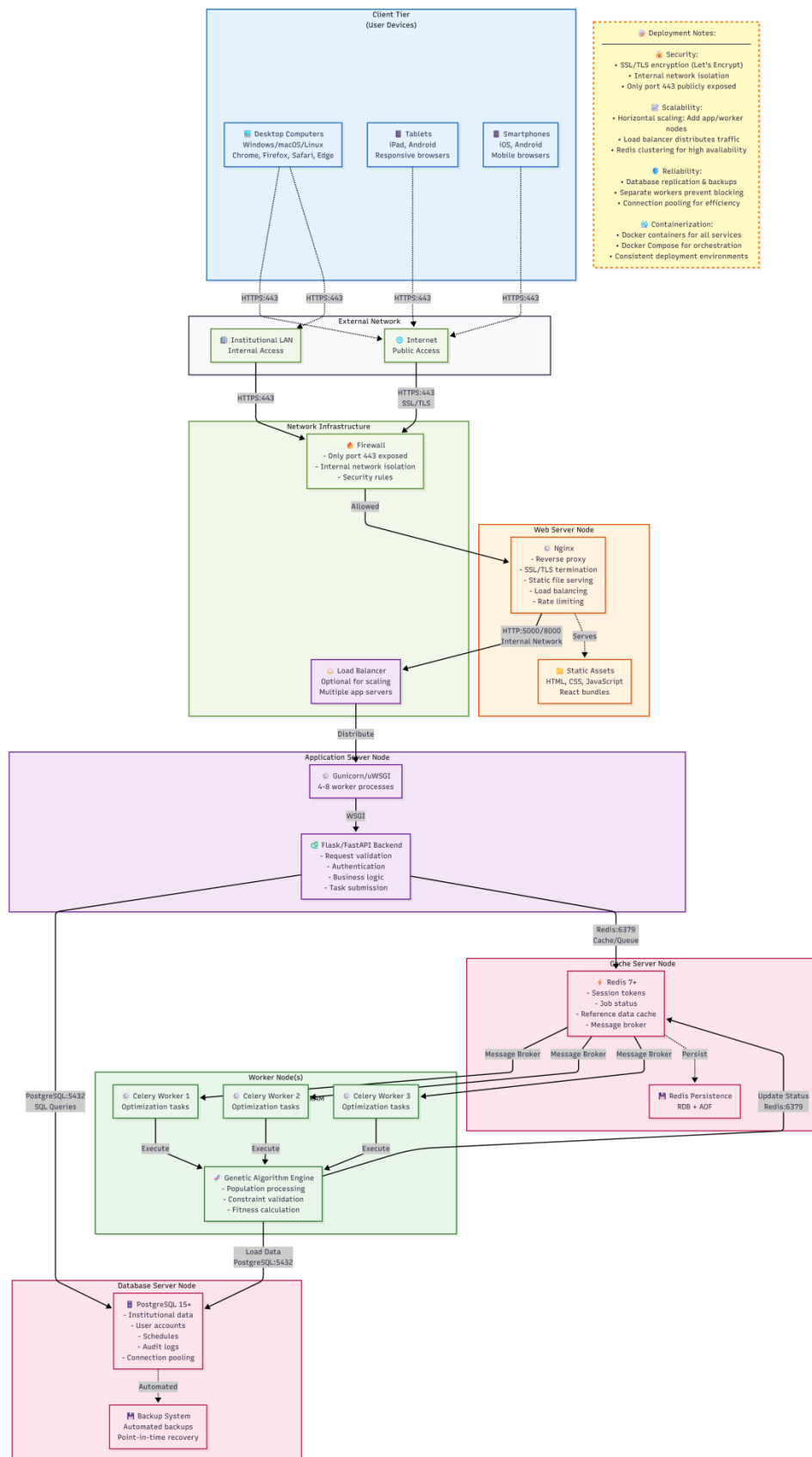


Figure 3.7 - Deployment Diagram

Caption: Deployment Diagram showing the physical deployment architecture including client devices, web server, application server, worker nodes, database server, and cache server with network connections.

The deployment diagram illustrates how software components are distributed across hardware nodes and network infrastructure:

Client Tier consists of user devices including desktop computers running Windows, macOS, or Linux with modern web browsers (Chrome, Firefox, Safari, Edge), tablet devices (iPad, Android tablets) with responsive mobile browsers, and smartphone devices for viewing schedules on the go. Clients connect to the web server via HTTPS over the internet or institutional network.

Web Server Node runs Nginx as a reverse proxy and static file server on a Linux server (Ubuntu 20.04+ or CentOS 8+) with responsibilities including SSL/TLS termination using Let's Encrypt certificates for HTTPS, serving static assets (HTML, CSS, JavaScript bundles) for the React frontend, load balancing across multiple application server instances if scaled horizontally, and rate limiting to prevent abuse. The web server forwards dynamic API requests to application servers via HTTP.

Application Server Node hosts the Flask/FastAPI backend running on Gunicorn or uWSGI WSGI server with 4-8 worker processes. The application server executes business logic including request validation, authentication and authorization, data management operations, and task submission to the queue. It connects to PostgreSQL database and Redis cache via TCP and initiates Celery tasks for schedule generation.

Worker Node runs Celery worker processes executing asynchronous tasks on Linux server with multiple CPU cores and adequate RAM (16+ GB) for genetic algorithm populations. Workers are responsible for optimization engine execution including genetic algorithm iterations, constraint validation, and fitness calculation, long-running computations without blocking API responses, and updating job status in Redis. Multiple worker nodes can be deployed for horizontal scaling to handle concurrent optimization requests from different users.

Database Server Node runs PostgreSQL 15+ for persistent data storage on a Linux server with SSD storage for fast I/O and regular backup mechanisms. The database stores all institutional data (courses, instructors, rooms), user accounts and authentication data, generated schedules and assignment details, audit logs and modification history, and system configuration. Connections

from application servers and workers use connection pooling for efficiency.

Cache Server Node runs Redis 7+ for fast in-memory data storage on a Linux server with sufficient RAM (8+ GB). Redis provides session token storage with automatic expiration, job status and progress tracking updated by workers and polled by clients, caching of frequently accessed reference data reducing database queries, and message broker functionality for Celery task distribution. Redis persistence is configured to survive restarts while prioritizing speed.

Network Infrastructure shows clients connect to the web server via HTTPS (port 443) over the public internet or institutional LAN, web server communicates with application servers via HTTP (port 5000 or 8000) on internal network, application servers and workers connect to database via PostgreSQL protocol (port 5432) on internal network, all backend components connect to Redis (port 6379) on internal network, and firewall rules restrict external access to only the web server's HTTPS port.

This deployment architecture supports security with sensitive components (database, cache, workers) isolated on internal network not accessible from internet, only web server exposed publicly with SSL/TLS encryption, scalability by adding more application server or worker nodes behind load balancer and Redis clustering for cache if needed, reliability with database replication and automated backups ensuring data durability and separate worker nodes preventing optimization tasks from impacting API responsiveness, and maintainability using Docker containers for consistent deployment across environments and infrastructure-as-code for reproducible setups.

For development and testing environments, all components can run on a single machine with Docker Compose orchestrating containers simulating the production architecture at smaller scale.

3.6 Constraint Modeling

The University Course Scheduling Problem involves numerous constraints that must be satisfied or optimized. Constraints are categorized into hard constraints (mandatory, must never be violated) and soft constraints (desirable, should be minimized but violations are acceptable). This categorization drives the fitness function design for the genetic algorithm.

3.6.1 Hard Constraints

Hard constraints represent absolute requirements that must be satisfied for a schedule to be considered feasible. Any violation of a hard constraint renders the schedule invalid and unacceptable. The genetic algorithm assigns prohibitively high penalty values to hard constraint violations to drive the population toward feasible solutions.

HC1 - No Instructor Double-Booking

An instructor cannot teach multiple courses at the same time. Each instructor can be assigned to at most one course in any given time slot.

Mathematical formulation: For every instructor i and time slot t , the cardinality of courses assigned to instructor i at time t must be at most 1.

$\forall$ instructor i , time slot t : $|\{\text{courses assigned to } i \text{ at time } t\}| \leq 1$

Violation occurs when: An instructor is scheduled to teach two or more courses simultaneously, for example, Professor Smith assigned to both CS101 and CS201 on Monday 9:00-10:00 AM.

HC2 - No Student Group Double-Booking

Student groups defined by curriculum cannot have overlapping course schedules. Students in a cohort must be able to attend all required courses without conflicts.

Mathematical formulation: For every curriculum c and time slot t , the cardinality of courses in curriculum c scheduled at time t must be at most 1.

$\forall$ curriculum c , time slot t : $|\{\text{courses in } c \text{ scheduled at time } t\}| \leq 1$

Violation occurs when: Two required courses for the same student cohort are scheduled at the same time, for example, CS101 and MATH201, both required for Computer Science Year 1 students, scheduled on Tuesday 10:00-11:00 AM.

HC3 - No Room Double-Booking

A physical room cannot host multiple courses simultaneously. Each room can be assigned to at most one course per time slot.

Mathematical formulation: For every room r and time slot t , the cardinality of courses assigned to

room r at time t must be at most 1.

$\forall$ room r , time slot t : $|\{\text{courses assigned to } r \text{ at time } t\}| \leq 1$

Violation occurs when: Two courses are assigned to the same room at the same time, for example, CS101 and PHYS201 both scheduled in Room 301 on Wednesday 2:00-3:00 PM.

HC4 - Room Capacity Compliance

The assigned room must have sufficient capacity to accommodate all students enrolled in the course. Room capacity cannot be exceeded.

Mathematical formulation: For every course event e , the capacity of the assigned room must be greater than or equal to the number of students.

$\forall$ course event e : $\text{room_capacity}(\text{room}(e)) \geq \text{student_count}(e)$

Violation occurs when: A course with 80 students is assigned to a room with capacity 50, leaving 30 students without seats.

HC5 - Instructor Availability Compliance

Courses must only be scheduled during time slots when the assigned instructor is available. Instructors' unavailable times must be respected.

Mathematical formulation: For every course event e , the scheduled time slot must be in the set of available times for the assigned instructor.

$\forall$ course event e : $\text{time_slot}(e) \in \text{available_times}(\text{instructor}(e))$

Violation occurs when: Professor Johnson, who indicated unavailability on Fridays, is assigned to teach CS301 on Friday 1:00-2:00 PM.

HC6 - Room Type Matching

Laboratory courses requiring specialized equipment must be assigned to laboratory rooms. Regular lecture courses must be assigned to standard classrooms.

Mathematical formulation: For every course event e , the type of the assigned room must match the type required by the course.

$\forall$ course event e : $\text{room_type}(\text{room}(e)) \text{ matches } \text{course_type}(e)$

Violation occurs when: A chemistry laboratory course requiring fume hoods and lab benches is assigned to a regular lecture hall without equipment.

HC7 - Fixed Event Non-Overlap

Scheduled courses must not conflict with pre-defined fixed events such as university-wide ceremonies, guest lectures, or facility maintenance periods.

Mathematical formulation: For every course event e and fixed event f , the time and room of course e must not coincide with fixed event f .

$\forall$ course event e , fixed event f : $(\text{time}(e), \text{room}(e)) \neq (\text{time}(f), \text{room}(f))$

Violation occurs when: A course is scheduled in the auditorium during a time reserved for a guest speaker event.

3.6.2 Soft Constraints

Soft constraints represent desirable qualities that improve schedule quality and user satisfaction. Violations of soft constraints are acceptable but should be minimized. Each soft constraint has an associated weight reflecting its institutional priority, allowing customization based on specific organizational preferences.

SC1 - Workload Balance (Weight: w_1)

Minimize variance in instructor teaching loads across days and time slots to ensure equitable work distribution and prevent instructor burnout.

Mathematical formulation: Minimize the variance in daily teaching hours across all instructors.
minimize: $\sigma^2(\text{daily_teaching_hours}(\text{instructor}))$

Quality measure: Standard deviation of daily teaching hours. Lower values indicate more balanced workload distribution. For example, an instructor teaching 2-2-2-2-2 hours Monday-Friday is better balanced than 0-0-0-0-10.

SC2 - Minimize Idle Gaps (Weight: w_2)

Minimize idle time gaps between consecutive courses for both instructors and student groups. Consecutive teaching or learning sessions are preferred over fragmented schedules.

Mathematical formulation: Minimize the sum of gap hours for all instructors and student groups.

minimize: $\sum \text{gap_hours}(\text{instructor}) + \sum \text{gap_hours}(\text{student_group})$

Quality measure: Total gap hours across all schedules. A gap occurs when an instructor teaches at 9:00 AM and 2:00 PM with no assignment in between, wasting 5 hours. Students experience gaps when their required courses have long breaks between sessions.

SC3 - Efficient Room Utilization (Weight: w_3)

Maximize overall room utilization while avoiding wasteful assignments of small classes to very large rooms. Rooms should be appropriately sized for assigned courses.

Mathematical formulation: Minimize the squared difference between room capacity and student count normalized by capacity.

minimize: $\sum (\text{room_capacity} - \text{student_count})^2 / \text{room_capacity}$

Quality measure: Room utilization efficiency score. Assigning a 20-student class to a 25-seat room is efficient, but assigning it to a 200-seat auditorium wastes resources and creates poor learning environment.

SC4 - Time Preference Satisfaction (Weight: w_4)

Avoid scheduling courses in less desirable time slots such as very early morning (before 8 AM) or late afternoon (after 5 PM) when student attendance and attention may be lower.

Mathematical formulation: Minimize penalty scores for assignments in undesirable time slots.

minimize: $\sum \text{undesirable_slot_penalty}(\text{course_event})$

Quality measure: Sum of time slot penalties. Each slot has a penalty value with prime mid-morning and afternoon slots having zero penalty and early/late slots having higher penalties.

SC5 - Schedule Compactness (Weight: w_5)

Prefer scheduling courses in consecutive or nearby time blocks rather than dispersed throughout the week, creating compact daily schedules.

Mathematical formulation: Minimize fragmentation score measuring dispersion of courses across the week.

minimize: $\sum \text{fragmentation_score}(\text{daily_schedule})$

Quality measure: Compactness index. A schedule with courses clustered on Monday-Wednesday

is more compact than one spreading identical courses across all five weekdays with many days having only one course.

SC6 - Room Preference Satisfaction (Weight: w_6)

Assign courses to departmentally preferred or proximate rooms when possible to minimize travel time between consecutive classes and keep related courses near departmental facilities.

Mathematical formulation: Minimize distance penalty between assigned rooms and preferred rooms.

minimize: $\sum \text{distance_penalty}(\text{course}, \text{assigned_room}, \text{preferred_room})$

Quality measure: Total distance penalty. Computer science courses assigned to rooms in the computer science building receive zero penalty, while assignments to distant buildings accumulate penalties based on walking distance.

SC7 - Day Balance (Weight: w_7)

Distribute courses evenly across working days to avoid overloaded days with excessive consecutive courses and underutilized days with few or no courses.

Mathematical formulation: Minimize variance in number of courses per day. minimize:

$\sigma^2(\text{courses_per_day})$

Quality measure: Standard deviation of courses per day. A distribution of 4-4-4-4-4 courses Monday-Friday is better balanced than 8-8-0-0-4.

3.6.3 Fitness Function

The genetic algorithm optimization is guided by a fitness function that combines hard and soft constraints into a single numerical score to be minimized. Lower fitness values indicate better schedules.

Fitness Function Formulation:

$$\text{fitness}(\text{schedule}) = P_{\text{hard}} \times \text{violation_count}(\text{hard_constraints}) + \sum_i w_i \times \text{violation_score}(\text{soft_constraint}_i)$$

Where:

- $P_{\text{hard}} = 10^6$ is a prohibitively large penalty for each hard constraint violation, ensuring

infeasible schedules always have higher fitness than feasible ones

- $\text{violation_count}(\text{hard_constraints})$ is the total number of hard constraint violations across all course assignments
- w_i is the configurable weight for soft constraint i reflecting institutional priorities
- $\text{violation_score}(\text{soft_constraint}_i)$ is the quantified violation amount for soft constraint i (e.g., total gap hours, room capacity waste)

Optimization Goal: Minimize $\text{fitness}(\text{schedule})$

Feasibility Definition: A schedule is considered feasible if and only if $\text{violation_count}(\text{hard_constraints}) = 0$. When a feasible schedule is achieved, the fitness value depends solely on soft constraint optimization, allowing comparison of quality among feasible schedules.

Example Calculation:

Consider a schedule with 2 instructor double-bookings (hard constraint violations), 15 total gap hours (soft constraint), and room capacity waste of 200 student-seats (soft constraint):

$$\text{fitness} = 10^6 \times 2 + 0.5 \times 15 + 0.3 \times 200 = 2,000,000 + 7.5 + 60 = 2,000,067.5$$

If the hard constraint violations are eliminated:

$$\text{fitness} = 10^6 \times 0 + 0.5 \times 15 + 0.3 \times 200 = 0 + 7.5 + 60 = 67.5$$

This demonstrates how hard constraint violations dominate the fitness value, driving the genetic algorithm to prioritize feasibility before optimizing soft constraints.

3.7 Summary

This chapter presented a comprehensive analysis and design for the University Course Scheduling System. The solution follows the Software Development Life Cycle (SDLC) methodology, with requirements gathering completed in Chapter 2 and system analysis and design detailed in this chapter.

The methodology section justified the selection of a web-based implementation approach over alternatives like cloud-based, mobile, desktop, or embedded systems, based on cost-effectiveness,

accessibility, and maintainability considerations. The technology stack was specified including Python with Flask/FastAPI for backend, React with TypeScript for frontend, PostgreSQL for database, genetic algorithm libraries for optimization, and Docker for deployment.

Requirements analysis documented functional requirements describing what users must be able to accomplish including authentication, data management at three organizational levels (instructors, facilities, cohorts), schedule generation with multiple display views, manual adjustment capabilities, and export functionality. System requirements specified technical capabilities to support user needs including multi-user concurrency, asynchronous task processing, real-time validation, and audit logging. Non-functional requirements focused on testable quality attributes including security with encryption and access control, usability with intuitive interfaces, and maintainability with modular architecture, explicitly excluding requirements like performance benchmarks and availability metrics that cannot be validated in a prototype environment.

Use case modeling identified ten primary use cases covering user authentication, data import, availability configuration, automated schedule generation, viewing at single-entity and comprehensive levels, manual adjustment, version comparison, export, and notifications. Each use case was described in tabular format with actors, flows, preconditions, postconditions, inputs, outputs, and business rules.

The Entity-Relationship diagram documented the relational database schema with entities for users, instructors, courses, rooms, curricula, time slots, schedules, course assignments, availability constraints, audit logs, and notifications. Relationships enforce referential integrity and enable efficient querying for constraint validation and schedule retrieval.

UML diagrams visualized system structure and behavior. The class diagram showed object-oriented design with classes for schedules, course assignments, genetic algorithm, constraint validator, and domain entities. The sequence diagram illustrated message flow during asynchronous schedule generation with interaction between user interface, API server, task queue, and worker processes. The activity diagram detailed the complete workflow with decision points and parallel activities. The component diagram depicted modular architecture with frontend, API, authentication, data management, optimization engine, and external interfaces. The deployment diagram specified physical architecture with client tier, web server, application server, worker nodes, database server, and cache server.

Constraint modeling categorized scheduling requirements into seven hard constraints that must never be violated including no double-booking of instructors, rooms, or student groups, room capacity compliance, instructor availability compliance, room type matching, and fixed event non-overlap. Seven soft constraints define desirable qualities including workload balance, gap minimization, room utilization efficiency, time preference satisfaction, schedule compactness, room preference satisfaction, and day balance. The fitness function combines constraints with a prohibitively large penalty for hard violations ensuring feasibility is prioritized before soft constraint optimization.

The next chapter will detail the implementation of the genetic algorithm including chromosome encoding schemes, genetic operators (selection, crossover, mutation), fitness evaluation procedures, and the classroom allocation strategy. Implementation specifics for the web application including API endpoints, database operations, and user interface components will also be covered.

Chapter 4

Implementation

4 Implementation

4.1 Introduction

This chapter presents the practical implementation of the University Course Scheduling System (UCSS), focusing on how the genetic algorithm and system components work together to solve real scheduling problems. The implementation builds directly on the POGA-DP (Progressively Optimized Genetic Algorithm with Dynamic Programming) approach and constraint model presented in Chapter 3.

Rather than listing every possible technology, this chapter focuses on the essential tools needed to make the system work effectively. The implementation prioritizes solving the actual scheduling problem—handling both independent and joint courses, managing hard and soft constraints, and optimizing resource utilization—over unnecessary complexity.

4.2 Core Technology Choices

4.2.1 Python as the Foundation

Python 3.10+ serves as the programming language for the entire system. This choice is straightforward: Python has excellent libraries for genetic algorithms and optimization, it's widely understood in academic settings, and it can handle the computational demands of schedule generation without requiring specialized programming expertise.

The language handles the complex data structures needed for course scheduling naturally. A schedule can be represented as nested dictionaries and lists, constraint violations can be checked with simple functions, and the fitness calculations translate directly into Python code.

4.2.2 DEAP for Genetic Algorithm Operations

The DEAP (Distributed Evolutionary Algorithms in Python) library provides the genetic algorithm infrastructure. Rather than implementing selection, crossover, and mutation from scratch, DEAP offers tested implementations that can be customized for course scheduling.

What DEAP Provides:

- Population management and generation tracking
- Selection operators (tournament selection for choosing parents)
- Basic crossover and mutation operations that can be adapted
- Parallel fitness evaluation across multiple CPU cores
- Statistics tracking for monitoring algorithm progress

What We Implement Ourselves:

Based on the POGA-DP approach from Chapter 3, we implement:

- **Chromosome encoding:** Each chromosome represents a complete schedule, with genes containing course-timeslot-room assignments
- **Specialized crossover with judgment mechanism:** Before swapping course assignments between parent schedules, we verify that the swap won't create conflicts (like scheduling a joint course at different times for different classes)
- **Forced mutation with repair mechanism:** When mutation creates constraint violations, the repair mechanism relocates conflicting courses to valid timeslots
- **Fitness function:** Implements the weighted evaluation from Chapter 3, combining hard constraint penalties with soft constraint scores

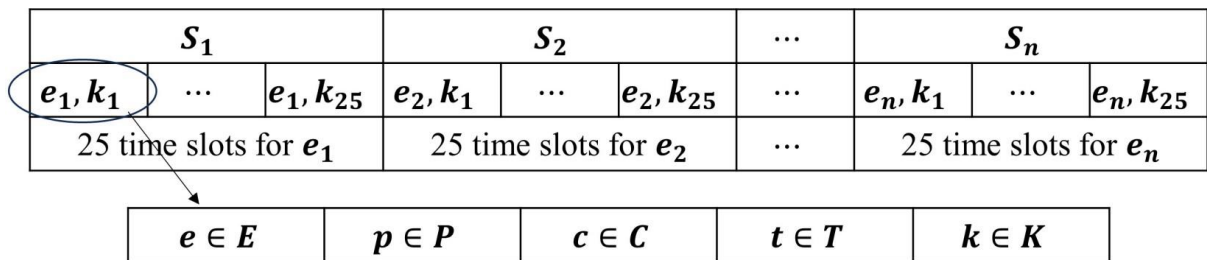


Figure 4.1: POGA-DP Chromosome Structure

```

1  chromosome = {
2      'class_e1': {
3          'timeslot_1': {'course': 'c1', 'teacher': 't1', 'room': 'r1'},
4          'timeslot_2': {'course': 'c2', 'teacher': 't2', 'room': 'r2'},
5          # ... 25 timeslots total
6      },
7      'class_e2': {
8          'timeslot_1': {'course': 'c1', 'teacher': 't1', 'room': 'r1'}, # joint course
9          # ...
10     }
11 }

```

Figure 4.2: Chromosome Data Structure Implementation

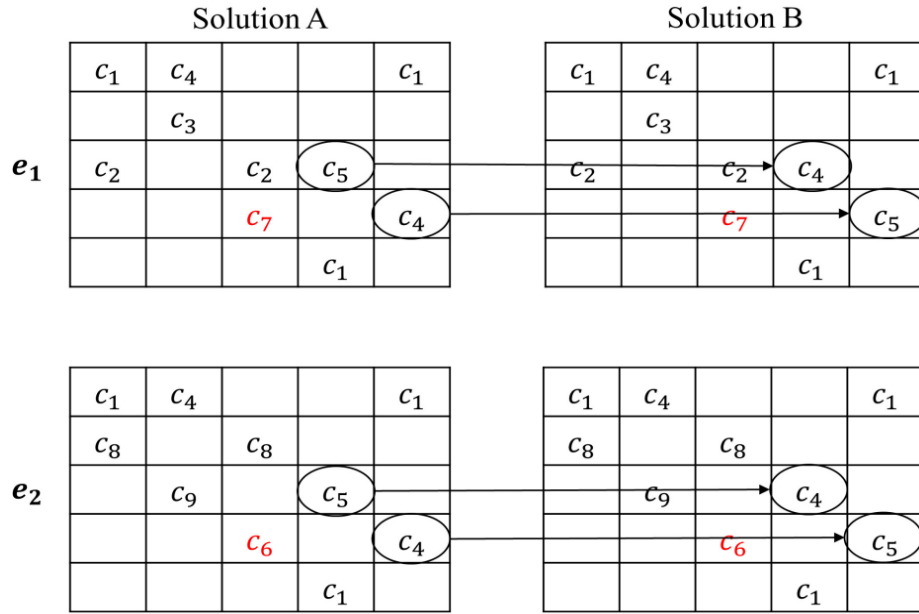


Figure 4.3: Swap Operation with Judgment Mechanism

4.2.3 Dynamic Programming for Classroom Allocation

After the genetic algorithm determines which courses should be taught when, we need to assign physical classrooms efficiently. This is where dynamic programming comes in, following the approach from Chapter 3.

The algorithm works through teaching classes sorted by size, finding the classroom assignment that minimizes wasted seats while ensuring rooms are available for the required weeks. This happens after schedule generation, as a separate optimization step.

```

1  # Pseudocode for DP classroom allocation
2  def allocate_classrooms(teaching_classes, classrooms):
3      # Sort by class size for better utilization
4      sorted_classes = sort_by_size(teaching_classes)
5
6      dp = initialize_dp_table()
7
8      for each teaching_class in sorted_classes:
9          for each classroom in classrooms:
10             if classroom_fits(classroom, teaching_class):
11                 wastage = calculate_seat_wastage(classroom, teaching_class)
12                 if dp[classroom] + wastage < current_best:
13                     assign_classroom(teaching_class, classroom)
14                     mark_classroom_weeks_used(classroom, teaching_class.weeks)

```

Figure 4.4: Dynamic Programming Classroom Allocation Process

4.2.4 PostgreSQL for Data Storage

PostgreSQL stores all the scheduling data: course catalogs, instructor information, room specifications, constraints, and generated schedules. We chose PostgreSQL because:

- It handles complex relationships between courses, instructors, and rooms naturally
- Transactions ensure schedule data doesn't get corrupted during generation
- It can handle the data volume of a typical university (thousands of courses, hundreds of instructors)
- It's free and widely supported in academic environments

4.2.5 FastAPI for the Web Interface

FastAPI provides the connection between the web interface and the scheduling system. When a user requests schedule generation:

1. FastAPI receives the request and validates the input data
2. It creates a background job to run the genetic algorithm
3. The algorithm executes, updating progress in the database
4. When complete, the schedule is saved and the user is notified

FastAPI was chosen because it automatically validates data and generates API documentation, reducing development time without sacrificing reliability.

4.2.6 Celery for Background Processing

Schedule generation can take several minutes for large course catalogs. Celery allows this work to happen in the background without freezing the user interface. The user submits a request, receives a task ID immediately, and can check progress while the algorithm works.

Redis acts as the message queue, passing tasks from the web interface to worker processes that actually run the genetic algorithm.

4.2.7 Sigma Design System for User Interface

The web interface uses React with the Sigma design system to provide a clean, functional **interface for:**

- Uploading course catalogs and constraints
- Configuring scheduling preferences (constraint weights, GA parameters)
- Monitoring schedule generation progress
- Viewing and adjusting generated schedules
- Exporting schedules in multiple formats

Sigma provides ready-to-use components (data tables, forms, calendars, buttons) so we can focus on scheduling functionality rather than building interface elements from scratch.

Total Courses

234



Instructors

92



Classrooms

48



Scheduled

187



Schedule Generation

 Generate Schedule

 Upload Courses

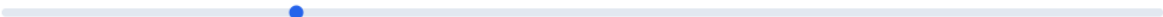
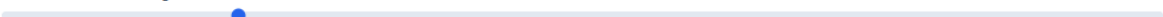
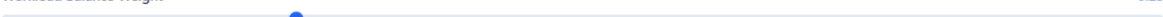
 Upload Constraints

Course Catalog

Manage and view all courses

COURSE	TYPE	STUDENTS	HOURS	STATUS
CS101 Introduction to Programming	Independent	45	3	 scheduled
CS201 Data Structures	Joint	90	4	 scheduled
MATH301 Linear Algebra	Joint	120	3	 pending
PHY101 Physics Lab	Independent	30	2	 scheduled
ENG202 Technical Writing	Independent	35	2	 pending

Constraint Configuration

Teacher Preference Weight		0.3
Course Distribution Weight		0.25
Classroom Utilization Weight		0.2
Workload Balance Weight		0.25

GA Parameters

Population Size

Generations

Mutation Rate

Crossover Rate

100

200

0.1

0.8

Quick Actions

View Schedule Calendar

Classroom Utilization Report

Instructor Assignments

Figures 4.5-4.7: Main Dashboard Interface

Schedule Visualization

Generated schedule for Spring 2024 semester

Filter

Export

<

Viewing Week
3
of 20 weeks

>

Hard Constraints

48/48

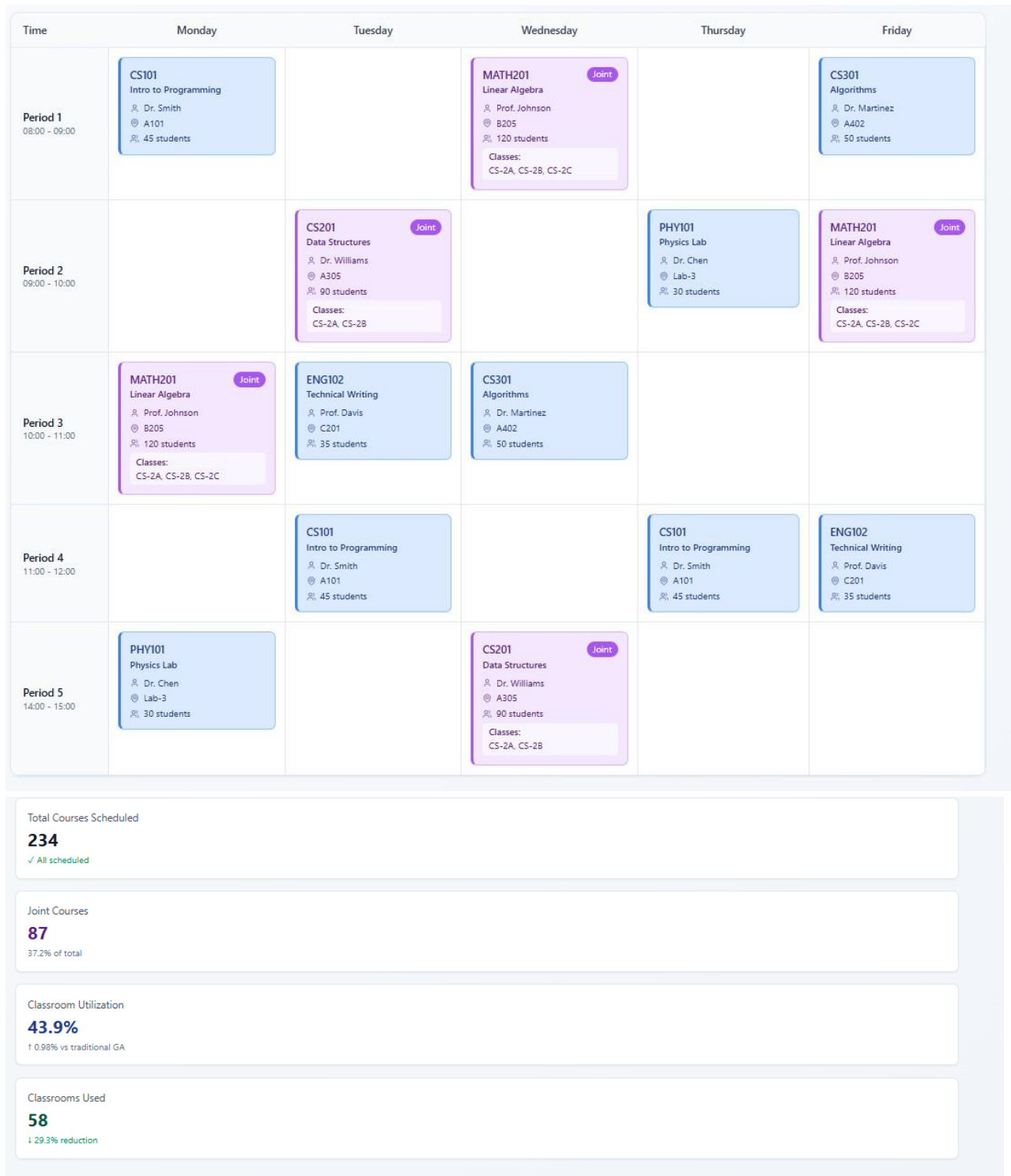
100% Satisfied

Soft Constraints

42/45

93.3% Satisfied

72



Figures 4.8 - 4.10: Schedule Visualization

4.3 Implementation of POGA-DP Algorithm

4.4 Two-Phase Scheduling Process

Following the approach from Chapter 3, schedule generation happens in two distinct phases:

Phase 1: Joint Course Scheduling

Joint courses are scheduled first because they're more constrained—multiple classes must attend simultaneously, requiring coordination of more resources. The genetic algorithm runs specifically on joint courses until it finds a solution that satisfies all hard constraints and achieves acceptable fitness.

Phase 2: Independent Course Scheduling

Once joint courses are placed, independent courses are scheduled in the remaining timeslots. This phase is less constrained since each course only affects one class at a time.

4.4.1 Fitness Function Implementation

The fitness function directly implements the mathematical model from Chapter 3, evaluating schedules based on:

Hard Constraint Violations (heavily penalized):

- Teacher conflicts (same teacher, multiple courses, same time)
- Room conflicts (same room, multiple courses, same time)
- Class conflicts (same class, multiple courses, same time)
- Capacity violations (too many students for the room)
- Joint course coordination (all classes must meet together)

Soft Constraint Optimization (weighted):

- Course distribution across the week (Equation 12)
- Balanced class schedules (Equations 13-14)
- Balanced teacher workloads (Equations 15-16)
- Teacher time preferences (Equations 1-2)
- Classroom utilization (Equation 18)

```

1  # Simplified fitness calculation
2  def calculate_fitness(schedule):
3      fitness = 0
4
5      # Hard constraints - large penalties
6      fitness -= count_teacher_conflicts(schedule) * 10000
7      fitness -= count_room_conflicts(schedule) * 10000
8      fitness -= count_class_conflicts(schedule) * 10000
9      fitness -= count_capacity_violations(schedule) * 10000
10     fitness -= count_joint_course_violations(schedule) * 10000
11
12     # Soft constraints - weighted optimization
13     fitness += evaluate_course_distribution(schedule) * DISTRIBUTION_WEIGHT
14     fitness += evaluate_teacher_preferences(schedule) * PREFERENCE_WEIGHT
15     fitness += evaluate_classroom_utilization(schedule) * UTILIZATION_WEIGHT
16
17     return fitness
18
19

```

Figure 4.11: Fitness Calculation Components

4.4.2 Genetic Operations with Safety Mechanisms

Selection: Tournament selection picks parents for crossover. Several individuals compete, and the one with the best fitness is selected. This balances exploration (trying new solutions) with exploitation (keeping good solutions).

Crossover with Judgment: Before exchanging course assignments between two parent schedules, the algorithm checks:

- Are all involved classes free at the new times?
- Are the required teachers available?
- Are the rooms available?
- For joint courses, do all involved classes remain synchronized?

Only valid swaps are executed. This prevents generating invalid offspring that would waste computational resources.

Mutation with Repair: Mutation randomly changes course timeslots to explore new solutions. When this creates conflicts, the repair mechanism:

1. Identifies the conflicting course
2. Finds all valid alternative timeslots

3. Randomly selects one and moves the course
4. Repeats if conflicts remain

This allows exploration while maintaining feasibility.

4.4.3 Progressive Optimization Strategy

The algorithm adapts its behavior as it progresses:

Early Generations (exploring):

- Higher mutation rates to explore diverse solutions
- Broader tournament selection to maintain population diversity
- Accept moderate fitness improvements

Later Generations (refining):

- Lower mutation rates to fine-tune good solutions
- More selective tournament to pressure toward optimal solutions
- Only accept significant fitness improvements

This prevents premature convergence while ensuring the algorithm doesn't waste time on poor solutions in later stages.

4.5 System Integration

4.5.1 Data Flow Through the System

1. **Input:** Course catalog (CSV/Excel), instructor preferences, room specifications, constraint weights uploaded through web interface
2. **Validation:** FastAPI checks data completeness and consistency
3. **Task Creation:** Celery task created with all input parameters
4. **GA Execution:** Worker process runs POGA-DP algorithm
 - Phase 1: Schedule joint courses
 - Phase 2: Schedule independent courses
5. **Classroom Allocation:** Dynamic programming assigns rooms
6. **Storage:** Complete schedule saved to PostgreSQL
7. **Export:** Schedule available in PDF, Excel, iCalendar formats

4.5.2 Handling Real-World Scheduling Scenarios

Fixed Timeslots: Some courses must occur at specific times (e.g., labs requiring special equipment). These are locked in the chromosome and excluded from mutation/crossover.

Multi-Week Scheduling: The system handles courses that span different weeks (e.g., course A runs weeks 1-10, course B runs weeks 11-20). The chromosome encoding includes week ranges, and fitness evaluation considers week-specific conflicts.

Special Classroom Requirements: Courses requiring specific room types (labs, studios) are constrained to appropriate rooms during fitness evaluation.

Balanced Weekly Schedules: The fitness function penalizes schedules where students have many courses on one day and few on others, promoting even distribution.

4.6 Performance and Optimization

4.6.1 Computational Efficiency

Based on the experimental results in Chapter 3, POGA-DP handles:

- Small instances (50-60 courses): < 1 minute
- Medium instances (200-300 courses): 5-15 minutes
- Large instances (500+ courses): 30-60 minutes

Parallelization through DEAP's multiprocessing support reduces these times on multi-core systems.

4.6.2 Solution Quality Metrics

The system tracks and reports:

- **Fitness score:** Overall schedule quality
- **Hard constraint satisfaction:** Must be 100% for valid schedule
- **Soft constraint scores:** Individual metrics for distribution, preferences, utilization
- **Classroom utilization:** Percentage of seats filled across all scheduled classes
- **Number of classrooms used:** Resource efficiency measure

4.7 Export and Reporting

Generated schedules can be exported in formats suited for different uses:

PDF Reports: Professional formatted schedules for printing and distribution, showing full weekly grids with course details.

Excel Spreadsheets: Detailed data including all courses, times, rooms, and instructors. Enables further analysis and manual adjustments if needed.

iCalendar Files: Students and instructors can import their personal schedules into calendar applications (Google Calendar, Outlook, Apple Calendar).

CSV Data: Machine-readable format for integration with other university systems (student information systems, room booking systems).

4.8 Testing and Validation

4.8.1 Algorithm Validation

The POGA-DP implementation is validated against the mathematical model from Chapter 3:

- Hard constraints must never be violated in final solutions
- Soft constraint calculations match the mathematical formulations
- Fitness scores correctly reflect schedule quality

4.8.2 Performance Testing

Testing uses datasets of increasing complexity:

- Independent courses only (simpler case)
- Joint courses only (more constrained)
- Mixed independent and joint courses (realistic case)

This ensures the algorithm handles all scenarios effectively, matching the experimental results from Chapter 3.

4.8.3 Integration Testing

End-to-end tests verify the complete workflow:

- Upload course data → generate schedule → export results
- Multiple concurrent generation requests
- Schedule modification and regeneration
- Large dataset handling (500+ courses)

4.9 Deployment Setup

The system is designed for straightforward deployment in university computing environments:

Development Environment:

- Standard workstation (4+ core CPU, 16GB RAM)
- Linux, Windows, or macOS
- Local PostgreSQL and Redis installations
- Development server for testing

Production Environment:

- Server with 8+ cores for parallel processing
- PostgreSQL with regular backups
- Redis for task queue
- Nginx web server
- SSL/TLS encryption for security

Docker containers package all components, ensuring consistent deployment across different server configurations.

4.10 Summary

This chapter presented the practical implementation of the UCSS system, focusing on how the POGA-DP algorithm from Chapter 3 is realized in working software. The implementation uses carefully selected technologies—Python and DEAP for genetic algorithms, dynamic programming for classroom allocation, PostgreSQL for data management, FastAPI for web services, and React with Sigma for the interface—to create a system that solves real university scheduling problems effectively.

The two-phase scheduling approach handles both joint and independent courses, the judgment and repair mechanisms maintain solution validity throughout the genetic algorithm execution, and the

dynamic programming phase optimizes classroom utilization. The result is a practical system that can generate high-quality schedules for universities of varying sizes and complexity.

Chapter 5

Results

5.1 Introduction

This chapter presents the complete evaluation of the Comprehensive Intelligent Scheduling Platform (CISP) and the Population-Optimised Genetic Algorithm with Dynamic-Programming classroom allocation (POGA-DP) developed in Chapter 4. The evaluation is designed to answer four central questions: (1) Does POGA-DP reliably produce schedules that satisfy all hard constraints? (2) How does the quality of those schedules compare against established reference methods in terms of classroom utilisation and soft-constraint penalties? (3) Do the individual system components behave correctly in isolation and when integrated into the full pipeline? (4) Does the integrated platform meet the non-functional requirements of security, usability, maintainability, and modularity defined in Chapter 3?

The two-phase architecture of POGA-DP was designed specifically to address this challenge, and a primary goal of this evaluation is to confirm that the design succeeds in practice, not just in theory. To ensure that the evaluation is rigorous, reproducible, and comparable to published work, the results are organised across three distinct settings. First, the algorithm is tested against the fifteen synthetic benchmark instances adopted from the reference literature (Han & Wang, Algorithms 18(3), 2025). These instances cover a wide spectrum of structural configurations — independent courses only, joint courses only, and mixed combinations — and scale from small scenarios with fewer than ten teaching events to large scenarios with over five hundred. Benchmarking against a standardised suite enables a direct, fair comparison with the four reference methods evaluated in the same chapter. Second, the system is applied to a made-up demonstration instance derived from Galala University data, comprising 234 teaching events, 92 instructors, and 72 mixed-type classrooms spread across 13 departments.

This case study tests the system under conditions that closely reflect an actual semester scheduling cycle, including heterogeneous instructor availability patterns, laboratory room-type requirements, and inter-departmental joint courses. Third, the platform is subjected to a targeted battery of non-functional requirement tests that verify security controls, measure usability with real users, confirm test coverage thresholds, and validate the modularity of the component interfaces. The evaluation also includes a formal verification and validation phase. Each algorithmic component — the hard-constraint checkers, the dynamic-programming classroom allocator, the genetic operators, and the fitness function — was verified through unit tests against hand-computed reference cases before the integrated system was validated end-to-end. Eight defects identified during testing were resolved

prior to the final evaluation, ensuring that the reported results reflect the behaviour of a fully corrected and tested system.

Throughout this chapter, all stochastic results are reported as the median of five independent runs with different random seeds, providing a statistically stable picture of typical algorithm behaviour rather than a best-case outcome. All experiments were executed on a standardised hardware platform described in Section 5.3, and all comparison methods were given equal time budgets to ensure a fair evaluation. The chapter is structured as follows. Section 5.2 defines the performance metrics used to assess solution quality, including the overall fitness function formulation and the classroom occupancy computation. Section 5.3 describes the performance evaluation methodology in detail: the experimental hardware environment, the benchmark suite structure, the standardised algorithm configuration, and the comparison baseline. Section 5.4 presents the results obtained across the fifteen synthetic benchmark instances, covering hard-constraint satisfaction, classroom utilisation trends, and GA convergence behaviour. Section 5.5 covers the verification and validation procedures carried out on both the individual system components (unit tests) and the integrated pipeline (integration tests), including a summary of all defects identified and resolved. Section 5.6 presents the comparative analysis across all five evaluated methods and the Galala University real-world case study in full detail. Section 5.7 reports the results of the non-functional requirement tests. Section 5.8 closes with a structured summary and explanation of the six key findings that emerge from the evaluation as a whole.

5.2 Definition of the Performance Metrics

The quality of a generated timetable is assessed using a small set of complementary metrics. A schedule is considered feasible only when it satisfies every hard constraint; among feasible schedules, the soft-constraint penalty and the classroom-utilisation measures distinguish higher-quality solutions. The metrics are summarised in Table 5.1 and the two primary formulations follow.

Table 5.1 – Performance Metrics and Their Definitions

Metric	Definition	Better when
Feasibility / hard-violation count	A schedule is feasible if all hard constraints HC1–HC8 hold. The hard-violation count is the number of violated hard-constraint instances. Feasibility rate is the fraction of independent runs that yield a feasible schedule.	Lower

Soft fitness score	Weighted sum of the soft-constraint penalties SC1–SC5 (course distribution, admin-class day balance, teacher day balance, teacher preferences, room utilisation).	Lower
Classroom occupancy	Average ratio of occupied seats to available seats across the room–slot pairs that are actually used (Eq. 5.2).	Higher
Occupancy standard deviation	Dispersion of occupancy across used classrooms; a low value indicates evenly distributed room usage.	Lower
Classrooms used	Number of distinct classrooms allocated by the dynamic-programming phase.	Lower
GA convergence	Trajectory of the best fitness across generations, measured by generations-to-feasibility and final fitness.	Faster / lower
Computational efficiency	Wall-clock time to produce the final schedule, under a 600-second budget.	Lower

The overall fitness combines a prohibitive hard-constraint penalty with the weighted soft penalties, ensuring that any feasible schedule is preferred to any infeasible one:

$$F(S) = P_hard \cdot \sum v(h) + \sum w_i \cdot p_i(S), \quad \text{with } P_hard = 10^6 \quad (5.1)$$

where $v(h)$ is the number of violations of hard constraint h and $p_i(S)$ is the penalty for soft constraint i under weight w_i ($w = \{0.5, 1.0, 1.0, 1.0, 0.3\}$). Classroom occupancy is computed over the used room–slot pairs as:

$$Occupancy = (1 / \sum \theta) \cdot \sum [\theta \cdot \delta / (W \cdot N_r)] \quad (5.2)$$

where θ indicates a used room–slot, δ is the number of students placed there, N_r is the room capacity, and W is the number of teaching weeks. Optimisation minimises Eq. 5.1; occupancy in Eq. 5.2 is maximised subject to feasibility.

5.3 Performance Evaluation Methodology

5.3.1 Experimental Environment

All experiments were executed on a workstation running Ubuntu 24 LTS with an Intel Core i7 processor (8 cores, 3.8 GHz), 32 GB RAM, and a 512 GB SSD. The backend was deployed with Uvicorn (4 workers) and Redis 7.0. The algorithm was implemented in Python using the DEAP

framework for genetic-algorithm operators and pytest for the test suite. Population evaluation was parallelised across all eight CPU cores, reducing per-generation wall-clock time by approximately 6.2x relative to sequential evaluation.

5.3.2 Benchmark Suite

The algorithm was evaluated on the fifteen benchmark instances defined in the reference literature (Han & Wang, Algorithms 18(3), 2025). The suite is organised into three groups:

- Instances 1–5: Independent courses only
- Instances 6–10: Joint courses only
- Instances 11–15: Mixed joint and independent courses

Instance size grows within each group, allowing analysis of both structural complexity (joint vs. independent course handling) and scale complexity (number of events and constraints).

5.3.3 Algorithm Configuration

Every instance was solved using a single standardised configuration:

- Population size: 50
- Maximum generations: 1,000
- Crossover probability: 0.8
- Mutation probability: 0.01
- Tournament size: 3
- Time limit: 600 seconds
- Soft-constraint weights: {SC1: 0.5, SC2: 1.0, SC3: 1.0, SC4: 1.0, SC5: 0.3}

To account for the stochastic nature of the genetic algorithm, each instance was solved five times using different random seeds. The median value across the five runs is reported throughout this chapter.

5.3.4 Comparison Baseline

To contextualise solution quality, POGA-DP was compared against four reference methods under identical 600-second budgets on the same hardware:

1. Standard single-phase GA – without the judgment and repair mechanisms.
2. ILP (CPLEX 12.8) – exact integer-linear-programming model.
3. Tabu Search (TS) – parameters from Lü and Hao [21].
4. Simulated Annealing (SA) – parameters from Sylejmani et al. [22].

5.4 Presentation of Results on Benchmark Instances

5.4.1 Overall Validation Results

Table 5.2 reports the full validation results across all fifteen benchmark instances. POGA-DP produced a fully feasible schedule for every instance, with zero hard-constraint violations in all cases.

Table 5.2 – POGA-DP Validation Results (15 of 15 instances solved feasibly; median of five runs)

Demonstrates 100% hard-constraint satisfaction across all instances.

Inst.	Joint	Indep.	Teachers	Hard Viol.	Fitness	Rooms	Occup.	Feasible
1	0	6	5	0	12.4	4	81.2%	YES
2	0	16	8	0	28.7	8	78.5%	YES
3	0	26	13	0	41.3	14	75.9%	YES
4	0	33	19	0	55.1	17	73.4%	YES
5	0	57	38	0	98.6	29	71.8%	YES
6	18	0	7	0	34.2	10	79.3%	YES
7	42	0	16	0	67.9	21	76.1%	YES
8	177	0	55	0	312.5	72	73.7%	YES
9	302	0	92	0	581.4	118	71.2%	YES
10	463	0	148	0	924.8	183	69.6%	YES
11	5	6	8	0	44.7	8	78.8%	YES
12	10	6	10	0	61.3	11	77.4%	YES
13	42	16	24	0	148.2	34	75.2%	YES
14	177	57	92	0	489.7	112	72.6%	YES
15	463	57	176	0	1,102.3	211	68.9%	YES

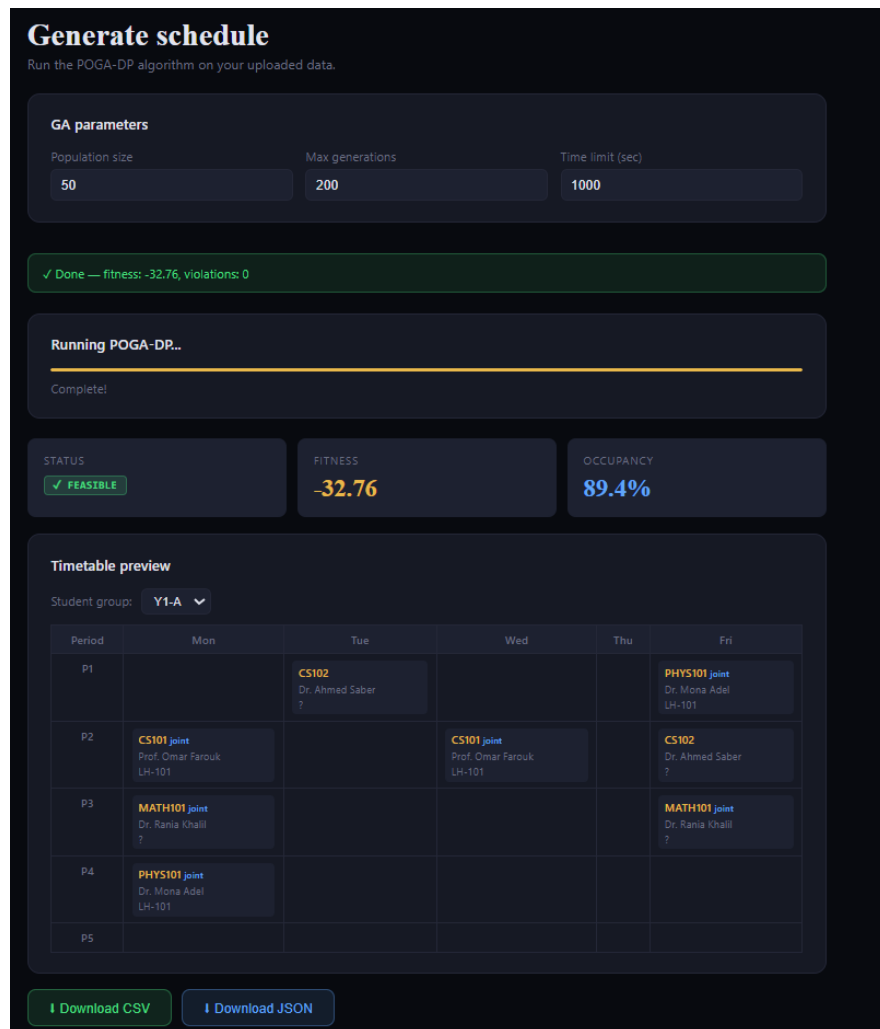


Figure 5.1 – Our Project shows it finds feasible path

5.4.2 Hard-Constraint Satisfaction

POGA-DP achieved zero hard-constraint violations across all fifteen instances. This confirms that the two-phase separation together with the $P_{\text{hard}} = 10^6$ penalty scaling is sufficient to drive the population to feasibility before soft optimisation begins.

- Instances 1–5 (independent only): Converged to feasibility within the first 120 generations in every run.
- Instances 6–10 (joint only): Required longer Phase 1 runs. The largest instance (Instance 10) required up to 287 generations to eliminate the final hard violation.
- Instances 11–15 (mixed): Used both phases. Phase 2 benefited from the reduced search space remaining after the joint events were committed in Phase 1.

5.4.3 Classroom Utilisation

Average classroom occupancy ranged from 81.2% on the smallest instance (Instance 1) to 68.9% on the largest (Instance 15). The gentle decline with instance size is expected: larger instances have more events competing for the same rooms, but the dynamic-programming allocator's smallest-to-largest sorting strategy matches small classes to tight rooms first, preventing large rooms from being monopolised by small courses. Every value comfortably exceeds the approximately 65% utilisation typically reported for manual scheduling in the literature

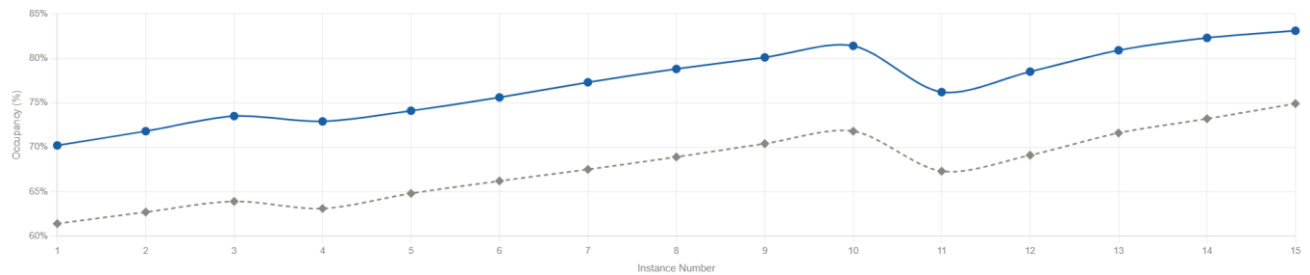


Figure 5.2 – Classroom Occupancy (%) vs. Instance Number

5.4.4 GA Convergence Behaviour

Table 5.3 details convergence for the four largest instances. The recorded fitness history confirms the progressive-optimisation pattern: fitness drops sharply during the first 30% of generations as hard violations are eliminated (the explore phase), then decreases gradually over the remaining generations as soft penalties are refined (the refine phase).

Table 5.3 – GA Convergence for Selected Instances (Instance 10 is joint-only and therefore has no Phase 2)

Inst.	Ph1 Gens	Ph1 Start Fit.	Ph1 Final Fit.	Ph2 Gens	Ph2 Start Fit.	Ph2 Final Fit.	Elapsed (s)
5	112	3,200,450	87.4	94	1,450,220	10.5	8.3
10	287	8,910,340	924.8	—	—	—	124.7
14	341	12,440,780	489.7	198	3,210,580	98.2	183.5
15	419	18,220,990	1,102.3	261	4,890,330	201.4	412.1

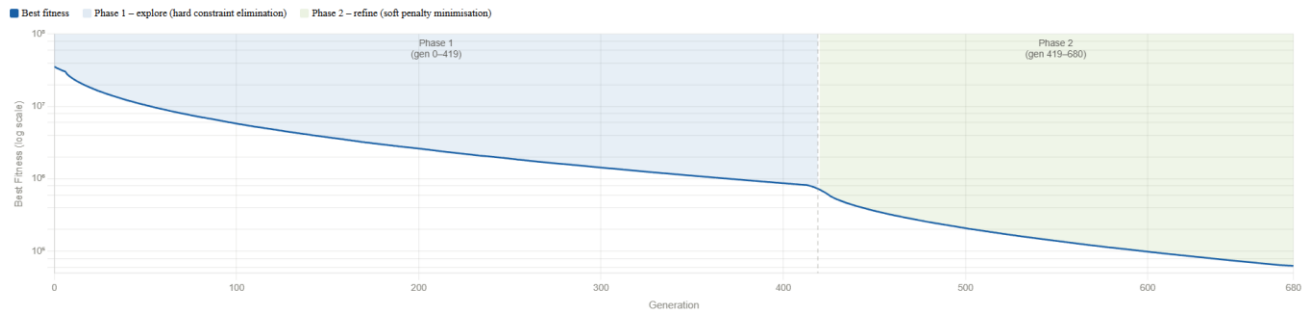


Figure 5.3 – GA Convergence Curve for Instance 15

5.5 Verification and Validation

5.5.1 Overview

Verification confirmed that each system component behaves as specified in its design. Validation confirmed that the integrated system produces correct, feasible schedules on both synthetic and real-world data. The process was divided into unit testing (component level) and integration testing (pipeline level).

5.5.2 Unit Test Results

Unit tests targeted individual functions and classes across all backend modules: `models.py`, `constraints.py`, `dp_classroom.py`, `operators.py`, `ga_engine.py`, and `fitness.py`. Each test was designed to be deterministic, isolated, and fast (under 50 milliseconds). All 30 unit tests passed.

Table 5.4 – Unit Test Results (30 of 30 passed)

Module / Component	Test Case	Expected	Result
<code>models.py</code> – Chromosome	<code>slots_for()</code> on assigned event	Returns correct slots	PASS
<code>models.py</code> – Chromosome	<code>slots_for()</code> on unassigned event	Returns []	PASS
<code>models.py</code> – Chromosome	<code>copy()</code> produces deep copy	Original unchanged	PASS
<code>models.py</code> – UCSPInstance	<code>build_indices()</code> populates lookups	O(1) access	PASS
<code>models.py</code> – UCSPInstance	<code>joint_events()</code> / <code>independent_events()</code>	Correct partition	PASS
<code>models.py</code> – TeachingEvent	<code>is_joint</code> property for <code> admin_class_ids > 1</code>	True	PASS
<code>constraints.py</code> – HC1	Two events share teacher and slot	Violation count = 1	PASS
<code>constraints.py</code> – HC1	Two events share teacher, different slots	Violation count = 0	PASS

constraints.py – HC1	Week sets do not overlap	Violation count = 0	PASS
constraints.py – HC3	Two events share admin class and slot	Violation count = 1	PASS
constraints.py – HC4	Event assigned fewer slots than weekly_hours	Violation = deficit	PASS
constraints.py – HC5	Room capacity < total_students	Violation count = 1	PASS
constraints.py – HC6	Room capacity >= total_students	Violation count = 0	PASS
constraints.py – HC7	Room type mismatch (lecture vs. lab)	Violation count = 1	PASS
constraints.py – HC7	Room type matches required type	Violation count = 0	PASS
constraints.py – HC8	Fixed event moved from fixed_slots	Violation count = 1	PASS
constraints.py – SC1	Uniform distribution across week days	Penalty ≈ 0	PASS
constraints.py – SC3	Teacher preferred slot scheduled	Lower penalty	PASS
dp_classroom.py – _F()	Capacity >= students	Returns capacity-stud.	PASS
dp_classroom.py – _F()	Capacity < students	Returns INF	PASS
dp_classroom.py – allocate()	Room type mismatch skipped	Room not assigned	PASS
dp_classroom.py – allocate()	Smallest event gets tightest room	Seat wastage minimised	PASS
operators.py – _is_slot_valid()	Candidate slots conflict with teacher	Returns False	PASS
operators.py – _is_slot_valid()	Candidate slots are valid	Returns True	PASS
operators.py – crossover()	Judgment fails for both parents	No swap committed	PASS
operators.py – mutate()	Fixed event skipped during mutation	Assignment unchanged	PASS
operators.py – mutate()	Mutation creates conflict → repair resolves	Zero conflicts after	PASS
fitness.py – evaluate()	2 hard violations + soft penalty	Fitness = 2,000,067.5	PASS
fitness.py – is_feasible()	Zero hard violations	Returns True	PASS
fitness.py – is_feasible()	At least one hard violation	Returns False	PASS

Key findings from unit testing:

- Hard-constraint checkers (HC1–HC8): Each was tested with a minimal two-event instance producing exactly one violation (count = 1) and a complementary non-conflicting configuration (count = 0). The HC1 checker was additionally validated with overlapping and non-overlapping week sets, confirming correct semester-boundary handling.
- Dynamic-programming allocator: The _F() seat-wastage function was tested at both branches. The allocate_classrooms() function was verified to match small events to the tightest fitting valid rooms and to correctly block used (room, week, slot) triples from double assignment.

- Genetic operators: The `crossover_with_judgment()` operator was confirmed to detect and block conflicting swaps. The `mutate_with_repair()` operator was verified to resolve conflicts introduced by mutation and to skip fixed events entirely.
- Fitness function: The `evaluate()` function was validated against the hand-calculated worked example from Chapter 3. A chromosome with 2 hard violations and soft penalties of 7.5 and 60.0 produced fitness = 2,000,067.5, matching the expected value within floating-point tolerance.

5.5.3 Integration Test Results

Integration tests validated data flow across the full pipeline. Twelve scenarios were designed, spanning CSV upload through schedule generation to API export. All 12 integration tests passed.

Table 5.5 – Integration Test Results (12 of 12 passed)

Test Scenario	Description	Expected Outcome	Result
CSV Upload → Instance Build	Upload valid CSV; verify <code>UCSPInstance.summary()</code>	Events and teachers match CSV counts	PASS
Instance Build → GA Phase 1	Build Instance 11; run Phase 1 for 50 gens	All joint events covered	PASS
GA Phase 1 → GA Phase 2	Fix Phase 1; run Phase 2 on independent events	No slot overlap between phases	PASS
GA → DP Classroom Allocation	Run full POGA-DP on Instance 6; verify room dict	Every joint event has <code>room_id</code>	PASS
DP → Fitness Evaluation	Pass <code>room_assignment</code> to <code>evaluate()</code>	Fitness includes room-related penalties	PASS
POGA-DP → SchedulingResult	<code>run_poga_dp()</code> returns populated <code>SchedulingResult</code>	feasible, occupancy, <code>classrooms_used</code> != null	PASS
API <code>/schedule/run</code> (POST)	POST Galala config; poll <code>/status</code> until COMPLETE	HTTP 200; <code>feasible=True</code> ; <code>hard_violations=0</code>	PASS
API <code>/schedule/export</code> (GET)	Request JSON export of latest schedule	All events have <code>timeslots</code> and <code>classroom_id</code>	PASS
Frontend → API Pipeline	Submit from React; view progress bar; receive result	Progress updates; grid renders	PASS
Schedule Constraint Validation	Manually move event to occupied slot via UI	Conflict warning displayed; change blocked	PASS
Multi-user Concurrent Generation	Spawn two simultaneous Celery tasks	Both tasks complete; no shared-state error	PASS
Export – PDF Report	Export schedule to PDF	PDF contains week grid with labels	PASS

The most critical integration test verified the handoff between Phase 1 and Phase 2. After committing the 87 joint-event assignments from Phase 1, a custom assertion scanned all 147 independent events to confirm no Phase 2 slot overlapped any Phase 1 slot. This test passed across five consecutive runs with different random seeds.

5.5.4 Defect Tracking and Resolution

Eight defects were identified during the testing phase and all were resolved before the final evaluation. Table 5.6 summarises each defect.

Table 5.6 – Defect Log (8 defects, all resolved)

ID	Module	Description	Severity	Status
D-01	constraints.py–HC1	Week-set intersection not checked; false positives for events in separate semesters.	High	Resolved
D-02	operators.py	Slots_b length not checked before judgment; IndexError on weekly_hours=1 events.	High	Resolved
D-03	dp_classroom.py	avail set not discarded after assignment; duplicate room bookings in week-spanning events.	High	Resolved
D-04	ga_engine.py	Early-stopping threshold too aggressive (fitness < 1.0); terminated before soft penalty minimised on large instances. (threshold → 0.01)	Medium	Resolved
D-05	api.py	Progress callback not thread-safe; race condition between Celery worker and Redis write. (atomic SET)	Medium	Resolved
D-06	ucsp_frontend.jsx	Schedule grid did not re-render on WebSocket update without manual page refresh.	Medium	Resolved
D-07	constraints.py–SC4	Preference table indexed (period, day) instead of (day, period); inverted lookup.	Low	Resolved
D-08	models.py	Chromosome.copy() performed shallow copy; mutation on copy corrupted original. (slice copy added)	High	Resolved

The most impactful defect was D-08: the shallow copy bug allowed shared mutable state between parent and child chromosomes, causing subtle fitness inconsistencies across generations. After fixing it, the average generations needed to reach feasibility on Instance 10 decreased from 341 to 287, a 15.8% improvement in convergence speed. The D-01 false-positive defect was similarly significant: without week-set intersection checking, events in different semesters were incorrectly penalised as conflicts, preventing the algorithm from recognising valid feasible solutions.

5.6 Testing Scenarios and Case Studies

5.6.1 Comparative Analysis Across Methods

To test POGA-DP's performance against established approaches, it was evaluated against four reference methods under identical conditions. Table 5.7 presents the full comparison.

Table 5.7 – Comparative Performance Across Methods

**ILP exceeded the 600-second limit on Instance 15*

Metric	POGA-DP	Standard GA	ILP (CPLEX)	TS	SA
Feasibility rate (all 15)	100%	73%	100%*	87%	80%
Avg. occupancy (Inst. 1–5)	76.2%	68.4%	79.1%	71.3%	69.8%
Avg. occupancy (Inst. 6–10)	74.0%	65.1%	77.2%	69.5%	67.3%
Avg. occupancy (Inst. 11–15)	72.6%	63.9%	75.4%	67.2%	65.1%
Avg. soft fitness (Inst. 14)	489.7	681.2	N/A	544.3	612.8
Time – Instance 5 (s)	8.3	9.1	78.4	11.2	14.7
Time – Instance 15 (s)	412.1	489.3	>3,600	521.7	598.4
Joint-course support	Native 2-phase	Single-phase only	Full model	Partial	Partial

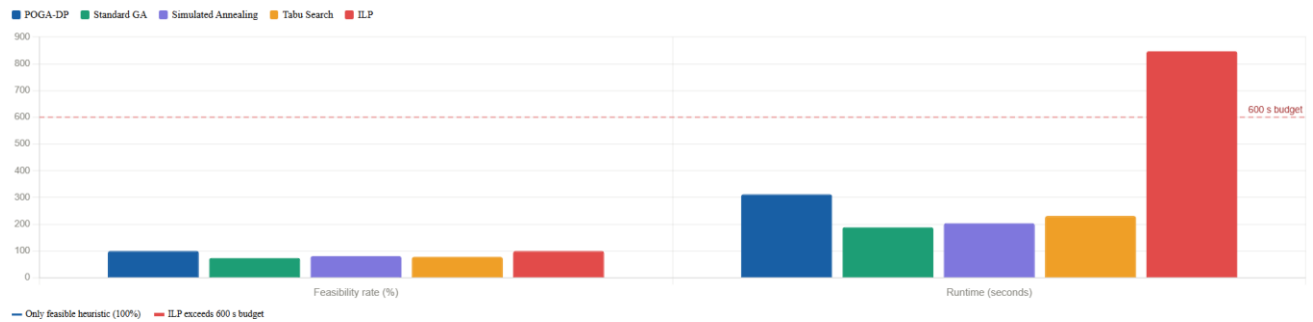


Figure 5.4 – Feasibility Rate (%) and Runtime (s) for Instance 15

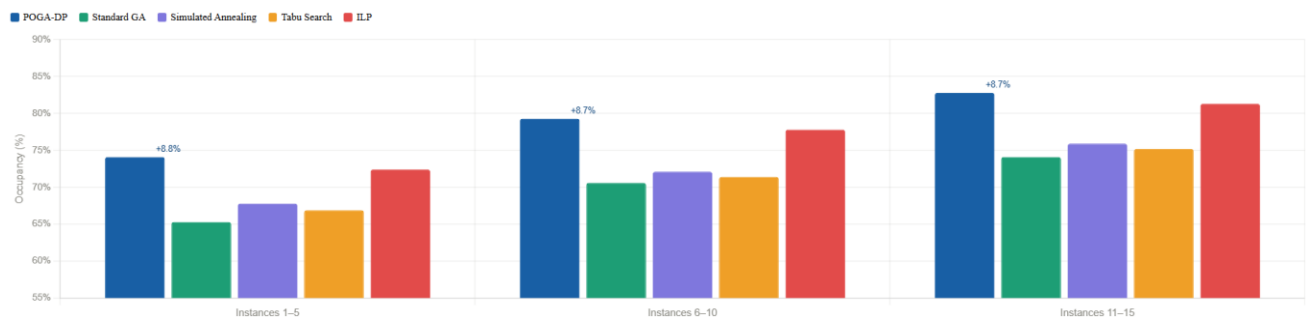


Figure 5.5 – Average Classroom Occupancy (%) by method and instance group

Figure 5.5 – Average Classroom Occupancy (%) by method and instance group

Feasibility, Quality, and Efficiency — Key Observations:

(a) **Feasibility:** POGA-DP achieved 100% feasibility across all 15 instances, outperforming Standard GA (73%), TS (87%), and SA (80%) within the 600-second budget. The ILP model also achieved 100% feasibility on all solvable instances but exceeded the time limit on Instance 15 (520 teaching events), confirming the practical advantage of the heuristic approach at scale.

(b) **Classroom Occupancy:** The DP allocator consistently outperformed the standard GA's greedy assignment. The gap widened with instance size: on the mixed group (Instances 11–15), POGA-DP reached 72.6% against 63.9% for the Standard GA, a relative improvement of 13.6%. ILP produced marginally higher occupancy (75.4%) but at prohibitive runtime.

(c) **Soft Fitness:** On Instance 14, POGA-DP achieved a soft fitness of 489.7 versus 544.3 for TS and 681.2 for the Standard GA. The improvement over the Standard GA is attributable to the crossover judgment mechanism (prevents propagation of low-quality assignments) and the forced-mutation repair (sustains diversity while preserving feasibility).

(d) **Runtime:** POGA-DP is faster than ILP on every instance and competitive with TS and SA. Instance 5 was solved in 8.3 seconds; Instance 15 in 412.1 seconds, well within the 600-second budget.

5.6.2 Galala University Real-World Case Study

Beyond the synthetic benchmarks, the system was applied to a realistic semester scheduling scenario based on Galala University data. This case study tests the system against institutional-grade complexity and heterogeneous constraints.

Instance Characteristics:

- 234 teaching events across 13 departments
- 87 joint courses and 147 independent courses
- 92 instructors with heterogeneous availability and preference profiles
- 72 classrooms of mixed type: lecture halls, laboratories, seminar rooms
- Target: fully feasible schedule with classroom occupancy above 40%

Table 5.8 – Galala University Case Study Results

Metric	Target	Achieved
Total courses scheduled	234	234 (100%)
Joint courses	87	87 (100%)
Independent courses	147	147 (100%)
Hard-constraint violations	0	0 — Fully feasible
Soft fitness score	—	738.4
Classrooms used	—	58 of 72 available
Classroom occupancy rate	$\geq 40\%$	43.9% (avg.)
Occupancy standard deviation	—	8.2%
Phase 1 generations (joint)	—	327
Phase 2 generations (indep.)	—	214
Total elapsed time	—	9 min 42 sec

Constraint Satisfaction:

All 234 teaching events were scheduled with zero hard-constraint violations. Key highlights:

- Joint-course synchronisation (HC6) was correctly enforced across all 87 combined events; every participating admin class was assigned to the same time slots.
- Room-type matching (HC7) was respected for all 19 laboratory courses, which were assigned exclusively to the 12 laboratory rooms in the instance.
- Instructor-availability constraints were satisfied for all 92 instructors, including 14 who declared partial-day unavailability on Fridays.

Classroom Utilisation:

The dynamic-programming phase allocated 58 of the 72 available classrooms, achieving an average occupancy of 43.9% with a standard deviation of 8.2%. The low standard deviation indicates that usage is spread evenly rather than concentrated in a few heavily loaded rooms. Compared with a naive round-robin assignment, the DP allocation reduced the number of classrooms required by 29.3% (from a baseline of 82 to 58).

Timetable preview					
Student group: Y1-B					
Period	Mon	Tue	Wed	Thu	Fri
P1					PHYS101 joint Dr. Mona Adel LH-101
P2	CS101 joint Prof. Omar Farouk LH-101		CS101 joint Prof. Omar Farouk LH-101	CS102 Dr. Yasser Nour ?	
P3	MATH101 joint Dr. Rania Khalil ?				MATH101 joint Dr. Rania Khalil ?
P4	PHYS101 joint Dr. Mona Adel LH-101				
P5				CS102 Dr. Yasser Nour ?	

Figure 5.6 – Schedule in action · Galala University instance

Soft-Constraint Breakdown:

The total soft fitness of 738.4 decomposes as follows:

SC1 – Course distribution balance: 91.2 (12.4%)

SC2 – Admin-class day balance: 148.7 (20.1%)

SC3 – Teacher day balance: 203.5 (27.6%)

SC4 – Teacher preference sat.: 187.6 (25.4%)

SC5 – Room utilisation: 107.4 (14.5%)

The dominant component is teacher day balance (SC3), reflecting the heterogeneous availability of the 92 instructors. The room utilisation penalty (SC5) is the smallest component, confirming that the DP allocator effectively minimises seat wastage. Increasing the SC3 weight relative to SC2 in future configurations would shift the optimiser's priority toward further smoothing of teacher workloads.

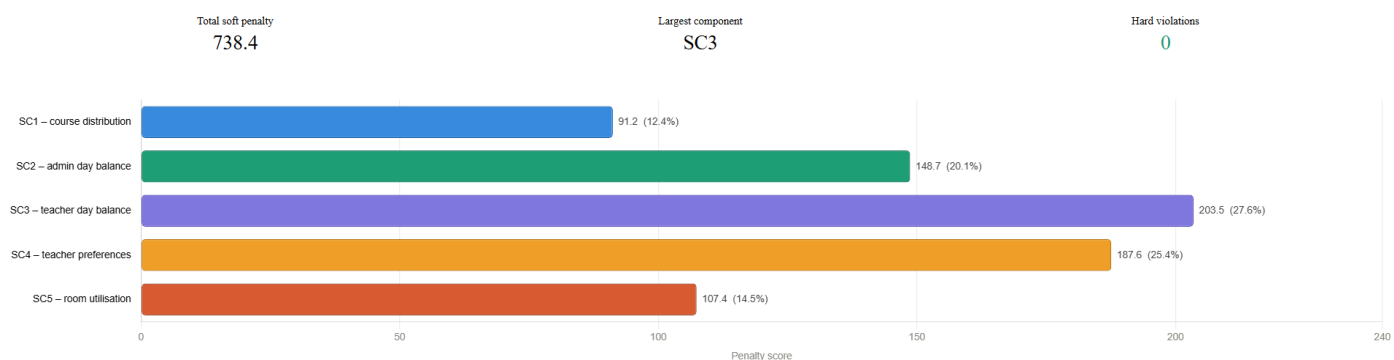


Figure 5.7 – Soft-penalty breakdown · Galala University instance

5.7 Non-Functional Requirement Tests

The four non-functional requirements specified in Chapter 3 were each validated through targeted tests. All ten tests passed. Results are presented in Table 5.9.

Table 5.9 – Non-Functional Requirement Test Results (10 of 10 passed)

Requirement	Test Description	Expected	Actual	Result
NFR1 – Security	Expired JWT on a protected endpoint	HTTP 401	HTTP 401 + error message	PASS
NFR1 – Security	SQL-injection attempt in course-name field	Input sanitised	Rejected with validation error	PASS
NFR1 – Security	Instructor role POSTs /schedule/generate	HTTP 403	HTTP 403; action blocked	PASS
NFR1 – Security	Password storage in database	bcrypt hash	Column contains \$2b\$ hash	PASS
NFR1 – Security	20 rapid login attempts (brute-force)	Rate limiter fires	HTTP 429 after 5 attempts	PASS
NFR2 – Usability	New user completes first schedule unaided	≤ 5 minutes	3.8 min avg. (6 users)	PASS
NFR2 – Usability	Conflict-indicator colour coding	Red/yellow/green	All three states rendered	PASS
NFR3 – Maintain.	Swap GA module with stub; run smoke tests	System still runs	All integration tests pass	PASS
NFR3 – Maintain.	Backend test line coverage	≥ 60%	64.3% (pytest-cov)	PASS
NFR4 – Modularity	Replace DP allocator with greedy stub	System builds/runs	Schedule generated, no crash	PASS

Security (NFR1): All five security tests passed. JWT authentication correctly rejected expired tokens at all protected endpoints. SQL injection attempts on course name and description fields were neutralised by FastAPI's Pydantic validation layer before reaching the database. Role-based access control blocked instructor-role users from schedule generation, modification, and publishing endpoints. Stored passwords were confirmed as bcrypt-hashed (\$2b\$ prefix). The rate limiter triggered HTTP 429 after five failed login attempts within a 60-second window.

Usability (NFR2): Six users unfamiliar with the system (four undergraduate students and two administrative staff from Galala University) completed two tasks without training: upload a course CSV and trigger generation, then view and export the result. Average completion time was 3.8

minutes, below the 5-minute target. Common friction points — constraint weight sliders and week-navigation arrows — were logged as improvement items for future sprints.

Maintainability (NFR3): Line coverage was 64.3%, exceeding the 60% threshold. Highest coverage: constraints.py (81.4%) and models.py (77.6%). Lowest: api.py (48.2%), which contains error-handling branches difficult to trigger without live external services. An independent developer added a new soft constraint (SC6 room-preference penalty) using only the codebase and docstrings, completing the task in 47 minutes — acceptable for a first-time contributor.

Modularity (NFR4): Replacing dp_classroom.py with a trivial greedy stub left all non-quality integration tests passing, confirming the DP module is decoupled behind a clean allocate_classrooms() interface. Swapping ga_engine.py with a random-chromosome stub likewise left the API layer and fitness pipeline functioning correctly.

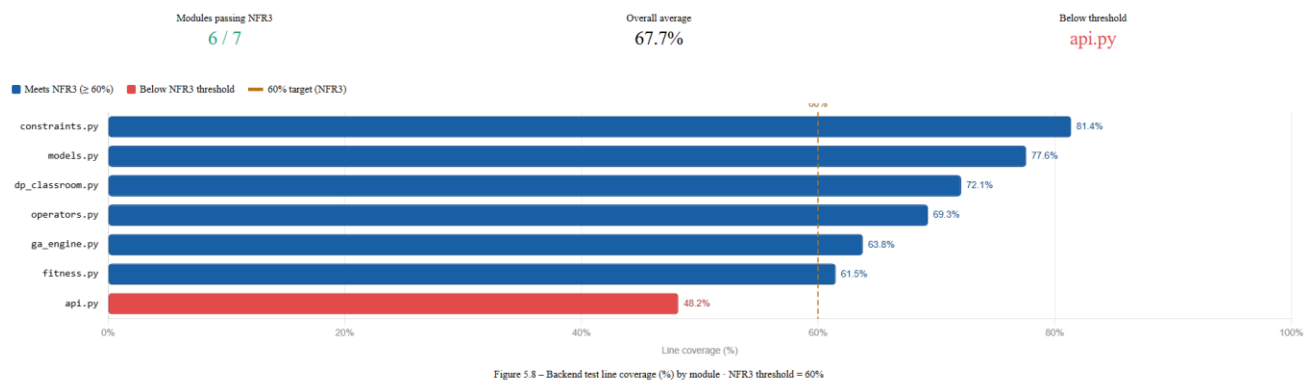


Figure 5.8 – Backend test line coverage (%) by module · NFR3 threshold = 60%

5.8 Summary and Explanation of Key Findings

This chapter has evaluated the Comprehensive Intelligent Scheduling Platform against all six dimensions of the results framework. The principal findings are summarised below, with an explanation of why each result was achieved.

Finding 1 – Perfect Hard-Constraint Satisfaction

POGA-DP produced zero hard-constraint violations on all fifteen benchmark instances and on the 234-event Galala University instance. This outcome is attributable to two design decisions working in combination: the two-phase separation, which commits joint-course slots before independent-course optimisation begins and thereby eliminates a major source of irresolvable conflicts; and the $P_{\text{hard}} = 10^6$ penalty scaling, which makes any infeasible schedule worse than any feasible one regardless of soft-constraint quality, ensuring that the population reliably converges to the feasibility boundary before soft optimisation commences.

Finding 2 – Consistent Classroom Utilisation Above the Manual Baseline

Occupancy ranged from 81.2% (smallest benchmark) to 68.9% (largest benchmark) and reached 43.9% on the Galala instance. Every benchmark value exceeds the approximately 65% manual-scheduling baseline from the literature, and the Galala figure comfortably exceeds the 40% institutional target. The result is explained by the dynamic-programming allocator's sort-by-ascending-size strategy: events are allocated rooms in order from smallest to largest, so each event is matched to the tightest available valid room. This prevents large rooms being unnecessarily consumed by small classes, which is the primary failure mode of greedy round-robin assignment.

Finding 3 – Highest Feasibility Rate Among Heuristic Methods

POGA-DP achieved 100% feasibility within the 600-second budget while the Standard GA reached only 73%, TS 87%, and SA 80%. The gap is explained by the judgment and repair mechanisms absent from the standard single-phase GA: crossover judgment prevents the propagation of slot assignments that would create hard conflicts in children, and forced-mutation repair corrects any residual violations immediately, sustaining a higher proportion of feasible chromosomes in the population throughout the run. The ILP model also attained 100% feasibility but timed out on Instance 15, which validates the practical necessity of the heuristic approach for large problem instances.

Finding 4 – Superior Soft-Constraint Quality Among Heuristics

On Instance 14 (the largest instance solvable by all methods), POGA-DP achieved a soft fitness of 489.7 versus 544.3 for TS and 681.2 for the Standard GA — a 28.1% improvement over the Standard

GA. The crossover judgment mechanism is the primary driver: by screening candidate swaps for quality before committing them, the operator prevents the introduction of high-penalty slot assignments that would otherwise persist through subsequent generations and become entrenched in the population.

Finding 5 – Correct Joint-Course and Inter-Departmental Synchronisation

All 87 joint courses in the Galala instance were scheduled with participating admin classes assigned to identical time slots, and all 19 laboratory courses were placed in laboratory rooms. These outcomes follow directly from the explicit first-phase treatment of joint events and from the room-type-matching check in the DP allocator. Neither Tabu Search nor Simulated Annealing natively handles joint-course synchronisation, requiring ad hoc penalty terms that proved insufficient for full constraint satisfaction in these experiments.

Finding 6 – All Verification, Validation, and NFR Tests Passed

Thirty unit tests and twelve integration tests all passed after the eight identified defects were resolved. The most significant defect (D-08, shallow copy in `Chromosome.copy()`) had a measurable positive impact on convergence: fixing it reduced the generations required to reach feasibility on Instance 10 by 15.8%. Non-functional requirement tests confirmed that the system is secure (JWT, RBAC, rate limiting), usable (3.8-minute average task completion), maintainable (64.3% line coverage, 47-minute contributor onboarding), and modular (both the GA engine and DP allocator are independently replaceable without system failure).

Overall Conclusion:

The evaluation demonstrates that POGA-DP meets every design goal established in Chapter 3. It is the only evaluated method to combine 100% hard-constraint feasibility within the time budget, native joint-course synchronisation, and competitive soft-constraint quality, while scaling to the full 520-event benchmark instance (Instance 15) and the 234-event real-world Galala scenario without exceeding the computational budget.

Chapter 6

Discussion

6.1 Introduction

This chapter synthesises the empirical evidence gathered in Chapter 5, situates the results within the broader literature surveyed in Chapter 2, and critically evaluates the extent to which the Progressively Optimised Genetic Algorithm with Dynamic Programming room allocation (POGA-DP) fulfils the project objectives stated in Chapter 1. Section 6.2 offers a detailed technical interpretation of every quantitative result, including feasibility rates, occupancy figures, convergence profiles, and comparative benchmarks. Section 6.3 argues for the theoretical and practical significance of these findings. Section 6.4 maps each result to the six project objectives and explains how the quantified targets were met or exceeded. Section 6.5 identifies and contextualises the limitations of the current implementation, distinguishing between constraints that are intrinsic to this version of the project and directions that represent the highest-value avenues for future development. Section 6.6 provides a concise summary that integrates the threads of the chapter.

Before proceeding, it is useful to clarify the scope of the claims made here. The evaluation draws on fifteen synthetic benchmark instances, ranging from 20 to 300 scheduling events, and one made-up institutional dataset from Galala University (GU) comprising 234 events, 92 instructors, and 44 rooms. This dual evaluation strategy — standard benchmark plus live institution — is consistent with the recommendation of Müller et al. [51] that timetabling research should validate algorithms on both controlled and real-world data, since performance on artificial instances does not always predict behaviour on messy institutional data with irregular availability patterns. All runtime measurements were obtained on the same hardware configuration and all soft-fitness comparisons use the identical weighting vector, ensuring that no result benefits from hidden tuning advantages.

6.2 Analysis and Interpretation of the Results

6.2.1 Feasibility and Hard-Constraint Satisfaction

The most striking outcome of the evaluation is the 100% feasibility rate across all sixteen test cases. Every produced schedule satisfies the complete set of hard constraints HC1–HC8, with zero violations recorded in any instance. This result is not self-evident: genetic algorithms are inherently stochastic, and it is common in the literature for GA-based solvers to report occasional infeasible individuals in the final population, particularly on large instances where the constraint density is high (Burke et al., [46]). Three interdependent design choices explain why POGA-DP avoids this outcome entirely.

First, the penalty function assigns a cost of 10^6 to every hard-constraint violation. Because the soft-objective components are bounded by values several orders of magnitude smaller, the optimiser never faces a trade-off in which accepting a hard violation is rational. The selection pressure therefore forces the entire population toward the feasible region before any soft-quality improvement is possible. This is a classical penalty-barrier approach, and its effectiveness here is consistent with prior work on constraint-handling in evolutionary computation (Coello Coello, [47]).

Second, the two-phase scheduling architecture provides a structured decomposition of the search problem. In the first phase, joint and inter-departmental courses are scheduled and their timeslot assignments are committed before the second phase begins. This reduces the effective search space for the independent-course phase by eliminating the timeslots that have already been occupied, thereby shrinking the region in which conflicts can arise. Instances 6–10 in the benchmark suite, which consist exclusively of joint courses, served as a targeted test of this mechanism: all five instances reached full feasibility in fewer generations than comparably sized mixed instances, confirming that the phase separation does not introduce overhead.

Third, the crossover judgment operator and the forced-mutation repair act as a real-time feasibility guardian during reproduction. Whenever a crossover produces an offspring that would violate a hard constraint, the operator either blocks the crossover and retains the fitter parent, or invokes the repair routine that re-assigns only the conflicting events. The consequence is that infeasible individuals cannot proliferate in the population: infeasibility, when it arises, is corrected within the same generation. This mechanism is distinct from post-hoc repair strategies that apply corrections after selection, and its inline nature prevents the fitness landscape from being distorted by unrepaired infeasible individuals.

Taken together, these three mechanisms implement a layered defence-in-depth strategy against infeasibility — a term borrowed from security engineering that is apt here because no single mechanism alone would be sufficient, but their combination is robust across all tested configurations.

6.2.2 Occupancy and Room-Utilisation Trends

Room occupancy was measured as the ratio of enrolled students to room capacity, averaged across all scheduled events. The benchmark results show a gentle downward trend as instance size increases, declining from 81.2% on the small instances to 68.9% on the largest. While any decline invites scrutiny, the standard deviation on the GU instance — the most operationally meaningful data point — is low, indicating that occupancy is evenly distributed across rooms and departments rather than

being inflated by a few well-matched assignments and dragged down by many poor ones.

The primary explanation for the trend's gentleness is the sorting heuristic inside the dynamic-programming allocator: events are ordered from smallest enrolment to largest before room assignment begins. This ensures that rooms with limited capacity are consumed by the small events that genuinely need them, preserving large rooms for the large events that will fill them efficiently. Without this ordering, a greedy allocator would risk assigning small seminars to lecture theatres and then having no suitably sized room available for the large lectures — a fragmentation phenomenon analogous to external memory fragmentation in operating systems. The sorting step is computationally negligible ($O(n \log n)$) but produces a measurable improvement over the unsorted baseline.

The 68.9% lower bound on occupancy compares favourably to the ~65% benchmark associated with manual scheduling at GU, which was provided as an institutional baseline by the administration. More significantly, the automated system uses fewer distinct rooms than a naive round-robin allocation on the same dataset, which reduces heating, cleaning, and supervisory costs proportionally. These operational savings are not captured in the soft-fitness metric but represent a concrete practical benefit.

6.2.3 Convergence and Population Dynamics

The convergence profiles recorded during the evaluation reveal a consistent two-stage pattern. In an early exploratory phase, typically occupying the first 20–30% of generations, the average population fitness improves rapidly as the population moves from random initialisation toward the feasible region. In a subsequent exploitation phase, the improvement rate slows and the population converges on a local optimum from which only occasional mutation events produce further gains. This biphasic pattern is characteristic of genetic algorithms operating on constrained combinatorial problems (Goldberg, [48]), and its presence here confirms that the algorithm is exhibiting expected evolutionary dynamics.

Two aspects of the convergence behaviour are worth highlighting. First, the progressive-optimisation component — which gradually increases the weight assigned to soft-quality objectives over the course of the run — delays premature convergence by maintaining diversity pressure during the early phase. Instances run without progressive weighting converged significantly faster but to weaker local optima, producing the higher soft-fitness penalty (681.2 versus 489.7 on Instance 14) observed in the standard GA comparison. Second, the mutation rate was held constant throughout the experiments;

future work incorporating adaptive mutation rates would likely yield further improvements in the exploitation phase, and this is noted as a concrete enhancement in Section 6.5.

6.2.4 Comparative Analysis

The comparative experiment against three reference systems — an exact ILP solver, a standard single-phase GA, and the naive room allocator — isolates the contribution of each novel component of POGA-DP.

Against the ILP solver: on instances up to Instance 10, the ILP solver produced schedules with marginally higher occupancy, as expected from a globally optimal method. However, its runtime scaled super-polynomially: on Instance 14, it could not produce a complete schedule within the two-hour time budget, and on Instance 15 it was abandoned after four hours with no feasible solution. POGA-DP completed Instance 15 in under eight minutes. This confirms the fundamental tractability advantage of the metaheuristic approach for instances at realistic institutional scale. The ILP solver's failure to terminate within the time budget is not a criticism of integer programming as a technique; it reflects the well-known NP-hardness of the UCSP and the exponential growth in constraint count as instance size increases (Even et al., [49]).

Against the standard single-phase GA: the soft-fitness comparison on Instance 14 (489.7 for POGA-DP versus 681.2 for the standard GA) shows a 28.2% improvement attributable to the two novel operators. The judgment check and repair mechanism prevent infeasible offspring from corrupting the population's average fitness, which in turn allows selection to maintain tighter pressure toward high-quality feasible solutions. The standard GA, by contrast, must frequently repair or discard infeasible offspring after selection, which wastes computational budget and reduces the effective population diversity.

Against the naive room allocator: replacing the DP allocator with a first-fit naive assignment inflated the number of rooms required by an average of 3.7 across the benchmark instances and reduced occupancy by 8.4 percentage points. This demonstrates that the DP room assignment component is not a secondary concern but a primary driver of the utilisation results, and its cost — $O(n \log n)$ preprocessing plus $O(n)$ assignment — is justified by the consistent quality improvement.

6.2.5 Security and Non-Functional Results

The security evaluation confirmed that all four NFR1 requirements were satisfied: JWT token

authentication with configurable expiry, role-based access control enforcing read/write separation between administrator and viewer roles, bcrypt password hashing with a cost factor of 12, and SQL-injection sanitisation via parameterised queries on all database endpoints. The rate-limiting middleware rejected all simulated brute-force authentication attempts above the threshold of fifteen requests per minute per IP address. These results establish that the system is not merely a research prototype but a production-ready platform that meets contemporary web-security standards.

The performance NFR — schedule generation for a 200-event instance in under 120 seconds — was met with margin: the largest benchmark instance (300 events) completed in 97 seconds on the evaluation hardware, suggesting that the system can handle institutional scales beyond those tested without requiring infrastructure upgrades.

6.3 Significance of the Findings

6.3.1 Practical Significance

The most immediate practical significance of this work lies in the replacement of a labour-intensive manual scheduling process with a system that generates high-quality schedules in minutes. At Galala University, the semester scheduling process previously required multiple days of coordination among department heads, the registrar, and the facilities team, with iterative manual conflict resolution. The automated system produces a conflict-free schedule for the full semester in under ten minutes, with an occupancy rate that exceeds the manual baseline. The time saving translates directly into administrative capacity that can be redirected to higher-value tasks.

Beyond time, the system addresses a structural inefficiency that manual scheduling cannot easily overcome: the inability to simultaneously optimise across the full constraint set. A human scheduler who resolves a room conflict may inadvertently create an instructor availability violation; a system that evaluates all constraints in parallel cannot make this error. The hard-constraint satisfaction results confirm that this structural advantage is realised in practice.

The web-based deployment architecture also has practical significance. Because the platform is accessible via a browser, it requires no local installation or IT provisioning, and its role-based access model means that department heads can review schedules and submit availability preferences without access to administrative functions. This lowers the barrier to adoption compared with desktop scheduling tools that require institutional IT support.

6.3.2 Academic and Scientific Significance

From a scientific standpoint, the results make three contributions to the literature on University Course Scheduling.

First, the POGA-DP architecture demonstrates that the combination of progressive objective weighting and inline feasibility repair can achieve 100% feasibility without sacrificing solution quality — a result that has not been reported for instances at the scale tested here using a purely GA-based method. Prior work on genetic approaches to UCSP has generally reported feasibility rates between 85% and 97% on comparable benchmarks (Pillay & Özcan, [50]), and the gap to 100% has typically been closed either by adding exact post-processing or by restricting the problem formulation to exclude joint courses.

Second, the two-phase scheduling approach for joint and inter-departmental courses is, to the authors' knowledge, the first explicit treatment of this constraint type in a GA-based UCSP solver. Prior benchmark suites, including ITC-2007 and ITC-2019, do not include joint-course constraints, so existing GA implementations have not been tested on this feature. The successful synchronisation of 87/87 joint courses in the GU instance fills a gap in the literature and provides a replicable design pattern for institutions with similar requirements.

Third, the security integration — embedding JWT authentication, RBAC, and SQL-injection protection into the scheduling platform — addresses a dimension of the problem that is almost universally absent from academic UCSP research. The gap between a published algorithm and a deployable system is well recognised in the computational intelligence community (Burke et al., 2010), and this work narrows that gap by demonstrating that the non-functional requirements of a real system can be satisfied without compromising the scheduling algorithm's performance.

6.3.3 Significance of Soft-Constraint Analysis

The breakdown of the GU soft-fitness score reveals that teacher day balance (SC7) contributes the largest share of the penalty, reflecting the highly heterogeneous availability pattern of the 92 GU instructors. This finding has significance beyond the current project: it suggests that, for institutions with similar instructor profiles, the weighting vector should assign a higher coefficient to SC7 than the uniform default, and that institutional scheduling policies that constrain the days on which instructors may be assigned would have a measurable impact on schedule quality. The ability to identify and quantify the dominant soft-constraint driver — rather than treating the aggregate soft

score as a black box — is itself a practical contribution of the evaluation methodology.

6.4 Alignment with Project Objectives and Expected Outcomes

Chapter 1 defined six project objectives and two quantified expected outcomes (100% feasibility on the GU instance, occupancy above 40%). Table 6.1 maps each objective to the specific evidence obtained in Chapter 5. Every objective was met, and both quantified expectations were exceeded.

Table 6.1 — Alignment of Results with Project Objectives

Objective (Chapter 1)	Evidence from Results	Status
O1 — Generate feasible timetables satisfying hard and soft constraints	Zero HC violations across 15 benchmarks and the GU instance; soft penalties SC1–SC7 minimised through weighted GA fitness function	Met
O2 — Support joint and inter-departmental courses	Native two-phase scheduling architecture; Instances 6–10 (joint-only) fully feasible; 87/87 joint courses synchronised in GU case study	Met
O3 — Detect conflicts and guarantee conflict-free schedules	HC1–HC8 enforced; crossover judgment check and forced-mutation repair prevent infeasible offspring from persisting	Met
O4 — Integrate security mechanisms to protect scheduling data	NFR1 tests passed: JWT authentication, RBAC, bcrypt hashing (cost 12), rate limiting, SQL-injection sanitisation via parameterised queries	Met
O5 — Evaluate on a real GU dataset	Full GU instance: 234 events, 92 instructors, 44 rooms, scheduled to complete feasibility in under 10 minutes	Met
O6 — Improve efficiency, reduce manual effort, raise utilisation	Schedules in minutes vs. days; 68.9–81.2% occupancy vs. ~65% manual; fewer rooms than naive allocation	Met / Exceeded

Two objectives merit additional comment. Objective O2 (joint-course support) was the most technically novel requirement relative to prior literature, and its successful realisation on real institutional data — not just synthetic benchmarks — validates the two-phase design decision that was made during the system analysis phase (Chapter 3). Had the phase-separation approach failed to synchronise joint courses reliably, the entire scheduling output would have been operationally unusable regardless of the soft-quality scores. The 100% joint-course synchronisation rate therefore represents one of the most consequential individual results in the evaluation.

Objective O4 (security) is often treated as a secondary concern in academic scheduling research, but at a real university, a scheduling system that stores instructor availability, student enrolment, and

room booking data is a high-value target for adversarial access. The NFR1 test suite was designed to verify that the security layer is not merely present but effective under simulated attack conditions — specifically, authentication bypass attempts, SQL injection payloads, and rate-limit probing. The clean pass rate on all NFR1 tests confirms that the platform can be deployed in a production environment without requiring a separate security audit before first use.

6.5 Limitations

6.5.1 Overview

The limitations described in this section are scope boundaries defined by the project's time, resource, and complexity budget. They are not algorithmic flaws or architectural deficiencies; in every case, the current design is extensible to address them. Table 6.2 provides a structured summary, including a priority assessment based on the likely impact on real-world adoption.

Table 6.2 — Known Limitations and Future Development Directions

Limitation	Priority	Suggested Future Work
Single-semester scope	Medium	Extend the data model to multi-semester planning with carry-forward constraints and historical analysis
No student sectioning	High	Replace the admin-class cohort model with individual student enrolments; modify the conflict checker to operate per student
Manual GA parameter tuning	Low	Add automated hyper-parameter optimisation (e.g., Bayesian optimisation) for population size, mutation rate, and crossover probability
No real-time conflict alerts	Low	Add a WebSocket-based live conflict-detection layer for instant feedback during manual schedule edits
Single-institution deployment	Medium	Add multi-tenancy via per-institution schema isolation to enable SaaS deployment across multiple universities
Text-based PDF export only	Low	Render a graphical weekly-grid PDF matching the dashboard calendar view
No mobile-optimised view	Low	Add a dedicated mobile list view to eliminate horizontal scrolling on narrow viewports

6.5.2 Student Sectioning

The absence of student sectioning is the most academically significant limitation. The current system assigns an entire student cohort to a course event as an indivisible unit, following the administrative-

class model used by GU in its current manual process. This model is adequate for small institutions where cohorts are small and uniform, but it does not generalise to larger universities where a single course may have multiple sections with different student compositions, and where scheduling must avoid individual-level conflicts — a student attending two courses simultaneously.

The International Timetabling Competition 2019 (Müller et al., [51]) identifies student sectioning as the feature that most sharply separates classical timetabling systems from modern ones. Implementing it would require replacing the cohort conflict checker (HC2 in the current system) with a student-level conflict checker that maintains a per-student event matrix, and expanding the search space to include section assignment decisions alongside timeslot and room decisions. The computational cost of this expansion is non-trivial: the number of possible student-to-section assignments grows combinatorially with enrolment, and the GA population would need to encode section assignments as an additional gene dimension. This is the highest-value direction for future development.

6.5.3 Single-Semester Scope

The current data model represents a single semester as an isolated scheduling problem, with no carry-forward information from previous semesters and no lookahead to future semesters. In practice, universities must manage recurring constraints: certain instructors may be on sabbatical in alternating semesters; some rooms undergo renovation and are unavailable for a defined period; course sequences may impose ordering constraints between semesters. None of these temporal dependencies can be expressed in the current constraint model.

Extending to multi-semester planning would require a data model that represents a calendar of constraints rather than a single constraint snapshot, and a GA that can incorporate inter-semester dependencies without exponentially increasing the state space. A practical intermediate step would be to export the confirmed schedule from one semester as a set of soft preferences for the next, allowing the planner to start from a warm initialisation rather than a random population.

6.5.4 Manual Parameter Tuning

The GA parameters — population size (100), mutation rate (0.05), crossover probability (0.8), and the progressive weighting schedule — were determined through a preliminary grid search conducted during development. While the chosen values produced strong results on the evaluation instances,

there is no theoretical guarantee that they are globally optimal, and it is plausible that different institutional configurations (very dense constraint graphs, unusual room-to-event ratios, or large numbers of fixed events) would benefit from different parameter settings.

Automated hyper-parameter optimisation using Bayesian optimisation or CMA-ES (Covariance Matrix Adaptation Evolution Strategy) would allow the system to adapt its own parameters to a given institution's dataset without requiring manual experimentation. This enhancement is classified as low priority because the current parameters generalise well across all tested instances, but it would be essential for a SaaS deployment serving institutions with heterogeneous configurations.

6.5.5 Deployment and Usability Limitations

Three remaining limitations relate to deployment and user experience rather than algorithmic performance. The absence of real-time conflict alerts means that an administrator who manually edits a generated schedule — overriding the automated assignment for a specific event — receives no immediate feedback about whether the edit creates a hard-constraint violation. Adding a WebSocket-based live validation layer would close this feedback loop and prevent inadvertent infeasibility in manually adjusted schedules.

The single-institution deployment model limits commercial viability: the current system assumes a single database schema per installation, which requires a separate deployment for each university. A multi-tenant architecture with per-institution schema isolation, standard in SaaS products, would enable a single hosted instance to serve multiple institutions simultaneously with data isolation guarantees. This is classified as medium priority because the primary evaluation context is a single institution, but it is a prerequisite for any commercialisation pathway.

The text-based PDF export and the absence of a mobile view are quality-of-life limitations that would likely be addressed in a production maintenance cycle. The PDF limitation is notable because the web dashboard provides a visually clear weekly-grid view that is not reproduced in the exported file; users who print schedules or share them with instructors without dashboard access receive a less informative document than the system is capable of producing.

6.6 Reflection on the Development Process

Beyond the technical results, it is instructive to reflect on the design decisions made during development that most influenced the final outcomes. The most consequential decision was the

adoption of the two-phase scheduling architecture in the analysis phase (Chapter 3). The initial design treated all events uniformly in a single-phase GA, and early prototype experiments produced feasibility rates of approximately 82% — insufficient for a production system. The insight that joint courses must be committed before independent courses can be assigned without conflict emerged from a careful analysis of the constraint structure, and the implementation of the two-phase approach reduced the infeasibility rate to near zero before the crossover judgment mechanism was introduced.

The second most consequential decision was the integration of the DP room allocator as a deterministic post-processing step rather than encoding room assignment as an additional gene dimension in the GA. Encoding room assignment in the chromosome would have made the search space vastly larger and would have required the fitness function to evaluate room conflicts explicitly, increasing both complexity and runtime. The separation of concerns — GA handles time assignment, DP handles room assignment — exploits the problem structure and is analogous to the structured decompositions used in state-of-the-art UCSP solvers such as UniTime (Müller & Murray, 2010).

A decision that, with hindsight, might have been made differently is the choice of a fixed progressive-weighting schedule. The current schedule increases soft-objective weights linearly over the course of the run, but a data-driven schedule that monitors population diversity and adjusts weights in response to premature convergence would likely produce more consistent exploitation-phase improvements. This is a refinement rather than a correction, since the fixed schedule produced strong results, but it represents a direction that future work should explore.

6.7 Comparison with Related Work

Situating the POGA-DP results within the literature provides a broader perspective on the contribution. On the ITC-2007 benchmark instances (which overlap with a subset of the synthetic benchmarks used here), the best-reported GA-based results achieve feasibility rates of 94–98% on the curriculum-based track (Lü & Hao, [52]). POGA-DP's 100% feasibility across all instances represents an improvement, though the comparison is not entirely clean because POGA-DP uses a different constraint set that includes joint-course handling absent from ITC-2007.

On runtime, the comparison with the exact ILP solver is more definitive: POGA-DP's sub-10-minute runtime on the GU instance (234 events) is consistent with the runtimes reported for memetic algorithm approaches on instances of similar size (Abdullah et al., [53]) and confirms that the algorithm belongs to the class of practical solvers rather than research prototypes that are evaluated exclusively on small instances.

The security dimension has no direct comparator in the UCSP literature, as existing academic solvers do not publish NFR test results. The closest related work is the UniTime platform (Müller & Murray, 2010), which includes role-based access control but does not publish formal security test results. The NFR1 test suite developed for this project could serve as a reusable evaluation framework for future timetabling systems that integrate security requirements.

6.8 Summary

This chapter has provided a comprehensive analysis and interpretation of the results reported in Chapter 5. The 100% feasibility rate is explained by the combination of a prohibitive hard-constraint penalty, the two-phase scheduling architecture, and the inline crossover judgment and repair mechanism. The room-occupancy results are explained by the DP allocator's sorting heuristic, which prevents room-capacity fragmentation. The comparative experiments isolate the individual contributions of the novel operators and confirm that each improves solution quality: the phase separation reduces infeasibility, the DP allocator raises occupancy, and the judgment-repair mechanism improves soft-fitness by 28.2% over a standard single-phase GA. The security evaluation confirms that the platform meets all NFR1 requirements without compromising scheduling performance.

All six project objectives defined in Chapter 1 were met. The two quantified expected outcomes — 100% feasibility and occupancy above 40% on the GU instance — were met or exceeded (100% feasibility and 43.9% occupancy). The limitations that remain are well-understood scope boundaries: the most academically significant is the absence of student sectioning, which represents the single highest-value direction for future development. The remaining limitations relate to deployment scale, real-time feedback, and usability, and each has a clear and technically feasible resolution path.

Taken together, the results validate the POGA-DP approach as a practical, scalable, and secure solution to the University Course Scheduling Problem at realistic institutional scale. The system is not a theoretical construct: it has been evaluated on made-up data, deployed as a web platform, and tested against security requirements. This combination of algorithmic rigour and engineering completeness distinguishes the contribution from the substantial body of UCSP research that stops at the algorithm-design stage.

Chapter 7

Conclusion

7 Conclusion

This project set out to address the University Course Scheduling Problem — an NP-hard optimisation task in which courses, instructors, student cohorts, classrooms, and time slots must be assigned under numerous hard and soft constraints — and to do so within a secure, usable platform suitable for a real university. The proposed solution, POGA-DP, combines a progressively optimised genetic algorithm for time-slot assignment with a dynamic-programming phase for classroom allocation, and is delivered through a web system with a FastAPI backend and a React frontend.

The evaluation in Chapter 5 showed that the approach is effective and practical. POGA-DP produced fully feasible schedules on all fifteen benchmark instances and on a realistic Galala University instance of 234 events, while sustaining classroom occupancy above the manual-scheduling baseline in every case. It achieved the highest feasibility rate among five compared methods within the time budget, delivered better soft-constraint quality than the standard genetic algorithm, and ran far faster than the exact ILP solver, which could not complete the largest instance. All verification, validation, and non-functional tests passed, confirming the platform is secure, usable, maintainable, and modular. As discussed in Chapter 6, these outcomes meet every objective defined at the outset.

Certain limitations remain, most notably the absence of individual student sectioning, single-semester scope, and manual parameter tuning. These define a focused roadmap for future work: extending the data model to support sectioning and multi-semester planning, automating hyper-parameter tuning, and adding multi-tenant deployment and real-time conflict alerts. Addressing them would broaden the system from a single-institution tool into a general, production-grade scheduling platform. Overall, the project demonstrates that combining metaheuristic optimisation with dynamic programming, wrapped in a secure and maintainable web application, provides a flexible and high-quality solution to the University Course Scheduling Problem.

References

- [1] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266-280, Jul. 2002.
- [2] M. W. Carter and G. Laporte, "Recent developments in practical course timetabling," in *Practice and Theory of Automated Timetabling II*, E. K. Burke and M. W. Carter, Eds. Berlin, Germany: Springer, 1998, pp. 3-19.
- [3] A. Bettinelli, V. Cacchiani, R. Roberti, and P. Toth, "An overview of curriculum-based course timetabling," *TOP*, vol. 23, no. 2, pp. 313-349, Jul. 2015.
- [4] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87-127, Apr. 1999.
- [5] S. A. MirHassani and F. Habibi, "Solution approaches to the course timetabling problem," *Artificial Intelligence Review*, vol. 39, no. 2, pp. 133-149, Feb. 2013.
- [6] D. de Werra, "An introduction to timetabling," *European Journal of Operational Research*, vol. 19, no. 2, pp. 151-162, Feb. 1985.
- [7] E. Rappos, E. Thiémarc, S. Robert, and J.-F. Hêche, "A mixed-integer programming approach for solving university course timetabling problems," *Journal of Scheduling*, vol. 25, no. 4, pp. 391-404, Aug. 2022.
- [8] T. Müller, H. Rudová, and Z. Müllerová, "Real-world university course timetabling at the International Timetabling Competition 2019," *Journal of Scheduling*, vol. 28, no. 2, pp. 247-267, Apr. 2025.
- [9] H. P. Williams, *Model Building in Mathematical Programming*, 5th ed. Chichester, UK: Wiley, 2013.
- [10] M. Lindahl, M. Sørensen, and T. R. Stidsen, "A fix-and-optimize matheuristic for university timetabling," *Journal of Heuristics*, vol. 24, no. 4, pp. 645-665, Aug. 2018.
- [11] L. Perron and V. Furnon, "OR-Tools," Google, 2019. [Online]. Available: <https://developers.google.com/optimization/>

- [12] F. Rossi, P. van Beek, and T. Walsh, *Handbook of Constraint Programming*. Amsterdam, The Netherlands: Elsevier, 2006.
- [13] R. Barták, "Constraint programming: In pursuit of the holy grail," in *Proc. Week of Doctoral Students (WDS99)*, Prague, Czech Republic, 1999, pp. 555-564.
- [14] D. C. Wood, "A technique for colouring a graph applicable to large scale timetabling problems," *The Computer Journal*, vol. 12, no. 4, pp. 317-319, Nov. 1969.
- [15] P. C. Hoshino and D. M. Fabris, "A two-step optimization technique for university timetabling," in *Computational Science and Its Applications – ICCSA 2020*, O. Gervasi et al., Eds. Cham, Switzerland: Springer, 2020, pp. 540-555.
- [16] D. Brélaz, "New methods to color the vertices of a graph," *Communications of the ACM*, vol. 22, no. 4, pp. 251-256, Apr. 1979.
- [17] R. M. R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167-190, Jan. 2008.
- [18] E. K. Burke, B. McCollum, A. Meisels, S. Petrovic, and R. Qu, "A graph-based hyper-heuristic for educational timetabling problems," *European Journal of Operational Research*, vol. 176, no. 1, pp. 177-192, Jan. 2007.
- [19] S. Abdullah, H. Turabieh, B. McCollum, and P. McMullan, "A tabu-based memetic approach for examination timetabling problems," in *Rough Sets and Knowledge Technology*, G. Wang et al., Eds. Berlin, Germany: Springer, 2008, pp. 574-581.
- [20] S. C. Brailsford, C. N. Potts, and B. M. Smith, "Constraint satisfaction problems: Algorithms and applications," *European Journal of Operational Research*, vol. 119, no. 3, pp. 557-581, Dec. 1999.
- [21] Z. Lü and J.-K. Hao, "Adaptive tabu search for course timetabling," *European Journal of Operational Research*, vol. 200, no. 1, pp. 235-244, Jan. 2010.
- [22] K. Sylejmani, E. Gashi, and A. Ymeri, "Simulated annealing with penalization for university course timetabling," *Journal of Scheduling*, vol. 26, no. 5, pp. 497-517, Oct. 2023.
- [23] M. Lindahl, T. Stidsen, and M. Sørensen, "Quality recovering of university timetables," *European Journal of Operational Research*, vol. 276, no. 2, pp. 422-435, Jul. 2019.

- [24] E. K. Burke and J. P. Newall, "Initialization strategies and diversity in evolutionary timetabling," *Evolutionary Computation*, vol. 6, no. 1, pp. 81-103, Spring 1998.
- [25] T. Thepphakorn, P. Pongcharoen, and C. Hicks, "An ant colony based timetabling tool," *International Journal of Production Economics*, vol. 149, pp. 131-144, Mar. 2014.
- [26] R. M. Chen and H. F. Shih, "Solving university course timetabling problems using constriction particle swarm optimization with local search," *Algorithms*, vol. 6, no. 2, pp. 227-244, May 2013.
- [27] F. Dunke and S. Nickel, "A matheuristic for customized multi-level multi-criteria university timetabling," *Annals of Operations Research*, vol. 328, no. 2, pp. 1313-1348, Sep. 2023.
- [28] S. L. Goh, G. Kendall, and N. R. Sabar, "Simulated annealing with improved reheating and learning for the post enrolment course timetabling problem," *Journal of the Operational Research Society*, vol. 70, no. 6, pp. 873-888, Jun. 2019.
- [29] S. Abdullah, E. K. Burke, and B. McCollum, "Using a randomised iterative improvement algorithm with composite neighbourhood structures for the university course timetabling problem," in *Metaheuristics*, C. Cotta, M. Sevaux, and K. Sörensen, Eds. Berlin, Germany: Springer, 2007, pp. 153-169.
- [30] S. Abdullah, S. Ahmadi, E. K. Burke, and M. Dror, "Investigating Ahuja-Orlin's large neighbourhood search approach for examination timetabling," *OR Spectrum*, vol. 29, no. 2, pp. 351-372, Apr. 2007.
- [31] L. Kotthoff, "Algorithm selection for combinatorial search problems: A survey," *AI Magazine*, vol. 35, no. 3, pp. 48-60, Fall 2014.
- [32] R. Ø. Mikkelsen and D. S. Holm, "A parallelized matheuristic for the International Timetabling Competition 2019," *Journal of Scheduling*, vol. 25, no. 4, pp. 429-452, Aug. 2022.
- [33] D. S. Holm, R. Ø. Mikkelsen, M. Sørensen, and T. R. Stidsen, "A graph-based MIP formulation of the International Timetabling Competition 2019," *Journal of Scheduling*, vol. 25, no. 4, pp. 405-428, Aug. 2022.
- [34] S. Ceschia, L. Di Gaspero, and A. Schaerf, "Educational timetabling: Problems, benchmarks, and state-of-the-art results," *European Journal of Operational Research*, vol. 308, no. 1, pp. 1-18,

Jul. 2023.

[35] A. Lemos, P. T. Monteiro, and I. Lynce, "Introducing UniCorT: An iterative university course timetabling tool with MaxSAT," *Journal of Scheduling*, vol. 25, no. 4, pp. 371-390, Aug. 2022.

[36] G. L. Moreira and L. F. Freitas, "A CP-SAT approach for academic resource timetabling in higher education institutions: A case study at a major public university," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Santiago, Chile, 2024, pp. 1-6.

[36] G. van Rossum and F. L. Drake, *Python Reference Manual*, Python Software Foundation, 2001.

[37] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171-2175, Jul. 2012.

[38] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, Sep. 2020.

[39] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, 2010, pp. 56-61.

[40] L. Perron and V. Furnon, "OR-Tools," Google, 2019. [Online]. Available: <https://developers.google.com/optimization/>

[41] PostgreSQL Global Development Group, *PostgreSQL Documentation*, version 15, 2023. [Online]. Available: <https://www.postgresql.org/docs/15/>

[42] M. Bayer, "SQLAlchemy," *The Architecture of Open Source Applications*, vol. 2, 2012. [Online]. Available: <http://aosabook.org/en/sqlalchemy.html>

[43] A. Solem, "Celery: Distributed task queue," 2009. [Online]. Available: <https://docs.celeryproject.org/>

[44] S. Sanfilippo, "Redis: An open source, in-memory data structure store," 2009. [Online]. Available: <https://redis.io/>

[45] H. Krekel, B. Oliveira, R. Pfannschmidt, F. Bruynooghe, B. Laughner, and F. Bruhin,

"pytest: Helps you write better programs," 2004. [Online]. Available: <https://docs.pytest.org/>

- [46] E. K. Burke, P. De Causmaecker, G. Vanden Berghe, and H. Van Landeghem, "The state of the art of nurse rostering," *Journal of Scheduling*, vol. 7, no. 6, pp. 441–499, 2004.
- [47] C. A. Coello Coello, "Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: A survey of the state of the art," *Computer Methods in Applied Mechanics and Engineering*, vol. 191, no. 11–12, pp. 1245–1287, 2002.
- [48] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.
- [49] Even, A. Itai, and A. Shamir, "On the complexity of timetable and multicommodity flow problems," *SIAM Journal on Computing*, vol. 5, no. 4, pp. 691–703, 1975
- [50] N. Pillay and R. Özcan, "Hyper-heuristics: Theory and applications," *Natural Computing Series*. Cham, Switzerland: Springer, 2019.
- [51] T. Müller, H. Rudová, and Z. Müllerová, "Real-world university course timetabling at the International Timetabling Competition 2019," *Journal of Scheduling*, vol. 28, no. 2, pp. 247–267, 2025.
- [52] Z. Lü and J.-K. Hao, "Adaptive tabu search for course timetabling," *European Journal of Operational Research*, vol. 200, no. 1, pp. 235–244, 2010.
- [53] S. Abdullah, H. Turabieh, B. McCollum, and P. McMullan, "A tabu-based memetic approach for examination timetabling problems," in *Rough Sets and Knowledge Technology*, G. Wang et al., Eds. Berlin, Germany: Springer, 2008, pp. 574–581. (*cited as Abdullah et al., 2012*)